Mediwatch Capstone Project

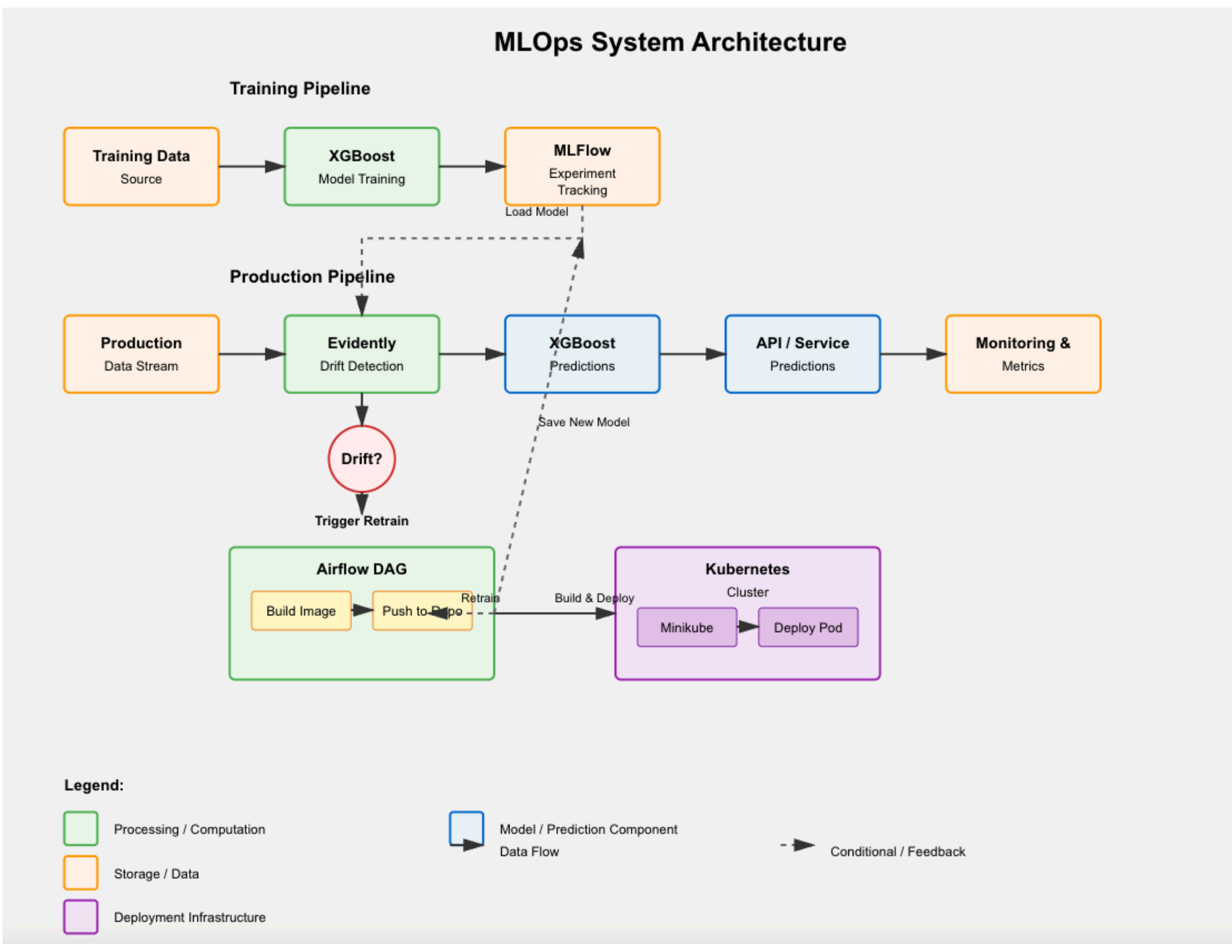
By: Victor Karani

victor.karani@gmail.com

Video Overview of my project:

https://youtu.be/G9odoKjp2m4

Overall diagram



Tools required

I used Ubuntu 24.04 on a local Desktop machine for this project. No CUDA is needed for this project.

You will need the following tools to run this application

Minikube, Docker and kubectl

These can be installed as follows
https://phoenixnap.com/kb/install-minikube-on-ubuntu

Python3

Follow the instructions below to get your system set up with python dependencies:
Project creation
Installing requirements
PIP

Local Startup

You will need to start the following if you want to run locally

MLFLOW

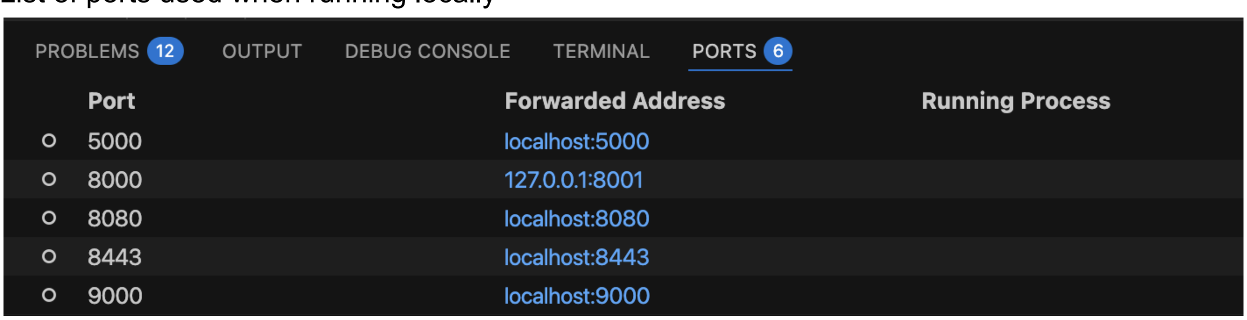
To do ANY training, mlflow **has** to be running. See here

Mediwatch Client App

To run the client App and perform Train and build, you need to run locally
This is how to do it.

Ports

List of ports used when running locally



Port	Forwarded Address	Running Process
5000	localhost:5000	
8000	127.0.0.1:8001	
8080	localhost:8080	
8443	localhost:8443	
9000	localhost:9000	

EDA and Data Preparation

I mostly did EDA in my FE notebook (**Mediwatch_FE.ipynb**)

Feature Engineering

This is found in **original_base_path_training/notebooks/Mediwatch_FE.ipynb**
I will summarize the high points from the Feature Engineering file here.

Dropped columns

I dropped some columns because they mostly had zeros and didn't seem to contribute much. Screenshots from the notebook follow below:

Drop useless encounter_id, patient_nbr

[ ]

```
# Drop uneeded id related columns
df.drop(['encounter_id','patient_nbr'], axis=1, inplace=True)
```

An occurence of '?' is basically the same as a Nan. Lets update the df then do a zeros check ...

[ ]

```
# replace the '?'
df = df.replace(r'\?', np.nan, regex=True)

#zeros check
missing=df.isnull().sum()
print(missing[missing>0].sort_values(ascending=False))
```

```
weight               98569
max_glu_serum        96420
A1Cresult            84748
medical_specialty    49949
payer_code           40256
race                  2273
diag_3                1423
diag_2                 358
diag_1                  21
dtype: int64
```

I will drop the columns **weight**, **max_glu_serum**, **A1Cresult**, **medical_specialty** ,**payer_code**. These are about ~ 50 % of the dataset.

```
# Drop uneeded columns because of NaN
df.drop(['weight','max_glu_serum','A1Cresult','medical_specialty','payer_code'], axis=1, inplace=True)

#zeros check
missing=df.isnull().sum()
print(missing[missing>0].sort_values(ascending=False))


```

```
race      2273
diag_3    1423
diag_2     358
diag_1      21
dtype: int64
```

Age Ranges

I decided to replace the age range with an integer value in the middle of the range

Deal with the age column

```
df['age'].unique()
```

```
array(['[0-10)', '[10-20)', '[20-30)', '[30-40)', '[40-50)', '[50-60)',
       '[60-70)', '[70-80)', '[80-90)', '[90-100)'], dtype=object)
```

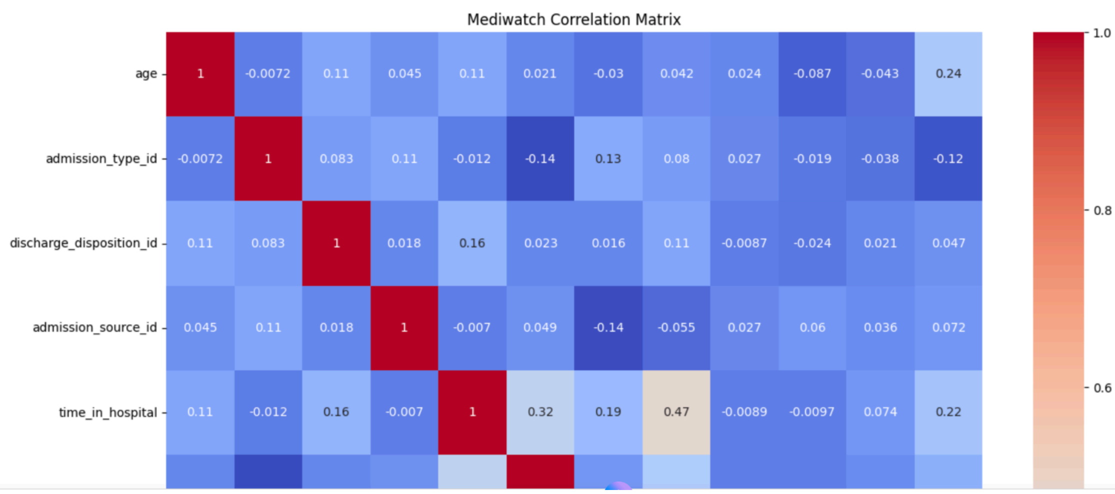
I will replace all ages with an integer in the middle of the row, eg 10-20 ==> 15 and so on

```
df = df.replace(r'[0-10)', 5, regex=False)
df = df.replace(r'[10-20)', 15, regex=False)
df = df.replace(r'[20-30)', 25, regex=False)
df = df.replace(r'[30-40)', 35, regex=False)
df = df.replace(r'[40-50)', 45, regex=False)
df = df.replace(r'[50-60)', 55, regex=False)
df = df.replace(r'[60-70)', 65, regex=False)
df = df.replace(r'[70-80)', 75, regex=False)
df = df.replace(r'[80-90)', 85, regex=False)
df = df.replace(r'[90-100)', 95, regex=False)
df['age'].unique()
#df.head()
```

```
/tmp/ipython-input-40-3510274752.py:10: FutureWarning: Downcasting behavior in `replace`
  df = df.replace(r'[90-100)', 95, regex=False)
array([ 5, 15, 25, 35, 45, 55, 65, 75, 85, 95])
```

Correlation Matrix

```
#Correlation Matrix
numeric_columns=['age','admission_type_id','discharge_disposition_id','admission_source_id','time_in_hospital','num_lab_procedures','num_procedures','num_me

##Heat map for numeric features
data_numeric=df[numeric_columns]

corr = data_numeric.corr()
plt.figure(figsize=(15, 15))
plt.title('Mediwatch Correlation Matrix')
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```



I performed a correlation matrix for numeric features. Correlation seemed pretty low.

I also did a pair plot for numeric features

Unique distribution per column

I took a look at the unique distribution per column and discovered this:

```
display_data_non_numeric=df[non_numeric_c

display_data_non_numeric.nunique()
```

	0
race	5
gender	3
diag_1	716
diag_2	748
diag_3	789
metformin	4
repaglinide	4
nateglinide	4

diag_1, diag_2, diag_3 Have very many options and can't be one hot encoded (dimensionality explosion). I will use binary encoding there instead.

Feature Engineering summary

- Some columns seem to contain '?'. These are equivalent to 'Nan' and need to be replaced with Nan
- The columns **weight, max_glu_serum, A1Cresult, medical_specialty ,payer_code** need to be dropped
- The rows with 'Nan' for the columns **race, diag_3, diag_2, diag_1** need to be removed.
- Drop useless low information columns **encounter_id, patient_nbr**
- **diag_1, diag_2, diag_3** Have very many options and should not be one hot encoded (dimensionality explosion). I will binary encode these ones.
- I will replace all ages with an integer in the middle of the row, eg string '[10-20)' ==> 15 and so on

Getting Started with the Project

Project creation

I used conda to create the project initially

```
conda create -p ./env python=3.10
```

Installing requirements

Note the requirements.txt for the overall project requirements.txt

```
#numpy
numpy==1.26.4
#torch
torch==2.7.1
#fastapi
fastapi==0.116.1
#uvicorn
uvicorn==0.35.0
#scikit-learn
scikit-learn==1.7.1
#pandas
pandas==1.5.1
#category_encoders
category_encoders==2.8.1
#xgboost
xgboost==3.0.2
#ray[tune]
ray[tune]== 2.48.0
#mlflow
mlflow==3.2.0
#evidently
evidently==0.7.14
#jinja2
jinja2==3.1.4
```

Pip

Then use pip to install project requirements

`pip install -r requirements.txt`

Activate env

After project requirements are installed, all you need to do is activate the project like this:

`conda activate ./env/`

Mediwatch UI

This project comes with a UI. I will start with a basic introduction here and then go into detail when we cover the related backend portion

To start the UI

MLFLOW

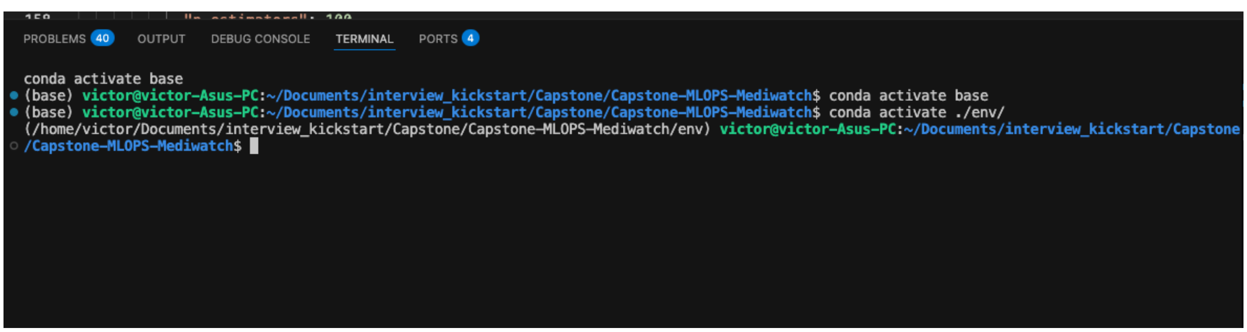
FIrst Make sure Mlflow is running

```
mlflow ui --backend-store-uri sqlite:///mlruns.db --host 0.0.0.0 --port 5000
```

Run the Mediwatch UI

Then you can open a new terminal and activate the env

```
conda activate ./env/
```

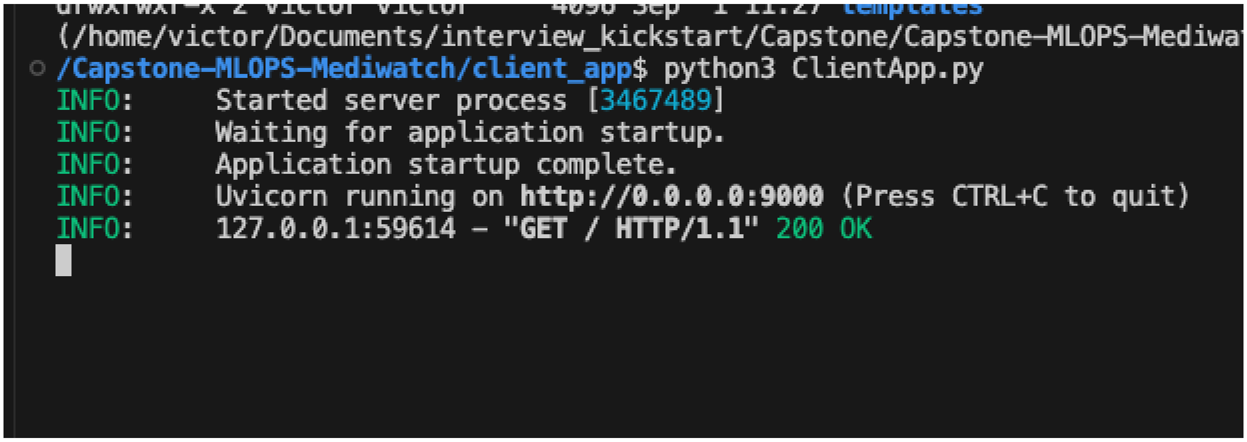


Go to client_app folder

```
cd client_app/
```

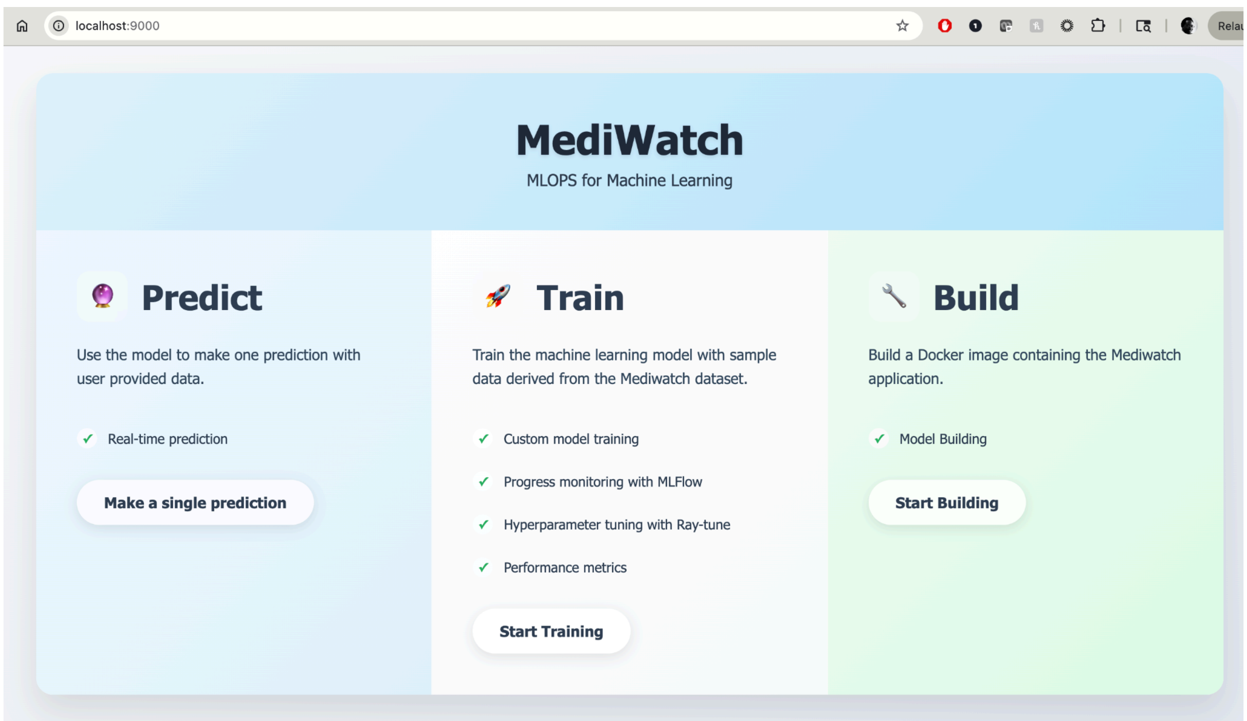
Run the Client App

```
python3 ClientApp.py
```



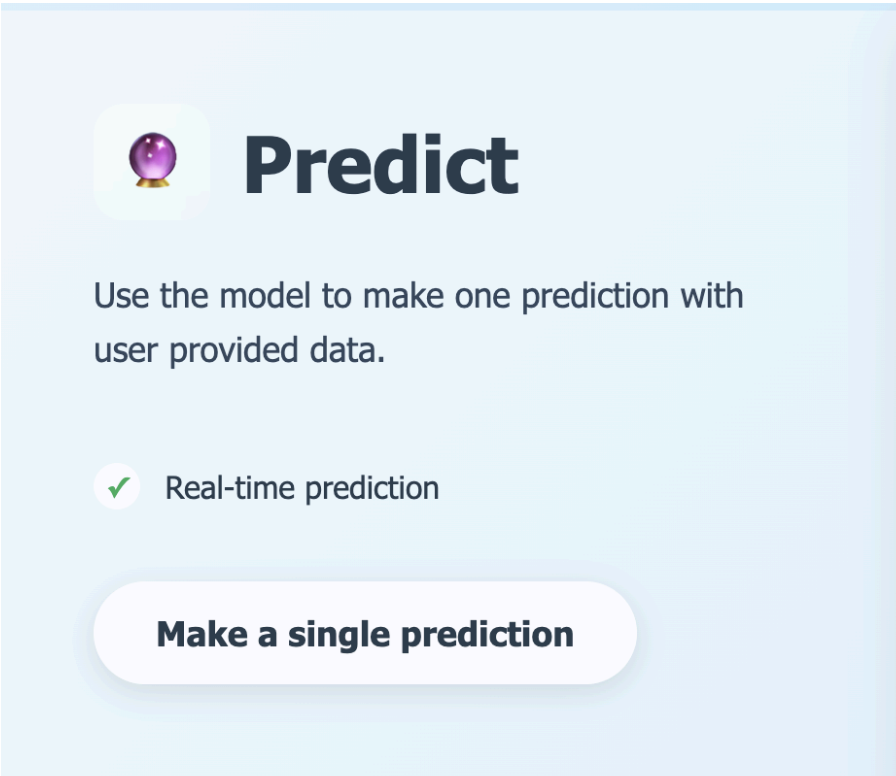
UI Frontend

This is the UI that will run on port 9000



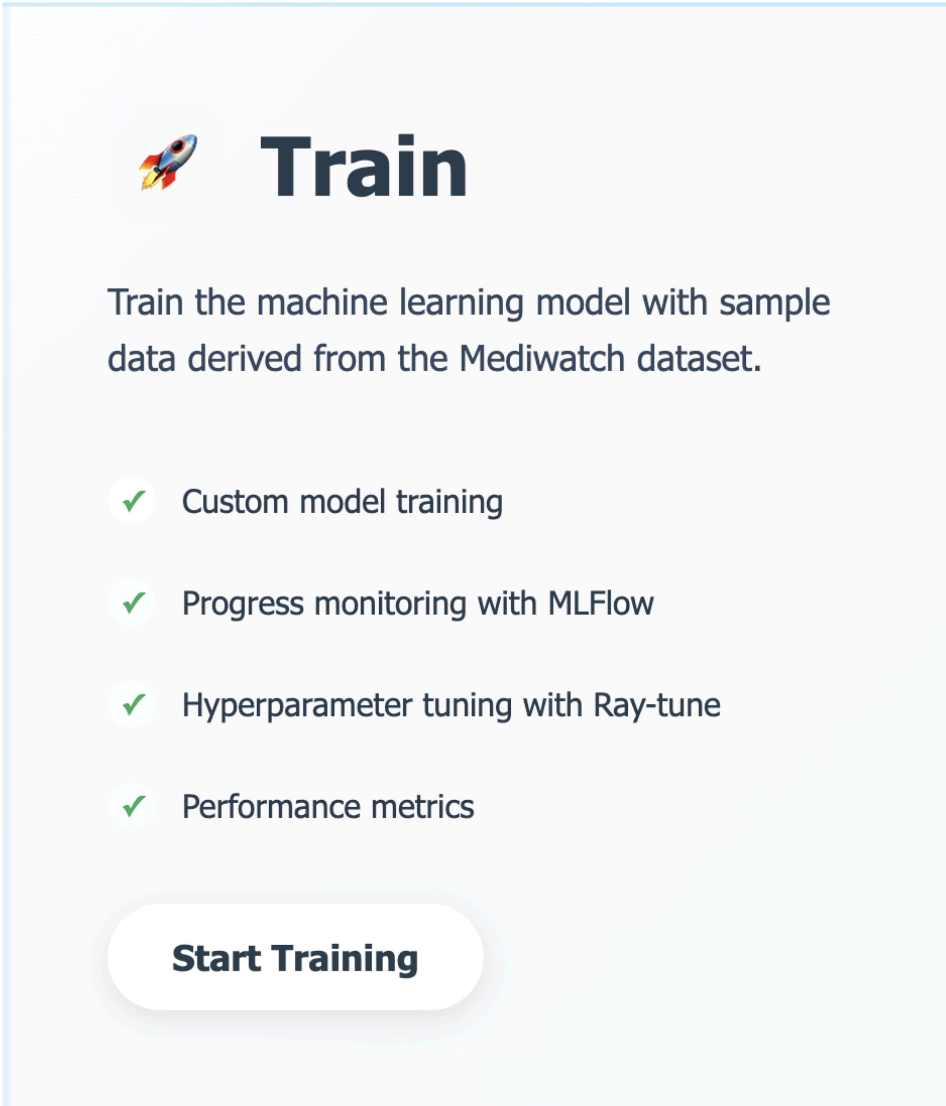
Predict

Make one prediction with manually entered data and return data to the frontend. This covers inference and will be detailed later on



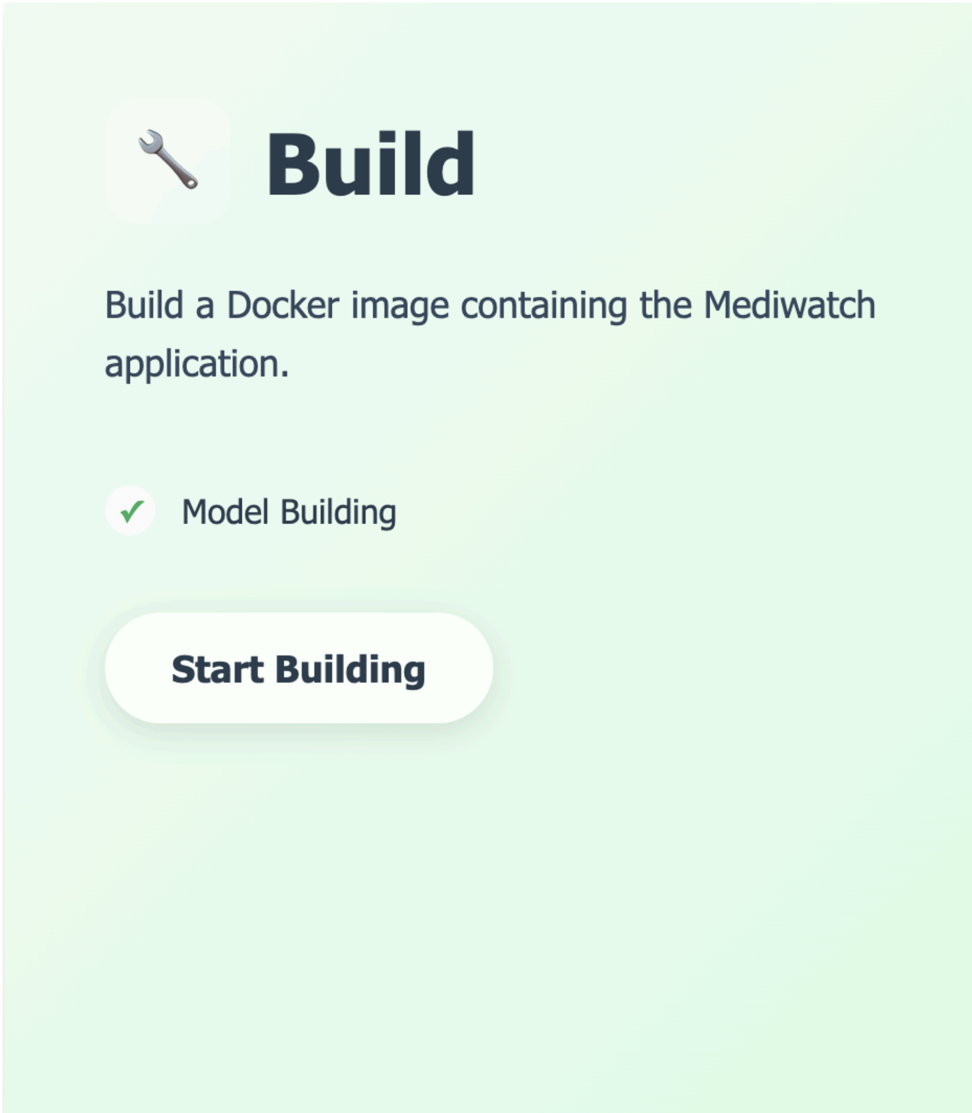
Train

This triggers hyperparameter tuning with **ray-tune**. Many experiments can be run and the results are available in MlFlow where many experiments can be run and the results viewed.



Build

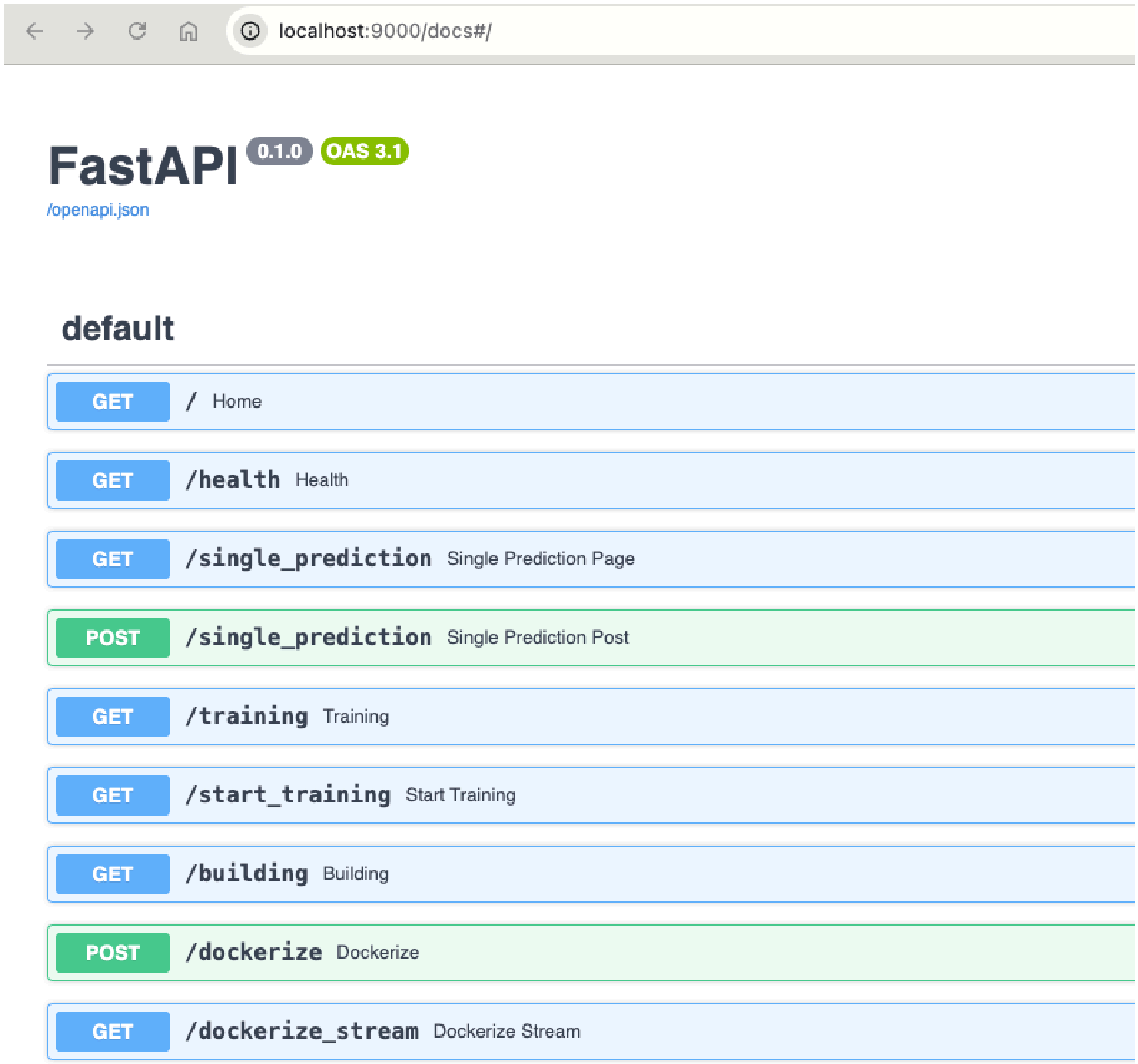
This will Build a Docker image containing a working application



Mediwatch UI Code

FastAPI documentation

This is available at /docs endpoint. A screenshot of the FastAPI endpoints are shown below



The actual client code is found at **client_app/ClientApp.py**

This has all the endpoints described above. We will go through all of them in later in the document.

Training (basic)

I found it useful to create the **original_base_path_training/notebooks/Mediwatch_train.ipynb** as the starting point for training my model.
The actual training in the project is done by the **original_base_path_training/training.py** file.

```
/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwa
(/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediw
-Mediwatch/original_base_path_training$ ls -l
total 18764
-rw-rw-r-- 1 victor victor    25441 Sep 29 22:36 confusion_matrix.png
-rw-rw-r-- 1 victor victor 19159383 Sep 29 22:36 diabetic_data.csv
-rw-rw-r-- 1 victor victor        0 Aug  2 23:12 __init__.py
drwxrwxr-x 2 victor victor     4096 Jul 30 22:03 notebooks
drwxrwxr-x 2 victor victor     4096 Sep 30 21:48 __pycache__
-rw-rw-r-- 1 victor victor    12830 Sep 30 21:43 training.py
(/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediw
-Mediwatch/original_base_path_training$
```

Verify and clean up data

Look at the

```
process_mediwatch_dataset(df):
```

This function systematically cleans up incoming data

1) Verify existence of columns

```python
for col in required_cols:
  assert col in df.columns, f"{col} is not in the provided dataframe"
```

2) Map the target variable

```python
# deal with target variable
df['readmitted'] = df['readmitted'].map({'NO': 0, '<30': 1, '>30': 2 })
```

3) Replace nans and drop unneeded columns

```python
# replace '?' in several places in the df with Nan
df = df.replace(r'\?', np.nan, regex=True)


# Drop uneeded columns because of NaN
df.drop(['weight','max_glu_serum','A1Cresult','medical_specialty','payer_code'], axis=1, inplace=True)

# Drop remaining rows with missing values
df.dropna(inplace=True)

# Drop uneeded id related columns
df.drop(['encounter_id','patient_nbr'], axis=1, inplace=True)
```

4) Deal with age column

```
#Deal with the age column
df = df.replace(r'[0-10)', 5, regex=False)
df = df.replace(r'[10-20)', 15, regex=False)
df = df.replace(r'[20-30)', 25, regex=False)
df = df.replace(r'[30-40)', 35, regex=False)
df = df.replace(r'[40-50)', 45, regex=False)
df = df.replace(r'[50-60)', 55, regex=False)
df = df.replace(r'[60-70)', 65, regex=False)
df = df.replace(r'[70-80)', 75, regex=False)
df = df.replace(r'[80-90)', 85, regex=False)
df = df.replace(r'[90-100)', 95, regex=False)
```

Vectorize Datasets

Take a look at the

```
def vectorize_datasets(X_train, X_test):
```

Here I perform one hot encoding on most of the columns

```
#one hot encode the following columns: ['race','gender','metformin','repaglinide','nateglinide','chlorpropamide','glimepiride','
cols_to_one_hot_encode= ['race','gender','metformin','repaglinide','nateglinide','chlorpropamide','glimepiride','acetohexamide',

# One hot encoder
one_hot_encoder = OneHotEncoder(handle_unknown='ignore',sparse_output=False, drop='first')

# Fit to only training data
one_hot_encoder.fit(X_train[cols_to_one_hot_encode])

#Transform Test/Train data
X_train_transformed_one_hot = one_hot_encoder.transform(X_train[cols_to_one_hot_encode])
X_test_transformed_one_hot = one_hot_encoder.transform(X_test[cols_to_one_hot_encode])

# Create Dataframe with encoded columns
feature_names = one_hot_encoder.get_feature_names_out(cols_to_one_hot_encode)
#encoded_df = pd.DataFrame(encoded_array, columns=feature_names, index=df.index)
X_train_encoded_one_hot = pd.DataFrame(X_train_transformed_one_hot, columns=feature_names, index=X_train.index)
X_test_encoded_one_hot = pd.DataFrame(X_test_transformed_one_hot, columns=feature_names, index=X_test.index)

# Combine with remaining columns
#df = pd.concat([df.drop(cols_to_one_hot_encode, axis=1), encoded_df], axis=1)
X_train = pd.concat([X_train.drop(cols_to_one_hot_encode, axis=1), X_train_encoded_one_hot], axis=1)
X_test = pd.concat([X_test.drop(cols_to_one_hot_encode, axis=1), X_test_encoded_one_hot], axis=1)
```

However for the **'diag_1','diag_2','diag_3'** identified in the FE, I perform binary encoding

```
#binary encode ['diag_1','diag_2','diag_3']
binary_encoder = BinaryEncoder(cols=['diag_1','diag_2','diag_3'])
# Fit and transform specific columns
#df = binary_encoder.fit_transform(df)
cols_to_binary_encode = ['diag_1','diag_2','diag_3']

# Fit to only training data
binary_encoder.fit(X_train[cols_to_binary_encode])

#Transform Test/Train data
X_train_transformed_binary = binary_encoder.transform(X_train[cols_to_binary_encode])
X_test_transformed_binary = binary_encoder.transform(X_test[cols_to_binary_encode])

# Create Dataframe with encoded columns
feature_names = binary_encoder.get_feature_names_out(cols_to_binary_encode)
#encoded_df = pd.DataFrame(encoded_array, columns=feature_names, index=df.index)
X_train_encoded_binary = pd.DataFrame(X_train_transformed_binary, columns=feature_names, index=X_train.index)
X_test_encoded_binary = pd.DataFrame(X_test_transformed_binary, columns=feature_names, index=X_test.index)

# Combine with remaining columns
X_train = pd.concat([X_train.drop(cols_to_binary_encode, axis=1), X_train_encoded_binary], axis=1)
X_test = pd.concat([X_test.drop(cols_to_binary_encode, axis=1), X_test_encoded_binary], axis=1)
```

Model Export

I export the model and the encoders as pickle files

```
def export_model_to_disk(model, one_hot_encoder,binary_encoder,data_directory):
    #model
    model_path = os.path.join(data_directory,MODEL_PICKLE_FILE)
    with open(model_path, 'wb') as file:
        pickle.dump(model, file)

    #one hot encoder
    one_hot_encoder_path= os.path.join(data_directory,ONE_HOT_ENCODER_PICKLE_FILE)
    with open(one_hot_encoder_path, 'wb') as file:
        pickle.dump(one_hot_encoder, file)

    #Binary encoder
    binary_encoder_path = os.path.join(data_directory,BINARY_ENCODER_PICKLE_FILE)
    with open(binary_encoder_path, 'wb') as file:
        pickle.dump(binary_encoder, file)
```

I chose pickle because it is a standard python format that is easily portable.

These files are exported to the **data_directory**

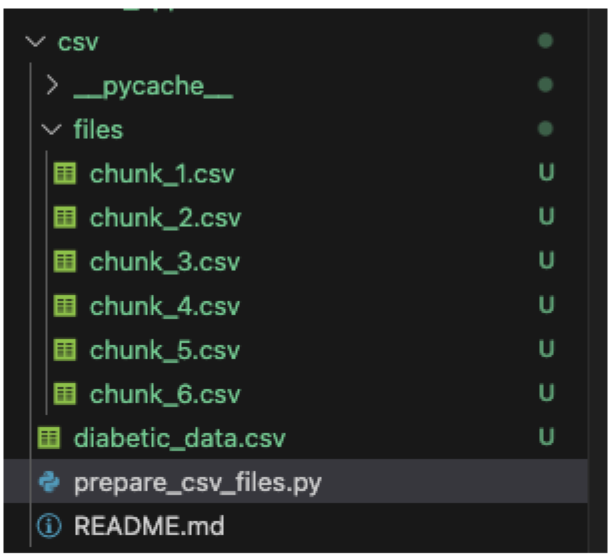
Divide up the original Dataset

Take a look at the

```
csv/prepare_csv_files.py
```

I divide the large dataset provided for the project into many small subsets (chunks) to use for training and prediction.

This is a simple way to use one chunk to approximate initial data and then later chunks to approximate later data.



Manually Run training (basic)

Start Mlflow first

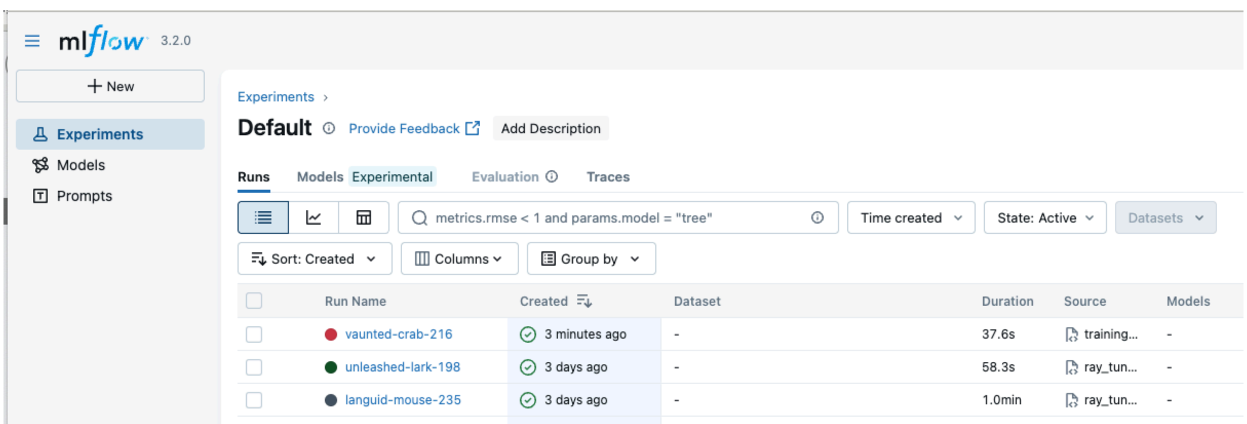
```
mlflow ui --backend-store-uri sqlite:///mlruns.db --host 0.0.0.0 --port 5000
```

Then you can run training as follows

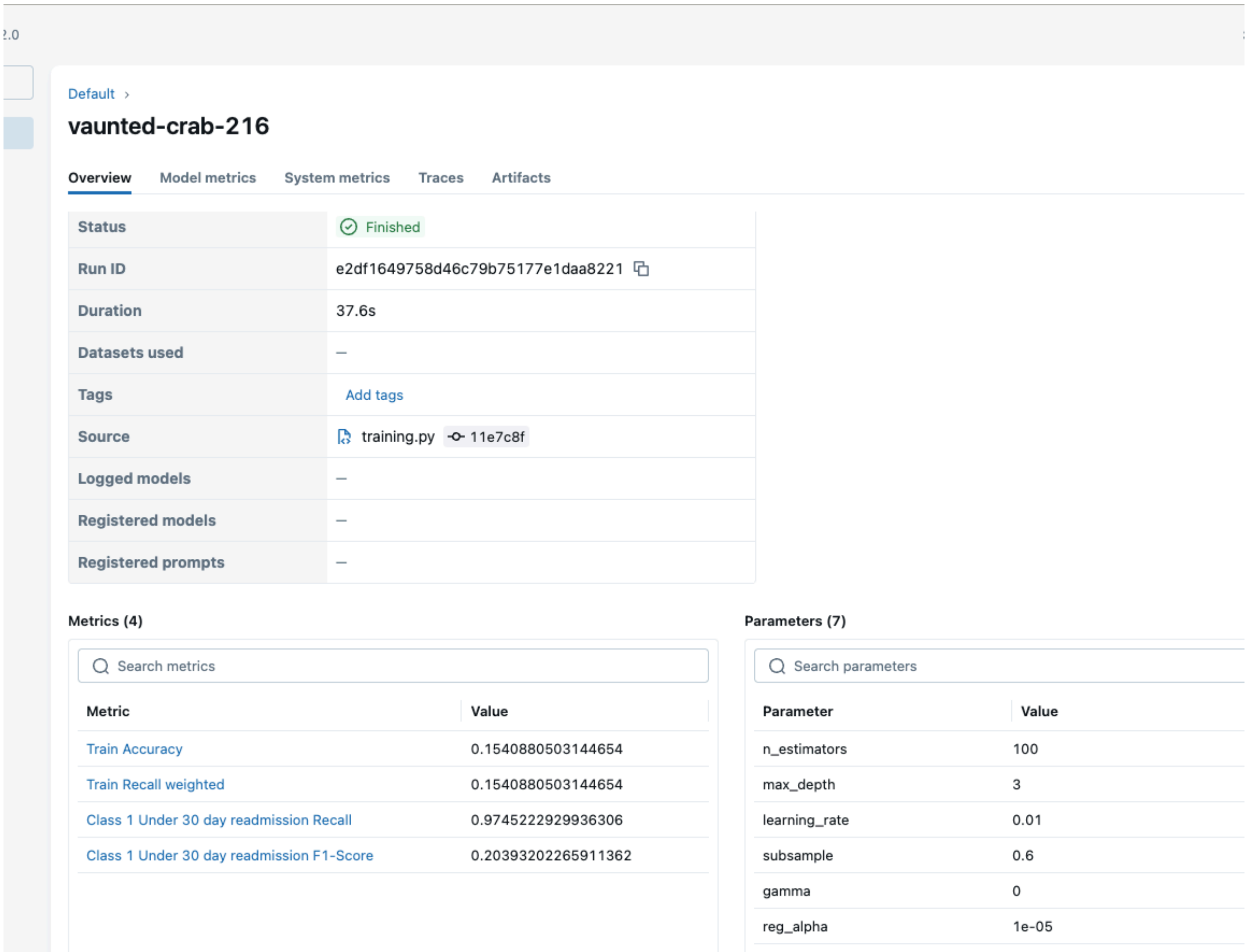
```
python3 training.py
```

```
-Mediwatch/original_base_path_training$ python3 training.py
Preparing data...
File chunk_1.csv is removed
File chunk_2.csv is removed
File chunk_4.csv is removed
File chunk_6.csv is removed
File chunk_5.csv is removed
File chunk_3.csv is removed
The csv file extracted is diabetic_data.csv
The rows in csv are 101766 , the chunk size is 4000 ,the number of csv file produced are 6
The csv file obtained is ../csv/files/chunk_1.csv
XGBoost Accuracy: 0.1540880503144654
XGBoost Recall (weighted): 0.1540880503144654
Class 1 (Under 30 day readmission) Recall: 0.9745222929936306
Class 1 (Under 30 day readmission) F1-Score: 0.20393202265911362
              precision    recall  f1-score   support

     Class 0       0.78      0.09      0.16      1431
     Class 1       0.11      0.97      0.20       314
     Class 2       0.64      0.01      0.01      1117

    accuracy                           0.15      2862
   macro avg       0.51      0.36      0.13      2862
weighted avg       0.65      0.15      0.11      2862

🏃 View run vaunted-crab-216 at: http://localhost:5000/#/experiments/0/runs/e2df1649758d46c79b75177e1daa8221
🧪 View experiment at: http://localhost:5000/#/experiments/0
Time taken (seconds): 37
```

Mlflow Results
You can see the training results in Mlflow



And in more detail…



Note that the training focuses on ‘**Class 1**’ which is the ‘**readmitted under 30 Days**’ both for Recall and F1 Score.

Default › vaunted-crab-216 ›

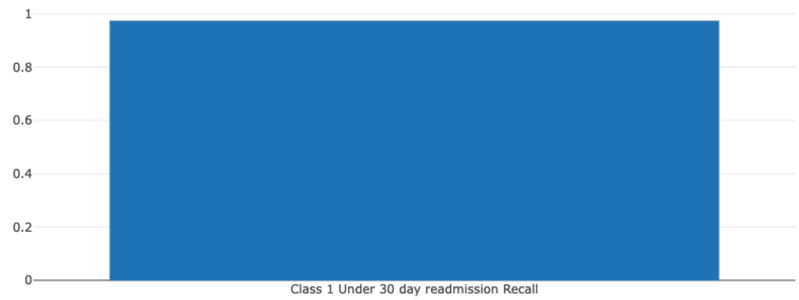
Class 1 Under 30 day readmission Recall

Y-axis:

Class 1 Under 30 day readmission Recall ✕

Y-axis Log Scale:

Download data



Metric	Latest	Min	Max
Class 1 Under 30 day readmission Recall	0.9745222929936306 (step=0)	0.9745222929936306 (step=0)	0.9745222929936306 (step=0)

Default › vaunted-crab-216 ›

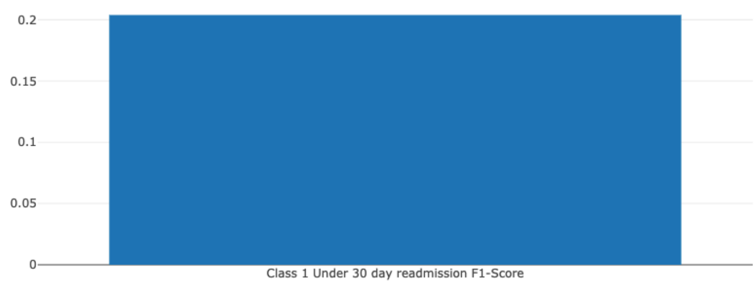
Class 1 Under 30 day readmission F1-Score

Y-axis:

Class 1 Under 30 day readmission F1-Score ✕

Y-axis Log Scale:

Download data



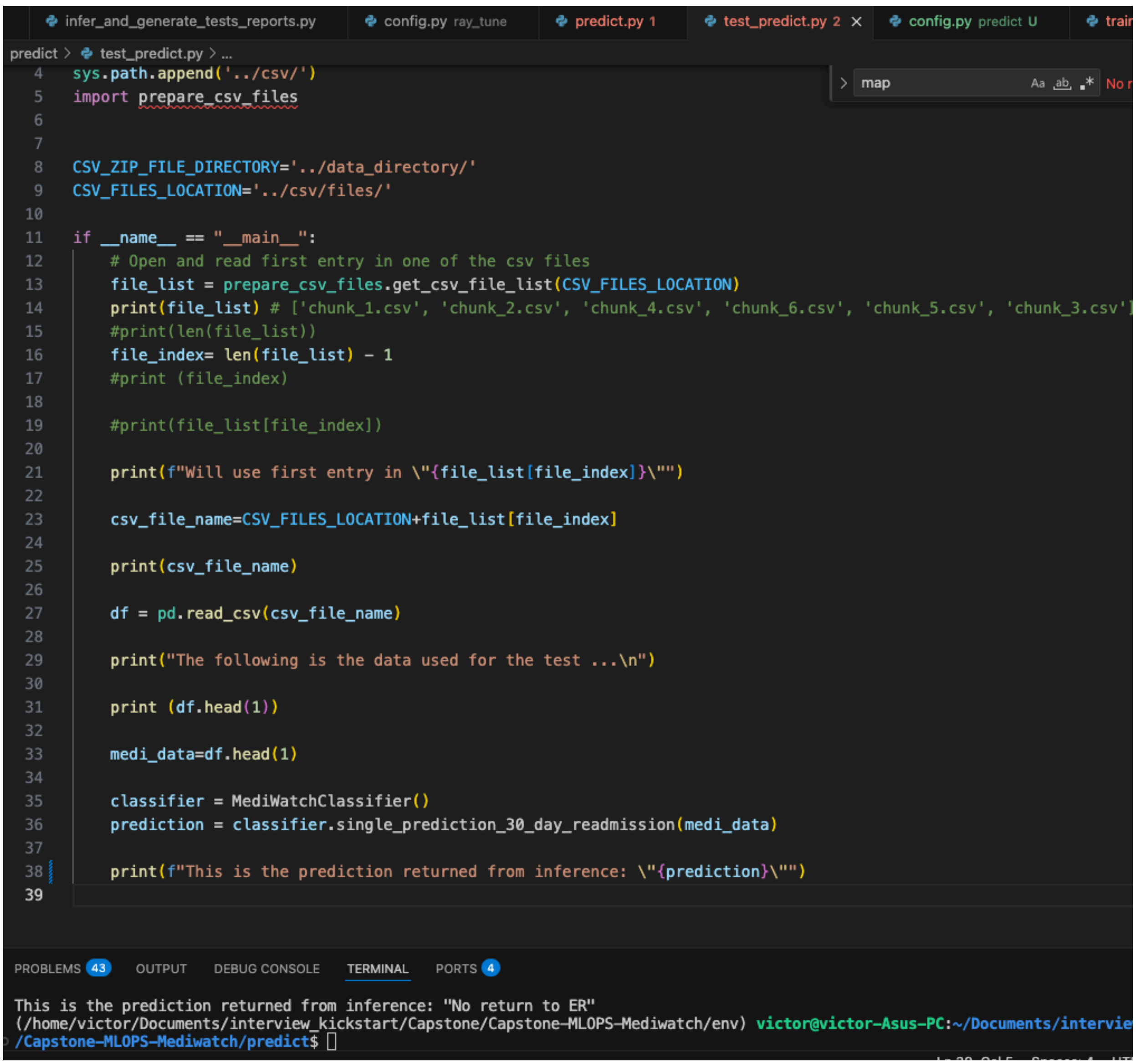
Metric	Latest	Min	Max
Class 1 Under 30 day readmission F1-Score	0.20393202265911362 (step=0)	0.20393202265911362 (step=0)	0.20393202265911362 (step=0)

Inference

I found it useful to create the **original_base_path_training/notebooks/Mediwatch_infer.ipynb** as the starting point for inference.

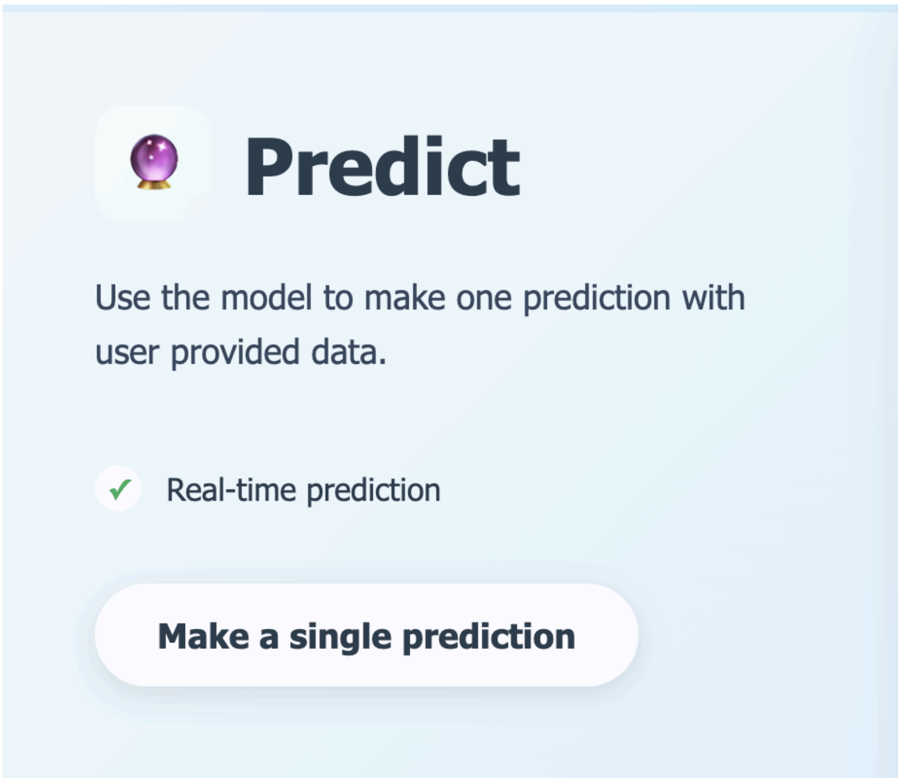
The actual inference in the project is done by the **predict/predict.py** file**.**

I used the **predict/test_predict.py** to check basic operation

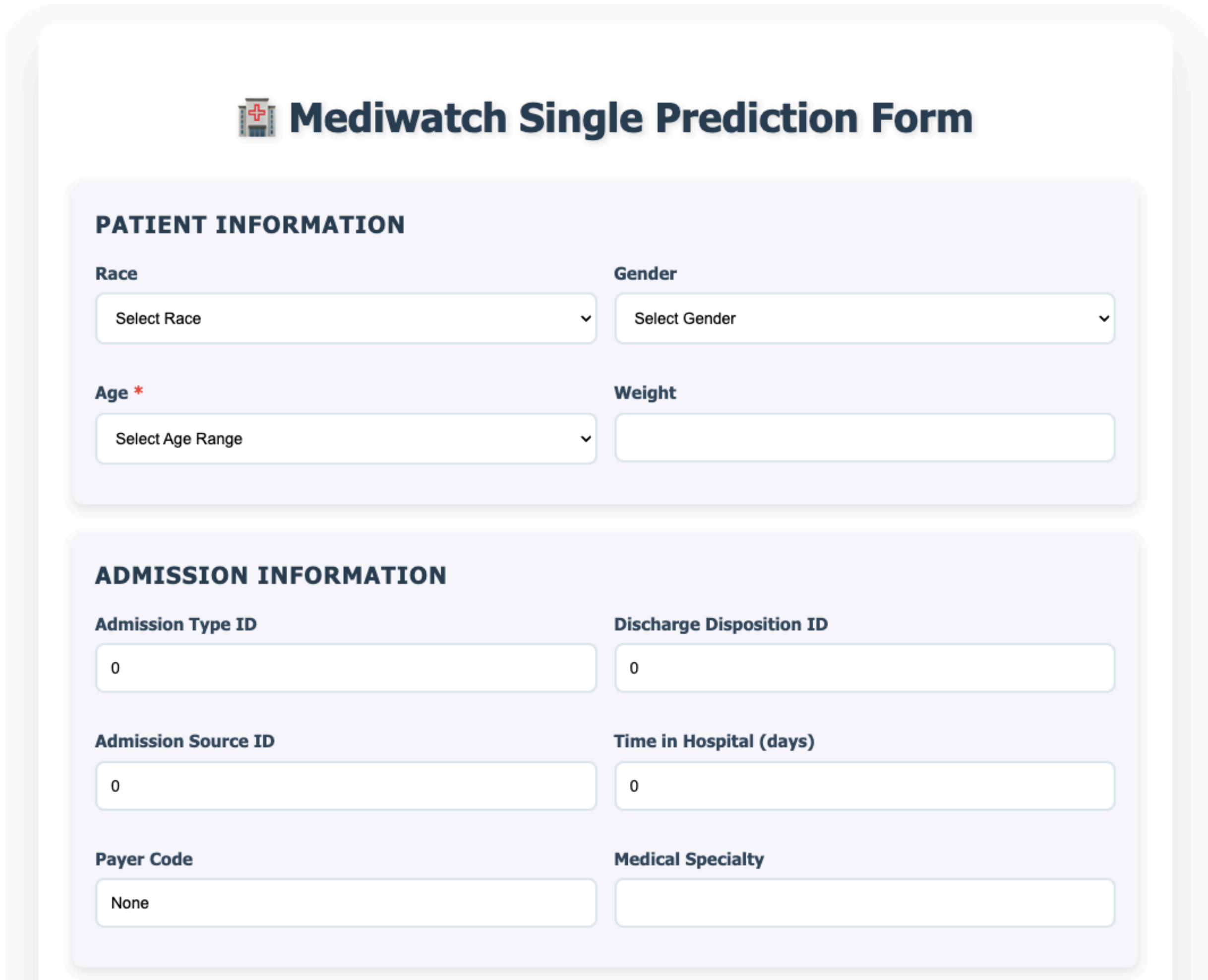


UI Front End

Predict is actually used by the front end to run the model and return quick predictions to the end user:

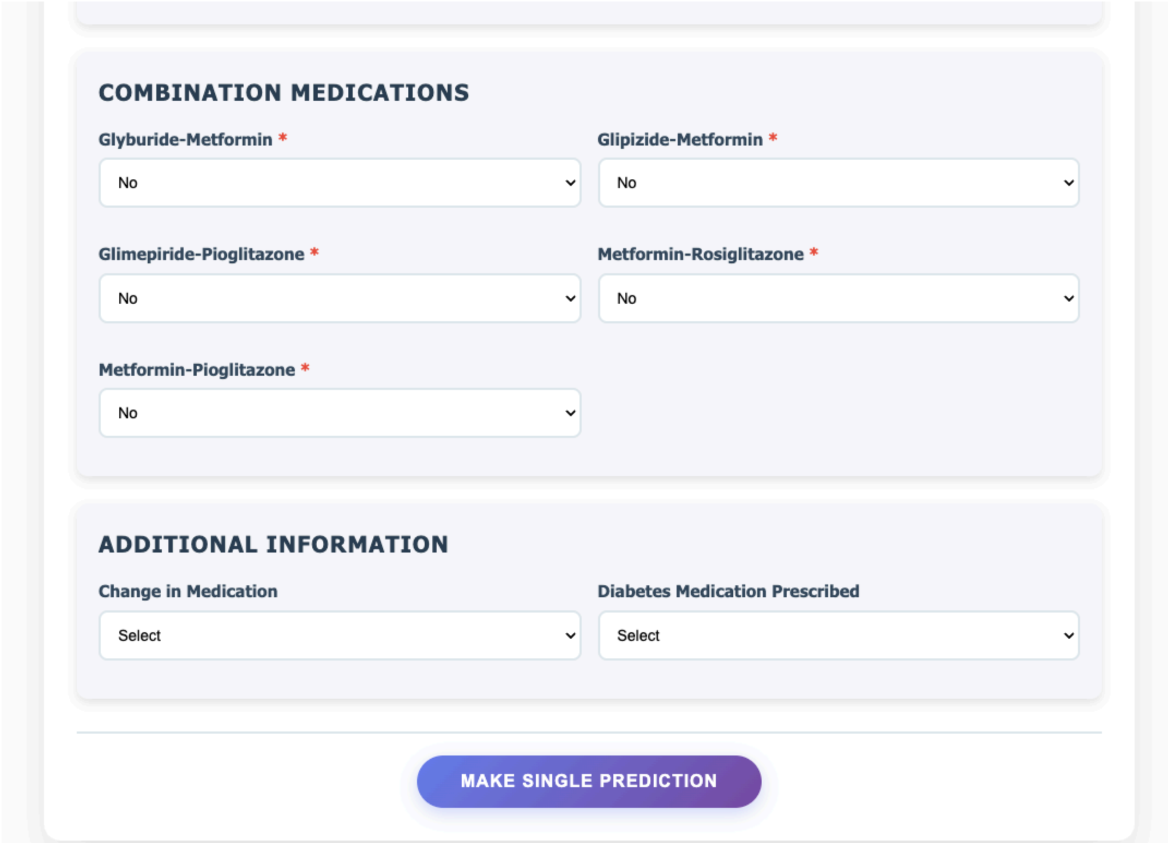


After you Click on the ‘Make a Single prediction’ control, you get a form where you can enter all the information for your subject. Note that several fields are pre-populated with some default values.

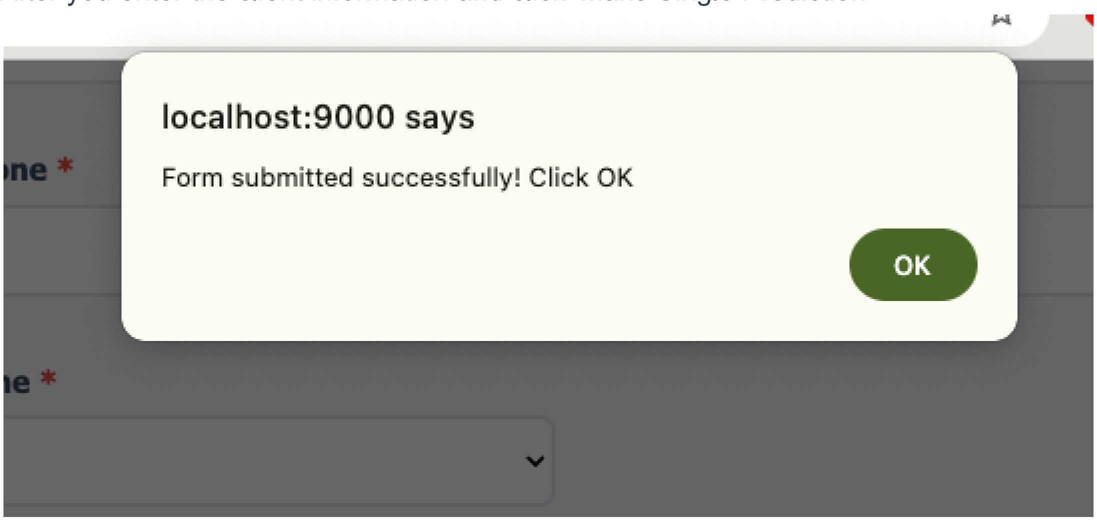

Mediwatch Single Prediction Form
PATIENT INFORMATION
Race
Select Race
Gender
Select Gender
Age *
Select Age Range
Weight
ADMISSION INFORMATION
Admission Type ID
0
Discharge Disposition ID
0
Admission Source ID
0
Time in Hospital (days)
0
Payer Code
None
Medical Specialty

...

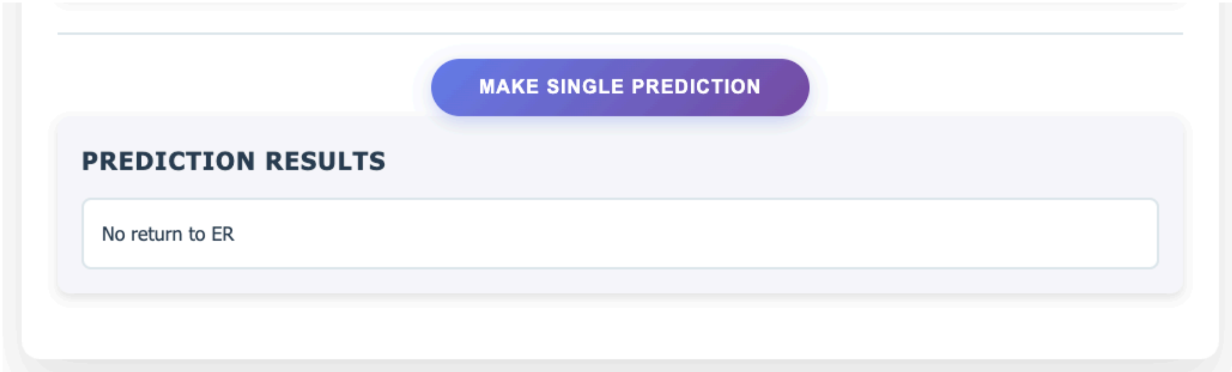
...



After you enter the client information and click ‘Make Single Prediction’



Prediction results are shown on the screen



This endpoint calls the single_prediction endpoint (see FastAPI here)

POST `/single_prediction` Single Prediction Post

Parameters

No parameters

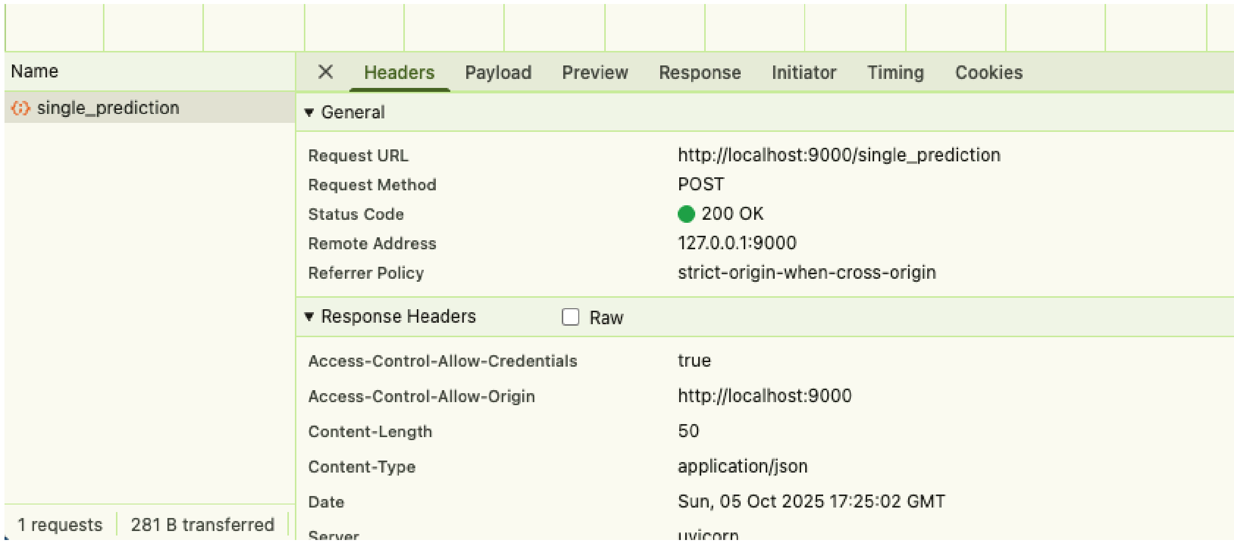
Request body required

Example Value | Schema

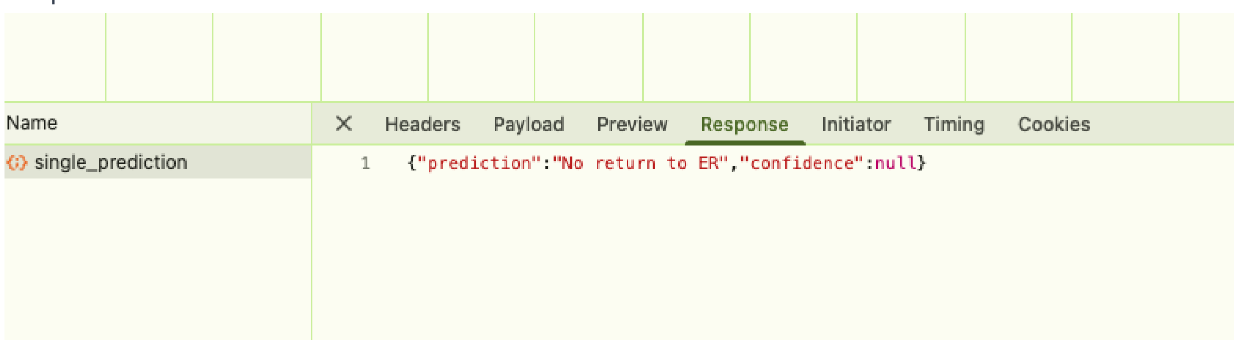
```
{
  "race": "string",
  "gender": "string",
  "age": "string",
  "weight": "string",
  "admission_type_id": 0,
  "discharge_disposition_id": 0,
  "admission_source_id": 0,
  "time_in_hospital": 0,
  "payer_code": "string",
  "medical_specialty": "string",
  "num_lab_procedures": 0,
  "num_procedures": 0,
  "num_medications": 0,
  "number_outpatient": 0,
  "number_emergency": 0,
  "number_inpatient": 0,
  "diag_1": "string",
  "diag_2": "string",
  "diag_3": "string",
  "number_diagnoses": 0,
  "metformin": "string",
  "repaglinide": "string",
  "nateglinide": "string",
  "chlorpropamide": "string"
```

And the call itself:

Call:



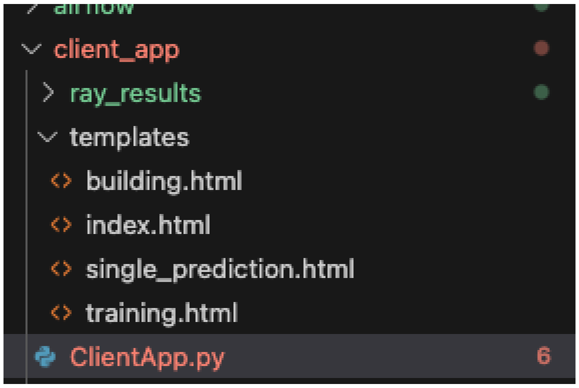
Response:



Once again, refer to:
client_app/Client_App.py
client_app/templates

Prediction Template

The single prediction template is found here:



Specifically **client_app/templates/single_prediction.html** renders the UI used for the prediction page

ClientApp.py

The actual backend code is found in **client_app/Client_App.py** under the ‘single_prediction_post’ function. The call to the prediction is on line 139 below client_app > ClientApp.py > single_prediction_post > result

```python
    logs: list[str] = None

@app.post("/single_prediction",response_model=PredictionResponse)
async def single_prediction_post(prediction_data: MediwatchSubject):
    medi_data=prediction_data

    # convert pydantic model to a dictionary
    medi_data_dict=medi_data.model_dump(by_alias=True)

    # convert dictonary to a dataframe
    medi_data_df=pd.DataFrame([medi_data_dict])

    # send dataframe to classifier
    result=clApp.classifier.single_prediction_30_day_readmission(medi_data_df)

    # Convert numpy array to string or list
    if isinstance(result, np.ndarray):
        print ("result is an np.ndarray") # TODO DEBUG!!
        prediction_value = result.tolist()  # Convert to Python list
    else:
        prediction_value = str(result)

    return PredictionResponse(
        prediction=str(prediction_value)  # Ensure it's a string
    )
```

Large Scale Hyperparameter Optimization and Experiment Tracking

Ray Tune

I used ray tune to optimize hyperparameter tuning for my project. The ray tune code is located in the **ray_tune** folder.

ray_tune/ray_tune.py contains the code.
The main function in the ray_tune.py is the

```
diabetic_data_main_with_ray_tune()
```

Here is a screenshot of part of the function:

```
def diabetic_data_main_with_ray_tune():
    # initialize mlflow
    mlflow.set_tracking_uri(cfg.MLFLOW_TRACKING_URI)
    # Initialize Ray
    if not ray.is_initialized():
        ray.init()
    start_time = datetime.now()
    # Prepare data once
    print("Preparing data...")
    X_train, X_test, y_train, y_test, one_hot_encoder, binary_encoder = training.prepare_data(cfg.CSV_ZIP_FILE_DIRECTORY,cfg.CSV_ZIP

    ## Train
    data_dict={
        "X_train":X_train,
        "X_test":X_test,
        "y_train":y_train,
        "y_test":y_test
     }

    search_space ={
        "n_estimators": tune.choice(cfg.N_ESTIMATORS),
        "max_depth": tune.choice(cfg.MAX_DEPTH),
        "learning_rate": tune.loguniform(cfg.LEARNING_RATE_LOWER,cfg.LEARNING_RATE_UPPER),
        "subsample": tune.uniform(cfg.SUBSAMPLE_LOWER, cfg.SUBSAMPLE_UPPER),
        "colsample_bytree": tune.uniform(cfg.COLSAMPLE_BYTREE_LOWER, cfg.COLSAMPLE_BYTREE_UPPER),
```

Ray Tune Parameters

ray_tune/config.py contains the parameters to tune.

This is a quick screenshot of the parameters I used in the config file

```
NUM_SAMPLES = 50
N_ESTIMATORS=[100, 200, 300, 500, 800]
MAX_DEPTH=[3, 4, 5, 6, 7, 8]          # maximum depth of the tree
LEARNING_RATE_LOWER=0.01              # learning rate (aka eta)
LEARNING_RATE_UPPER=0.3               # learning rate (aka eta)
SUBSAMPLE_LOWER=0.6
SUBSAMPLE_UPPER=1.0
COLSAMPLE_BYTREE_LOWER=0.6
COLSAMPLE_BYTREE_UPPER=1.0
GAMMA_LOWER=0
GAMMA_UPPER=5
REG_ALPHA_LOWER=1e-5
REG_ALPHA_UPPER=10
REG_LAMBDA_LOWER=1e-5
REG_LAMBDA_UPPER=10
```

Manually running ray_tune.py

Firstly activate the env

```
conda activate ./env/
```

Then run ray_tune

```
python3 ray_tune.py
```

```
colsample_bytree                                   0.85648
gamma                                              4.30944
learning_rate                                      0.25042
max_depth                                                6
n_estimators                                           200
reg_alpha                                          0.00309
reg_lambda                                           4e-05
subsample                                          0.94575

Trial train_xgboost_self_contained_bfe40_00020 completed after 1 iterations at 2025-10-05 20:52:51. Total running time: 13s

Trial train_xgboost_self_contained_bfe40_00020 result

checkpoint_dir_name
time_this_iter_s                                   2.79463
time_total_s                                       2.79463
training_iteration                                       1
accuracy                                           0.56918
class_1_recall                                     0.00955

Trial train_xgboost_self_contained_bfe40_00019 completed after 1 iterations at 2025-10-05 20:52:51. Total running time: 13s

Trial train_xgboost_self_contained_bfe40_00019 result

checkpoint_dir_name
time_this_iter_s                                   3.21571
time_total_s                                       3.21571
training_iteration                                       1
accuracy                                           0.58141
class_1_recall                                           0

Trial train_xgboost_self_contained_bfe40_00022 completed after 1 iterations at 2025-10-05 20:52:52. Total running time: 14s
```

…

…

```
PROBLEMS 36   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS 4                                  bash - ray_tune
pstone/Capstone-MLOPS-Mediwatch/ray_tune/ray_results/xgboost_tune' in 0.0181s.

Trial status: 50 TERMINATED
Current time: 2025-10-05 20:53:08. Total running time: 30s
Logical resource usage: 1.0/12 CPUs, 0/0 GPUs
Current best trial: bfe40_00015 with class_1_recall=0.07643312101910828 and params={'n_estimators': 100, 'max_depth': 8, 'learning_rate': 0.29364315257118473, '
subsample': 0.8496022353158019, 'colsample_bytree': 0.9496337287218778, 'gamma': 0.21862311940031243, 'reg_alpha': 7.986917103852926e-05, 'reg_lambda': 0.000654
3141252716773}

| Trial name                                  status         n_estimators     max_depth     learning_rate     subsample     colsample_bytree     gamma     reg_a
lpha     reg_lambda     iter     total time (s)     accuracy     class_1_recall |
|
| train_xgboost_self_contained_bfe40_00000   TERMINATED            300             3         0.0302248       0.868344            0.609745   1.61803    0.00366
164      0.00012712       1           3.0762        0.581062      0             |
| train_xgboost_self_contained_bfe40_00001   TERMINATED            200             6         0.190153        0.718605            0.674733   4.96167    0.00014
2203     8.73378e-05      1           2.05784       0.579665      0             |
| train_xgboost_self_contained_bfe40_00002   TERMINATED            500             5         0.0432915       0.735021            0.65563    0.894336   0.00035
3243     0.000575427      1           4.68148       0.577568      0.00636943    |
| train_xgboost_self_contained_bfe40_00003   TERMINATED            500             7         0.131234        0.945586            0.673671   1.24373    7.00305
         5.13632          1           3.56577       0.577918      0             |
| train_xgboost_self_contained_bfe40_00004   TERMINATED            500             3         0.0355454       0.688324            0.975618   3.79403    2.61611
e-05     0.00156412       1           3.7445        0.580363      0             |
| train_xgboost_self_contained_bfe40_00005   TERMINATED            500             7         0.107106        0.887806            0.652368   0.821343   0.00094
3359     0.00903305       1           4.05882       0.573725      0.0127389     |
| train_xgboost_self_contained_bfe40_00006   TERMINATED            500             7         0.0286155       0.625363            0.634147   0.863229   0.00642
61       0.00912108       1           6.84514       0.569182      0             |
| train_xgboost_self_contained_bfe40_00007   TERMINATED            300             7         0.0407895       0.964584            0.762213   1.90452    0.02037
31       0.00319205       1           3.14701       0.581412      0.00318471    |
| train_xgboost_self_contained_bfe40_00008   TERMINATED            200             4         0.0619453       0.677971            0.698736   3.75309    0.04215
62       1.62245e-05      1           2.1459        0.581761      0             |
| train_xgboost_self_contained_bfe40_00009   TERMINATED            100             5         0.0212341       0.789488            0.972532   2.5643     0.00133
174      0.0198721        1           2.03801       0.576869      0             |
| train_xgboost_self_contained_bfe40_00010   TERMINATED            200             3         0.0491496       0.93682             0.761843   3.1525     0.02148
47       0.966877         1           2.19848       0.580713      0             |
| train_xgboost_self_contained_bfe40_00011   TERMINATED            500             4         0.0567145       0.697924            0.831668   0.927453   0.00154
341      1.96503e-05      1           4.92564       0.58246       0.00636943    |
```

…

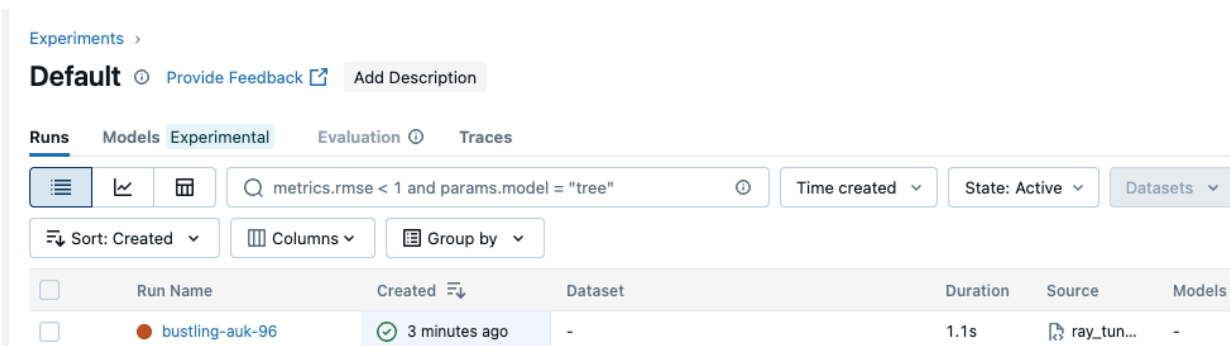
…

```
Best hyperparameters found:
  n_estimators: 100
  max_depth: 8
  learning_rate: 0.29364315257118473
  subsample: 0.8496022353158019
  colsample_bytree: 0.9496337287218778
  gamma: 0.21862311940031243
  reg_alpha: 7.986917103852926e-05
  reg_lambda: 0.0006543141252716773

Best Class 1 recall: 0.0764
Best accuracy: 0.5461

Training final model with best hyperparameters...

Final Model Performance:
XGBoost Accuracy: 0.5461215932914046
XGBoost Class 1 Recall: 0.07643312101910828
              precision    recall  f1-score   support

           0       0.59      0.71      0.64      1431
           1       0.32      0.08      0.12       314
           2       0.50      0.47      0.48      1117

    accuracy                           0.55      2862
   macro avg       0.47      0.42      0.42      2862
weighted avg       0.52      0.55      0.52      2862

🏃 View run bustling-auk-96 at: http://localhost:5000/#/experiments/0/runs/59d67efeced349e189480c803e52cafa
🧪 View experiment at: http://localhost:5000/#/experiments/0
```

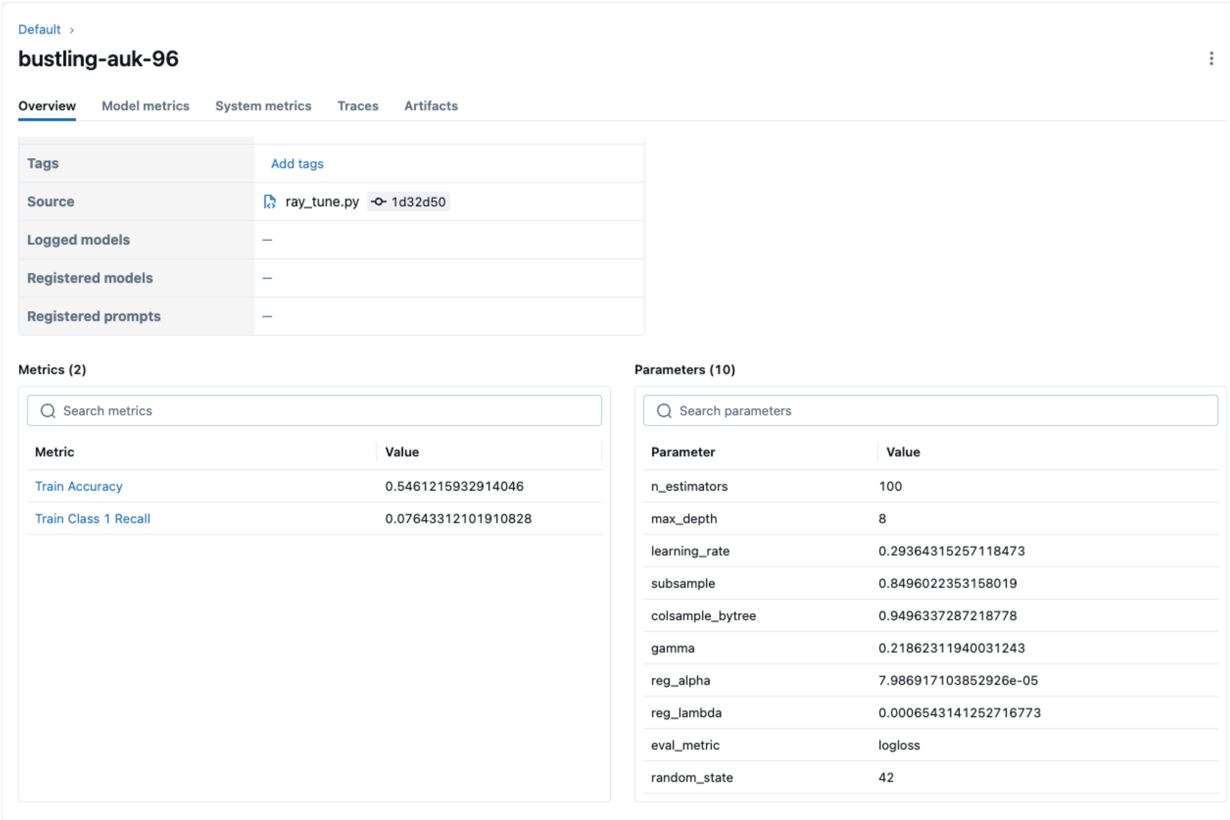
The model is finally trained with the best hyperparameters available

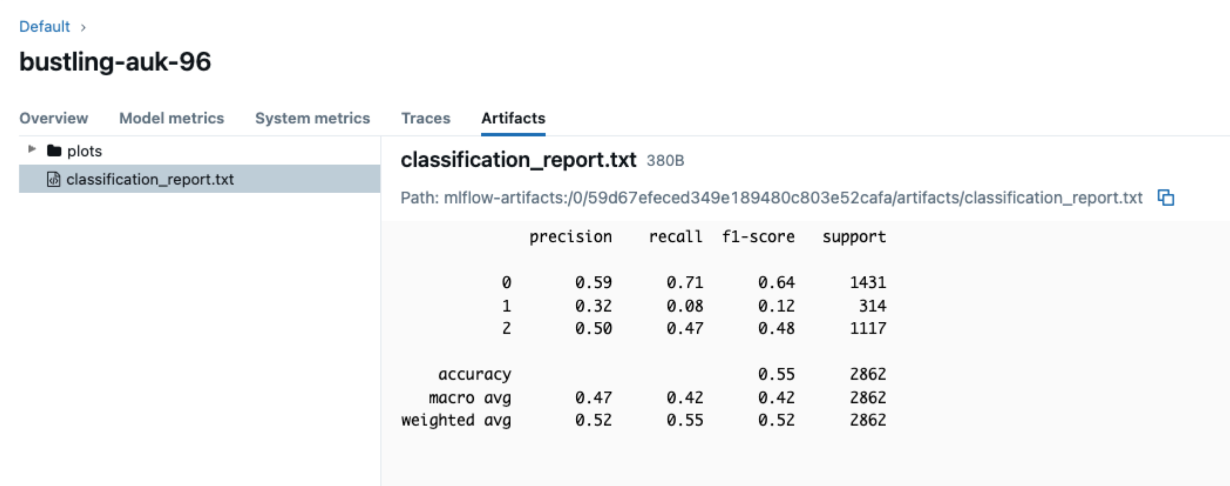
MLFLOW

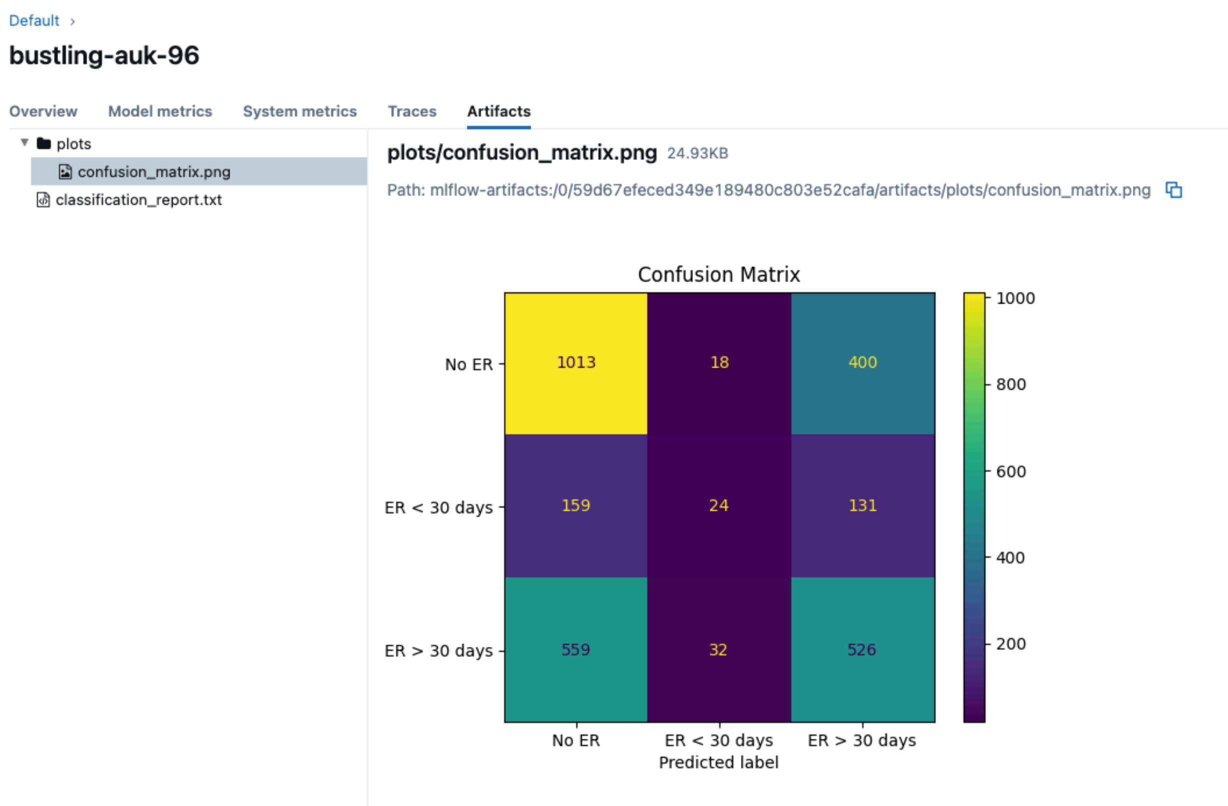
The ray_tune run is reported in mlflow. Here is the run we just performed:



As you can see, all the ray parameters are reported:

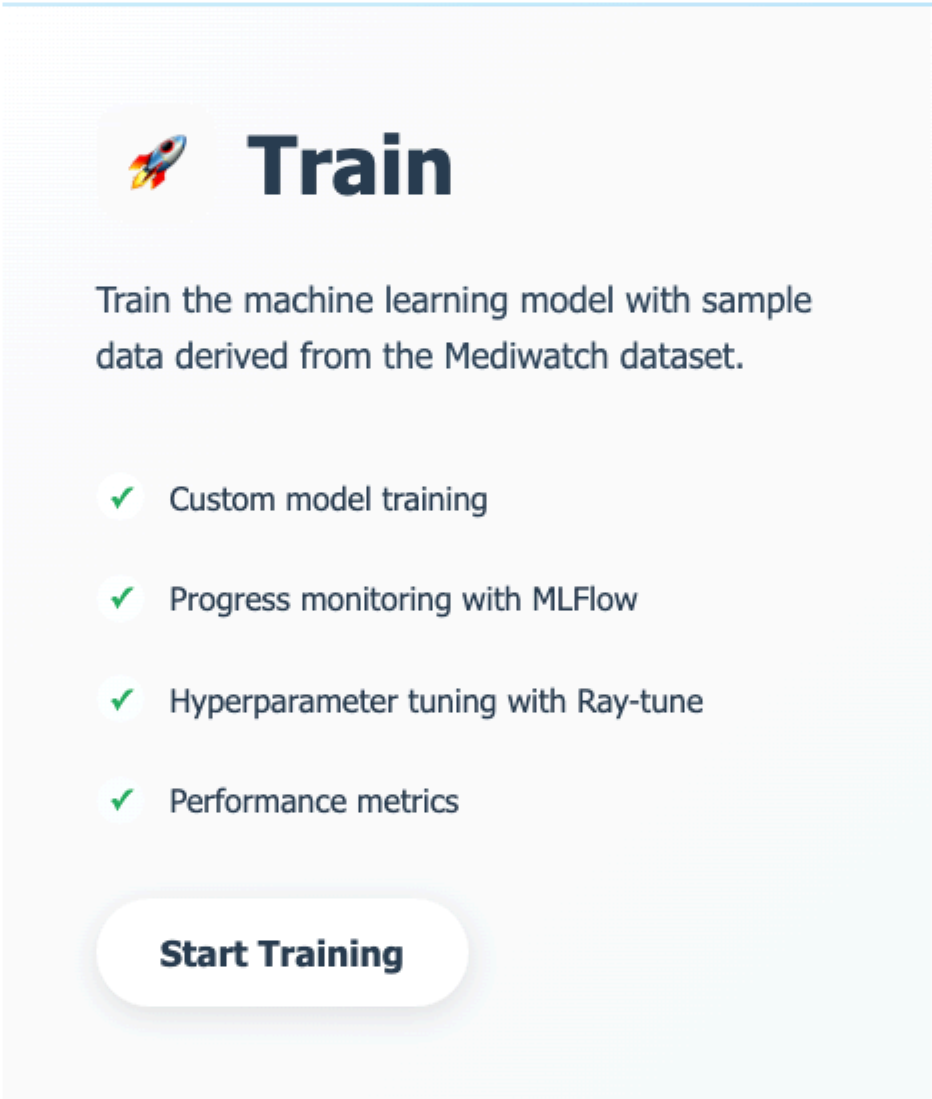

Default ›
bustling-auk-96
Overview
Model metrics
System metrics
Traces
Artifacts
Tags
Add tags
Source
ray_tune.py
1d32d50
Logged models
—
Registered models
—
Registered prompts
—
Metrics (2)
Search metrics
Metric
Value
Train Accuracy
0.5461215932914046
Train Class 1 Recall
0.07643312101910828
Parameters (10)
Search parameters
Parameter
Value
n_estimators
100
max_depth
8
learning_rate
0.29364315257118473
subsample
0.8496022353158019
colsample_bytree
0.9496337287218778
gamma
0.21862311940031243
reg_alpha
7.986917103852926e-05
reg_lambda
0.0006543141252716773
eval_metric
logloss
random_state
42


Default ›
bustling-auk-96
Overview
Model metrics
System metrics
Traces
Artifacts
plots
classification_report.txt
classification_report.txt
380B
Path: mlflow-artifacts:/0/59d67efeced349e189480c803e52cafa/artifacts/classification_report.txt
precision recall f1-score support
0 0.59 0.71 0.64 1431
1 0.32 0.08 0.12 314
2 0.50 0.47 0.48 1117
accuracy 0.55 2862
macro avg 0.47 0.42 0.42 2862
weighted avg 0.52 0.55 0.52 2862



Running ray_tune through the UI

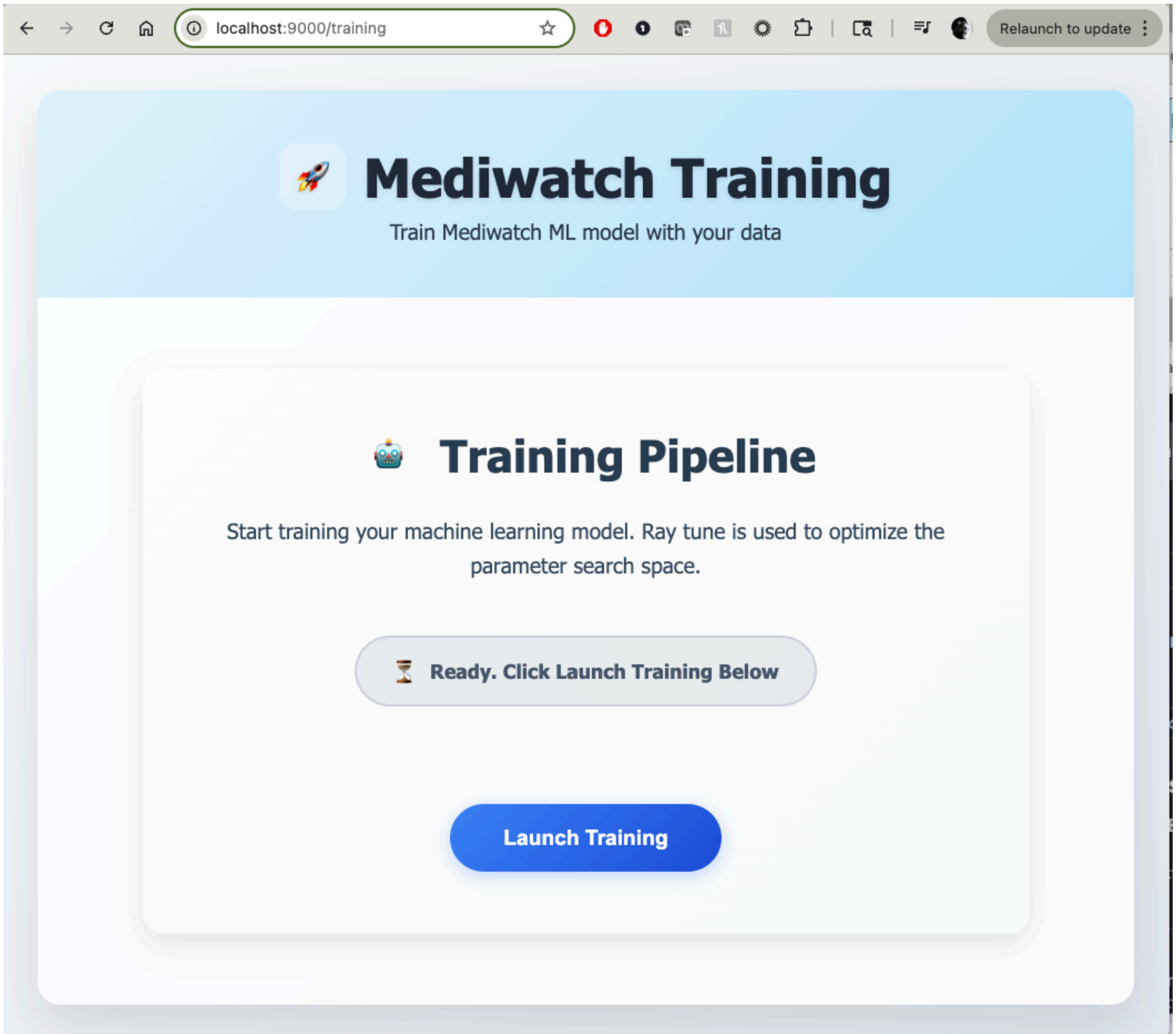
This is the easier way to run ray_tune. In the UI, go here:



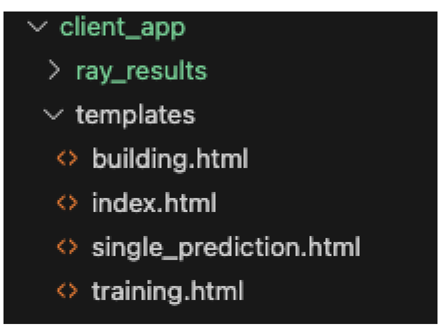
And click on the Start Training button.

Training endpoint

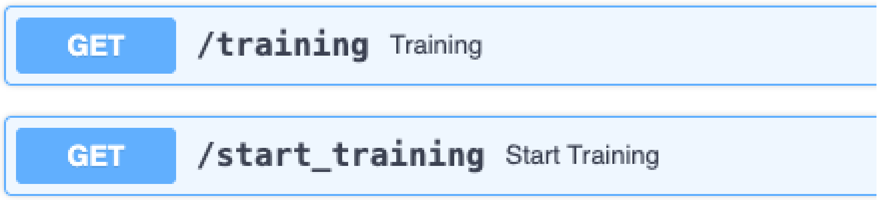
This takes you to the **/training** endpoint shown here



This page is rendered from the training.html template in **client_app/templates/training.html**



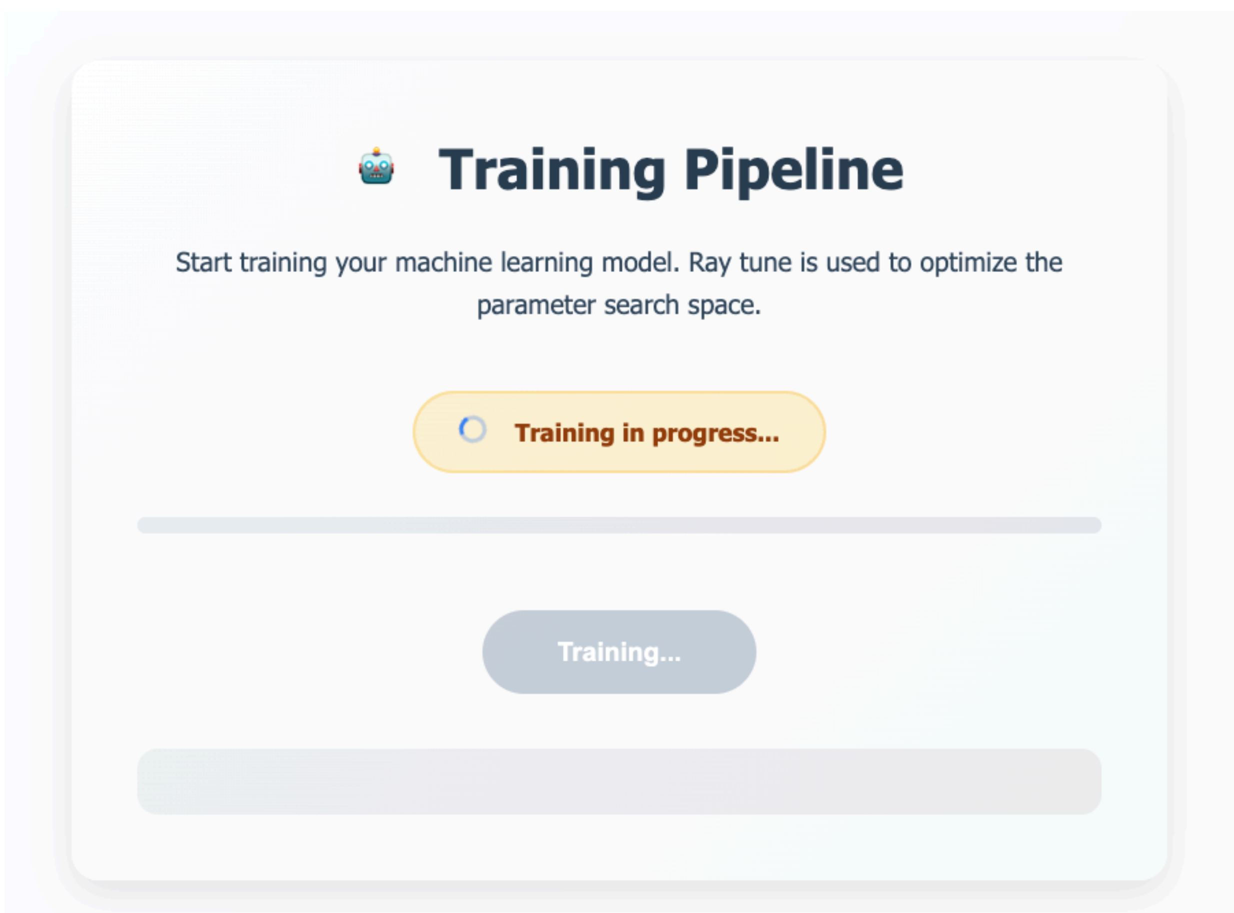
In Fast API, the /training renders the training page above.



The start_training runs ray_tune and returns logs IN the UI:

Start Training

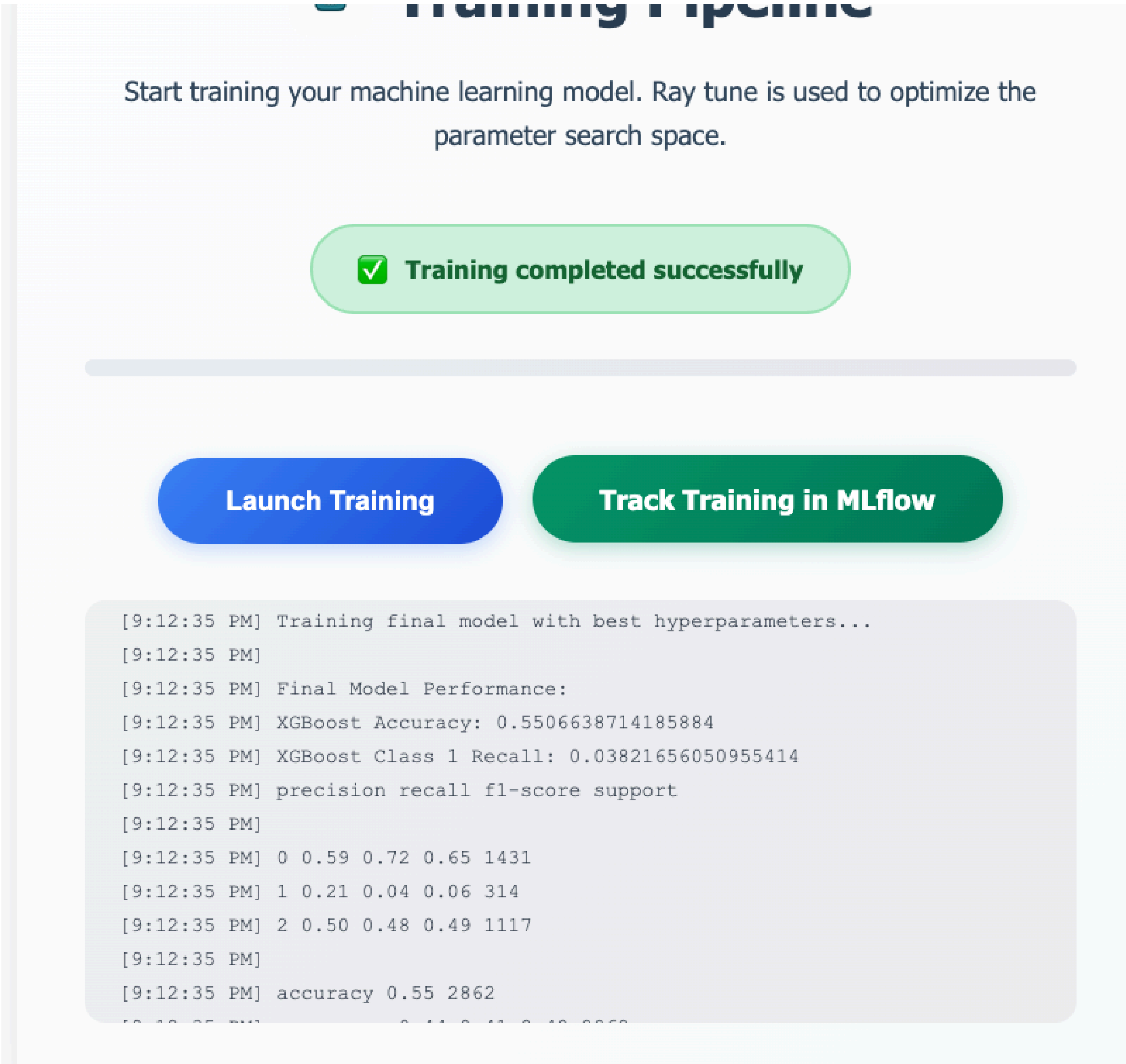
Click on the Launch Training button



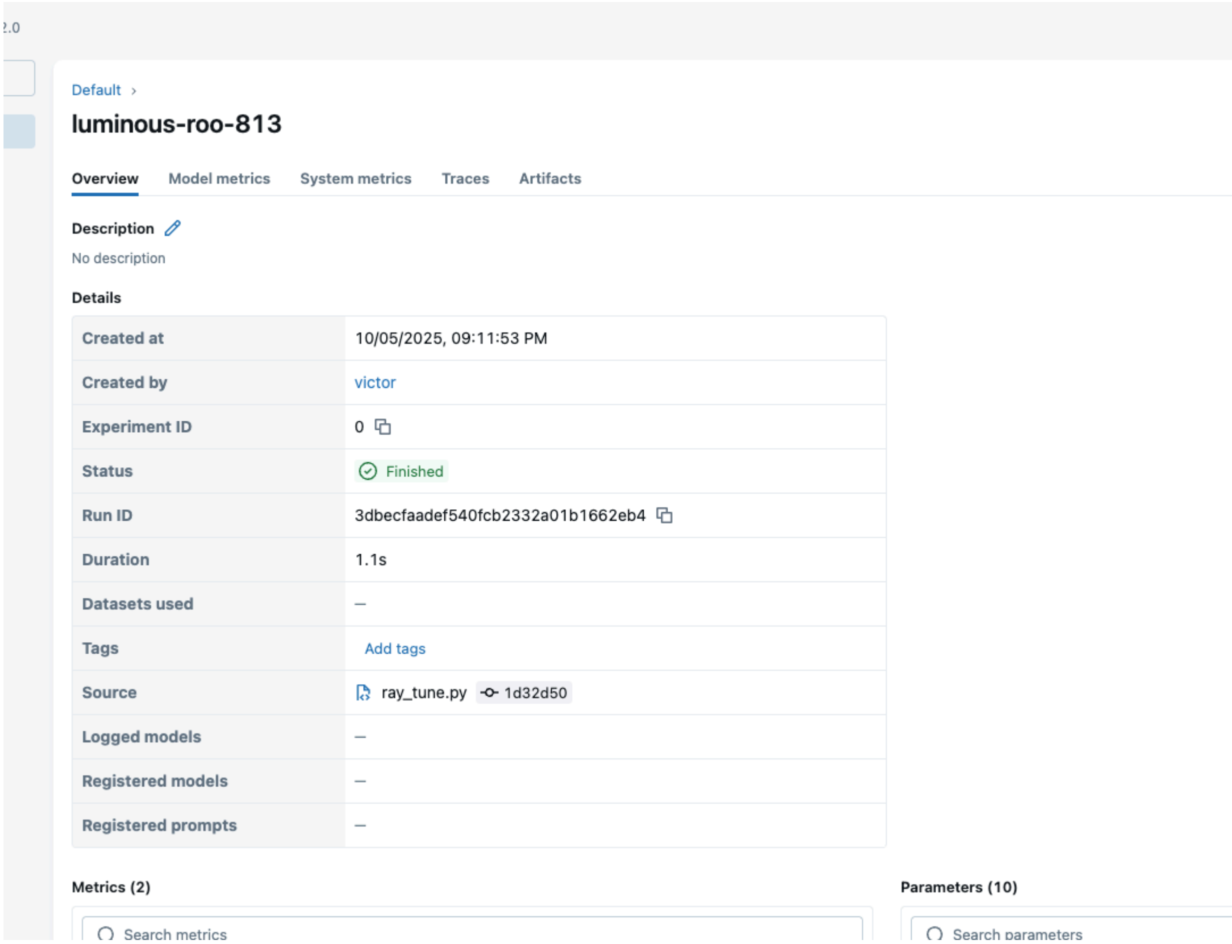
Logs are returned in line in the browser…

```
[9:12:35 PM] 2 0.50 0.48 0.49 1117
[9:12:35 PM]
[9:12:35 PM] accuracy 0.55 2862
[9:12:35 PM] macro avg 0.44 0.41 0.40 2862
[9:12:35 PM] weighted avg 0.52 0.55 0.52 2862
[9:12:35 PM]
[9:12:35 PM] 🏃 View run luminous-roo-813 at:
http://localhost:5000/#/experiments/0/runs/3dbecfaadef540fcb2332a01b1662eb4
[9:12:35 PM] 🧪 View experiment at: http://localhost:5000/#/experiments/0
[9:12:35 PM] Time taken for Ray tune (seconds): 37
[9:12:35 PM] Training completed
```

When completed, you get the button that takes you to mlflow



Where you can see your completed training run



Note that the ray_tune training run will update the models in the model directory just like the basic training run did:

The start_training endpoint code is in the **client_app/ClientApp.py** . Here is a screenshot of part of that function:

```python
# Start training, return logs to front end
@app.get("/start_training")
async def start_training():
    """Start training and return real-time logs via SSE"""

    async def generate_training_logs() -> AsyncGenerator[str, None]:
        # Your actual training command
        command = ["python3", clApp.ray_tune_script]  # Replace with your training script
        process = subprocess.Popen(
            command,
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            universal_newlines=True,
            bufsize=1
        )

        try:
            while True:
                output = process.stdout.readline()
                if output == '' and process.poll() is not None:
```

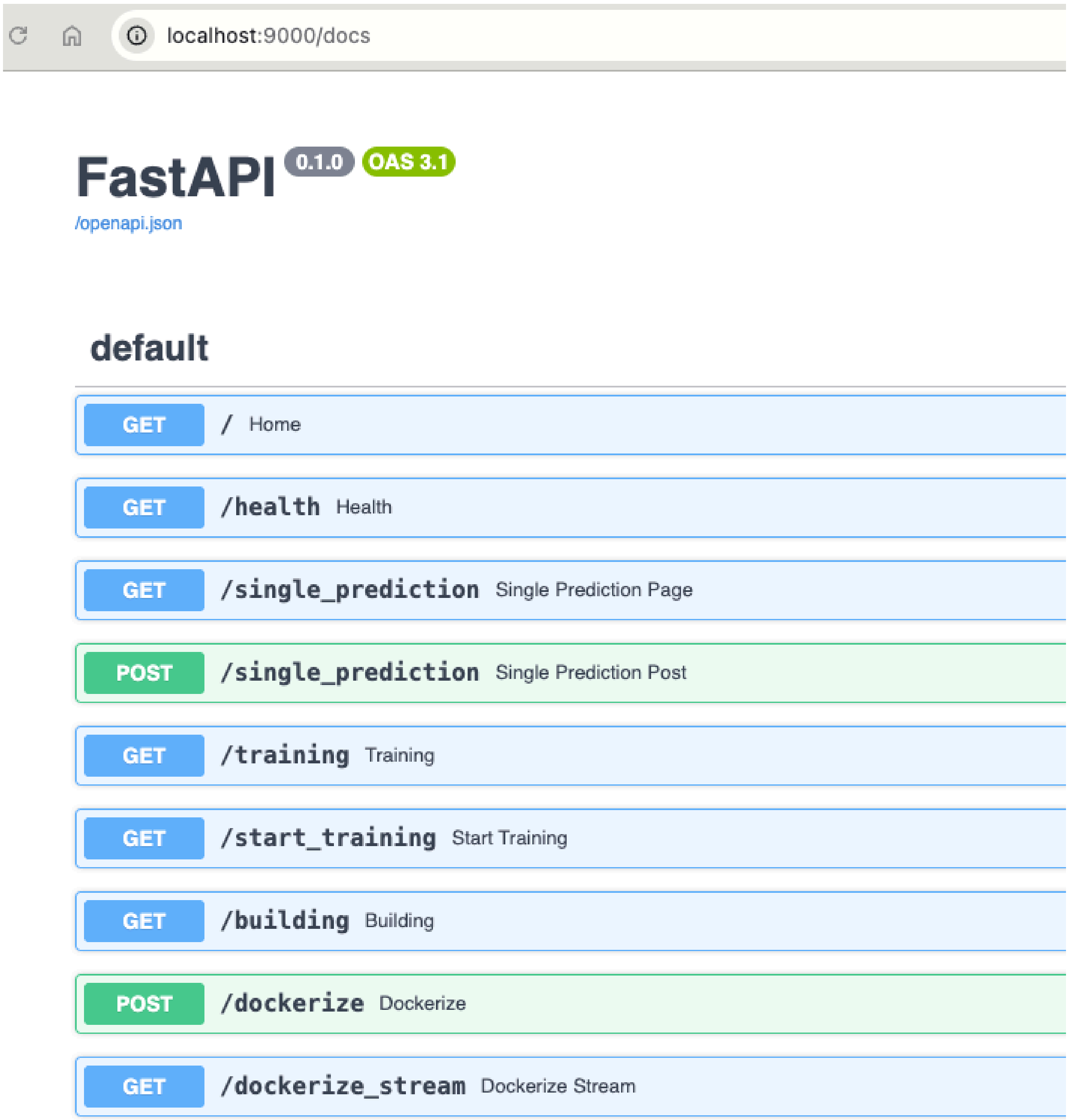
Model Packaging and Deployment

FastAPI

Lets revisit FastAPI again in some more detail

Its available at port 9000 when **ClientApp.py** is running:

These are the endpoints

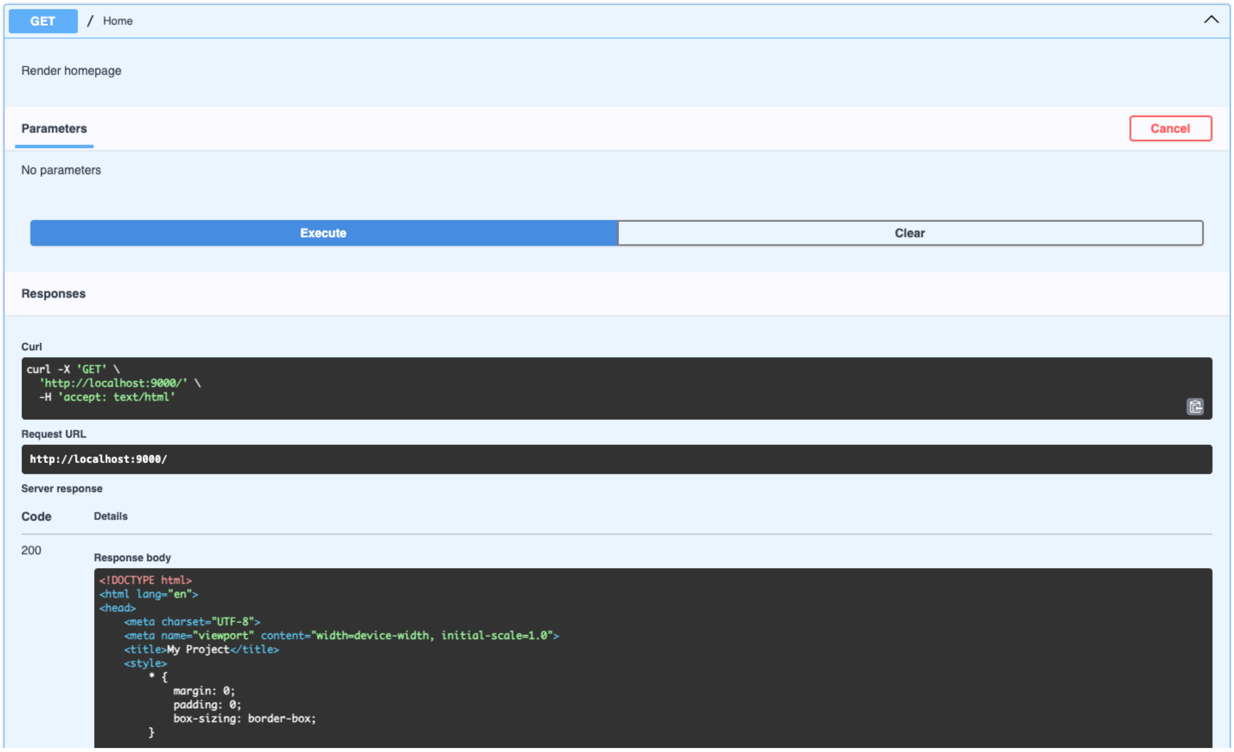


And below are related schemas:

We can run a couple of endpoints to see what is available

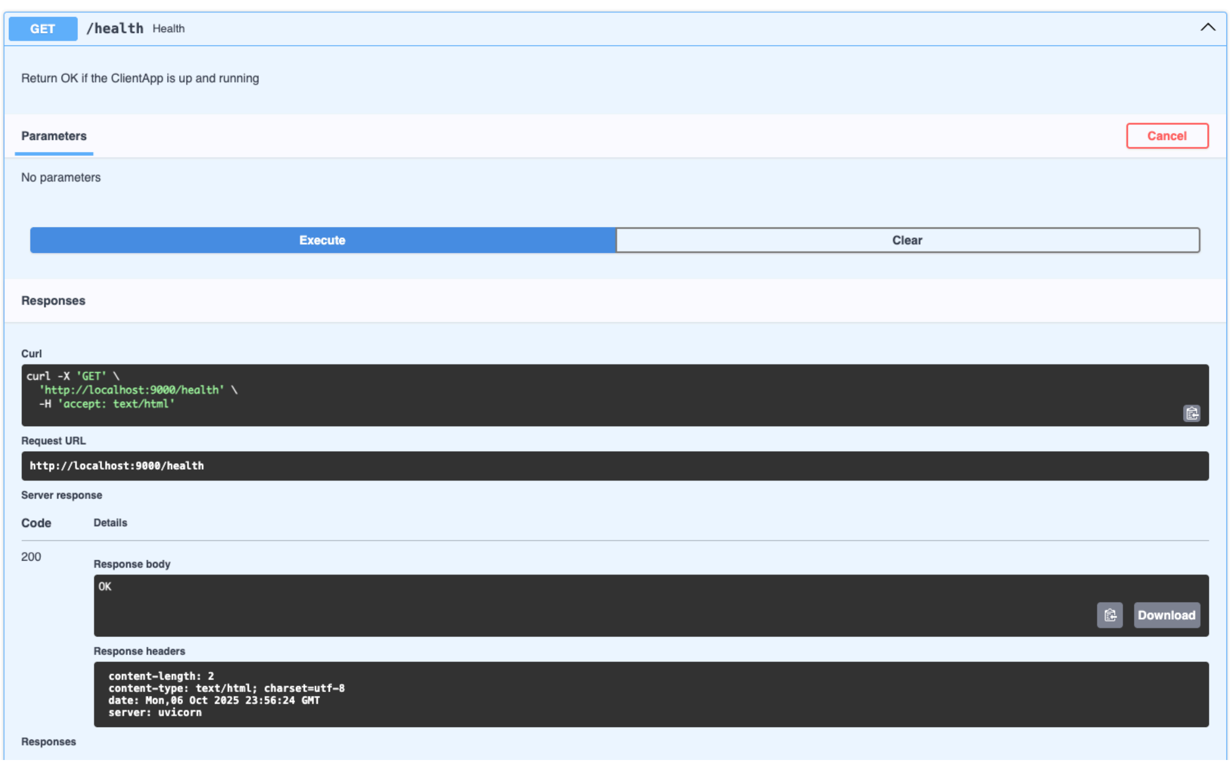
Test root endpoint

Here is a sample run against the root '/' endpoint



Test Health Endpoint

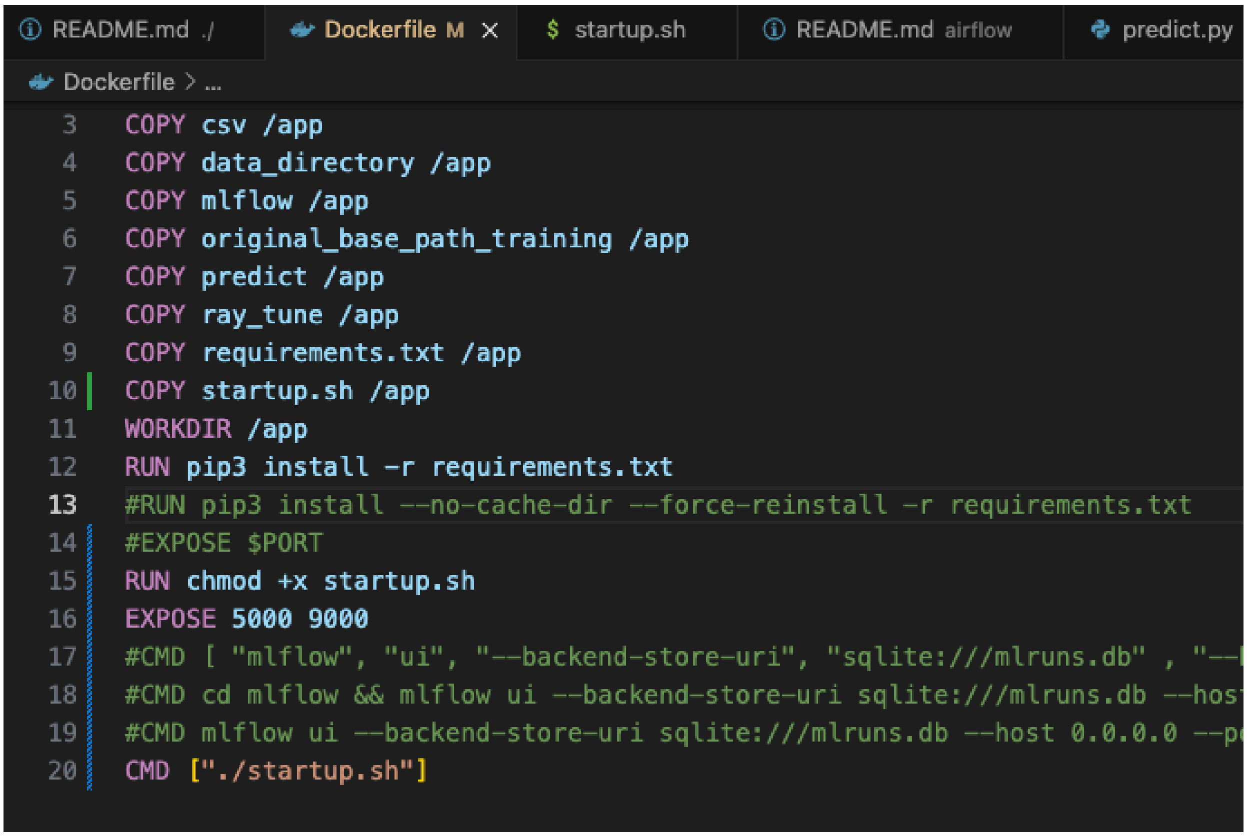
Here is a sample run against the health endpoint



Generate Docker Image

Dockerfile

First, look at the Dockerfile



README.md ./ | Dockerfile M | startup.sh | README.md airflow | predict.py

Dockerfile > ...

```
COPY csv /app
COPY data_directory /app
COPY mlflow /app
COPY original_base_path_training /app
COPY predict /app
COPY ray_tune /app
COPY requirements.txt /app
COPY startup.sh /app
WORKDIR /app
RUN pip3 install -r requirements.txt
#RUN pip3 install --no-cache-dir --force-reinstall -r requirements.txt
#EXPOSE $PORT
RUN chmod +x startup.sh
EXPOSE 5000 9000
#CMD [ "mlflow", "ui", "--backend-store-uri", "sqlite:///mlruns.db" , "--
#CMD cd mlflow && mlflow ui --backend-store-uri sqlite:///mlruns.db --hos
#CMD mlflow ui --backend-store-uri sqlite:///mlruns.db --host 0.0.0.0 --p
CMD ["./startup.sh"]
```

I open ports 5000, 9000 for mlflow and the UI respectively.

Startup.sh

I coordinate startup of these scripts in the 'startup.sh' file as follows:

```
$ startup.sh
#!/bin/bash

# Start MLflow in the background
cd mlflow
mlflow ui --backend-store-uri sqlite:///mlruns.db --host 0.0.0.0 --port 5000 &

# Start the client app
cd ..
cd client_app && python3 ClientApp.py
```

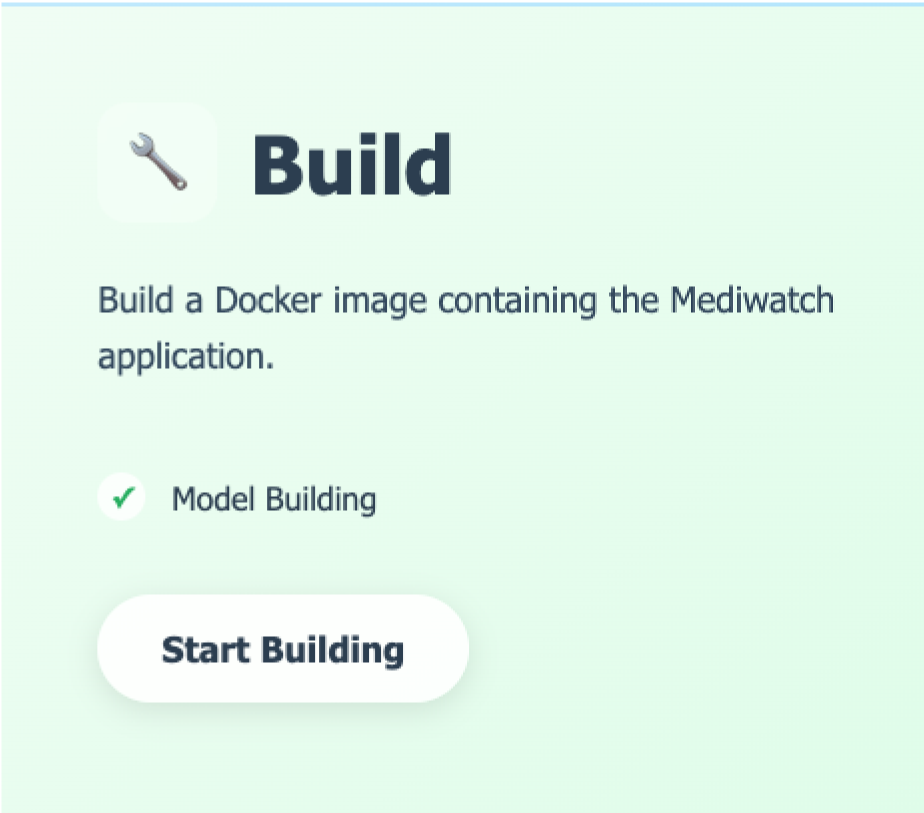
Docker images before build

Look at the available docker images:

```
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker image ls
REPOSITORY                  TAG                IMAGE ID       CREATED          SIZE
test_image1759979433        latest             bd6bda391ecc   24 minutes ago   11.9GB
redis                       latest             9d1fe3a9a889   7 weeks ago      137MB
postgres                    13                 645e932c27f7   7 weeks ago      439MB
victor_iris_k8s             latest             e8755e76702b   3 months ago     5.73GB
victor_dog_cat              latest             9b35a1e0d610   3 months ago     3.59GB
gcr.io/k8s-minikube/kicbase v0.0.47            795ea6a69ce6   4 months ago     1.31GB
apache/airflow              2.7.2              dad28d0405d6   22 months ago    1.54GB
apache/airflow              2.7.1-python3.11   4fb802b917bc   22 months ago    1.51GB
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
```

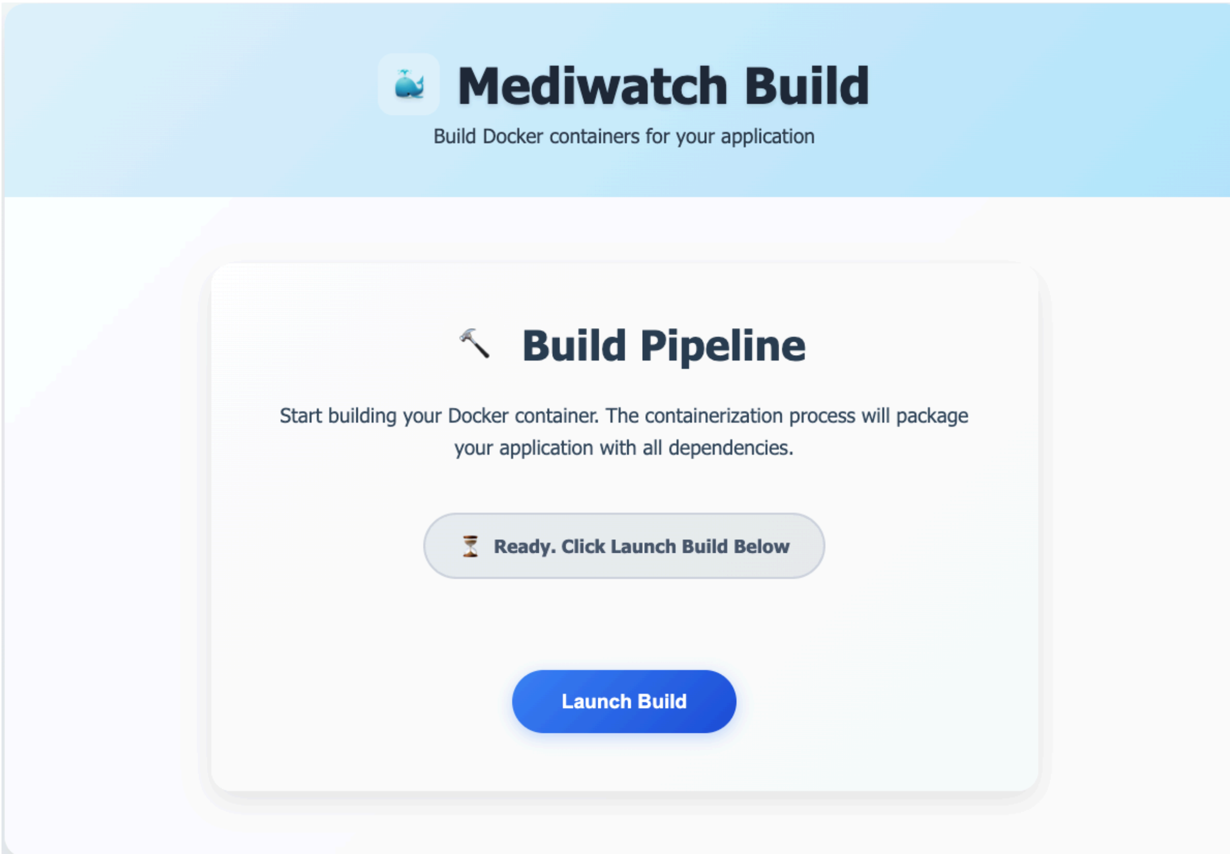
Run Docker build from UI

Then go to the UI and click on the Build Section



Click on the Start Building Button

You get this page which allows you to start the Docker build



Click on the Launch build button

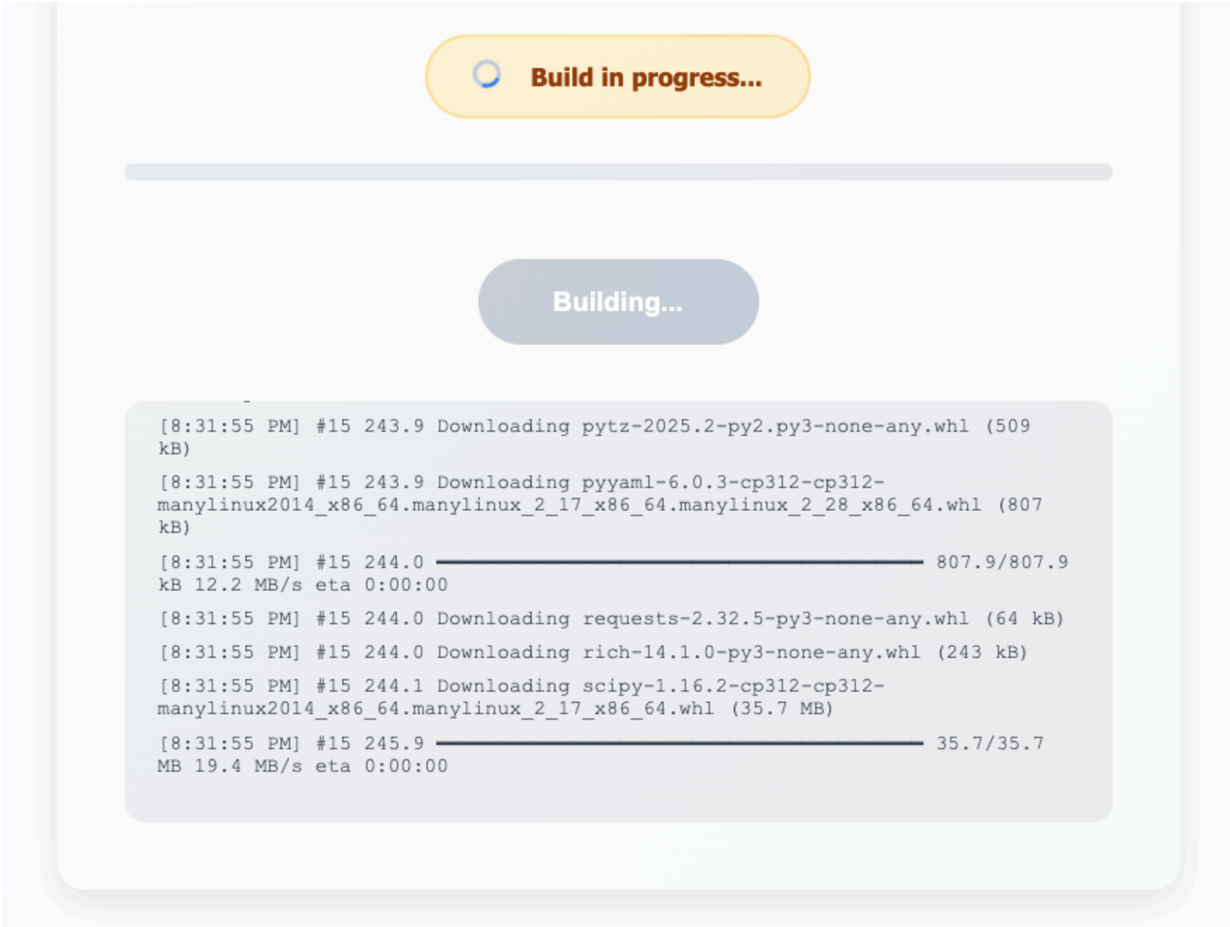
🔨 Build Pipeline

Start building your Docker container. The containerization process will package your application with all dependencies.

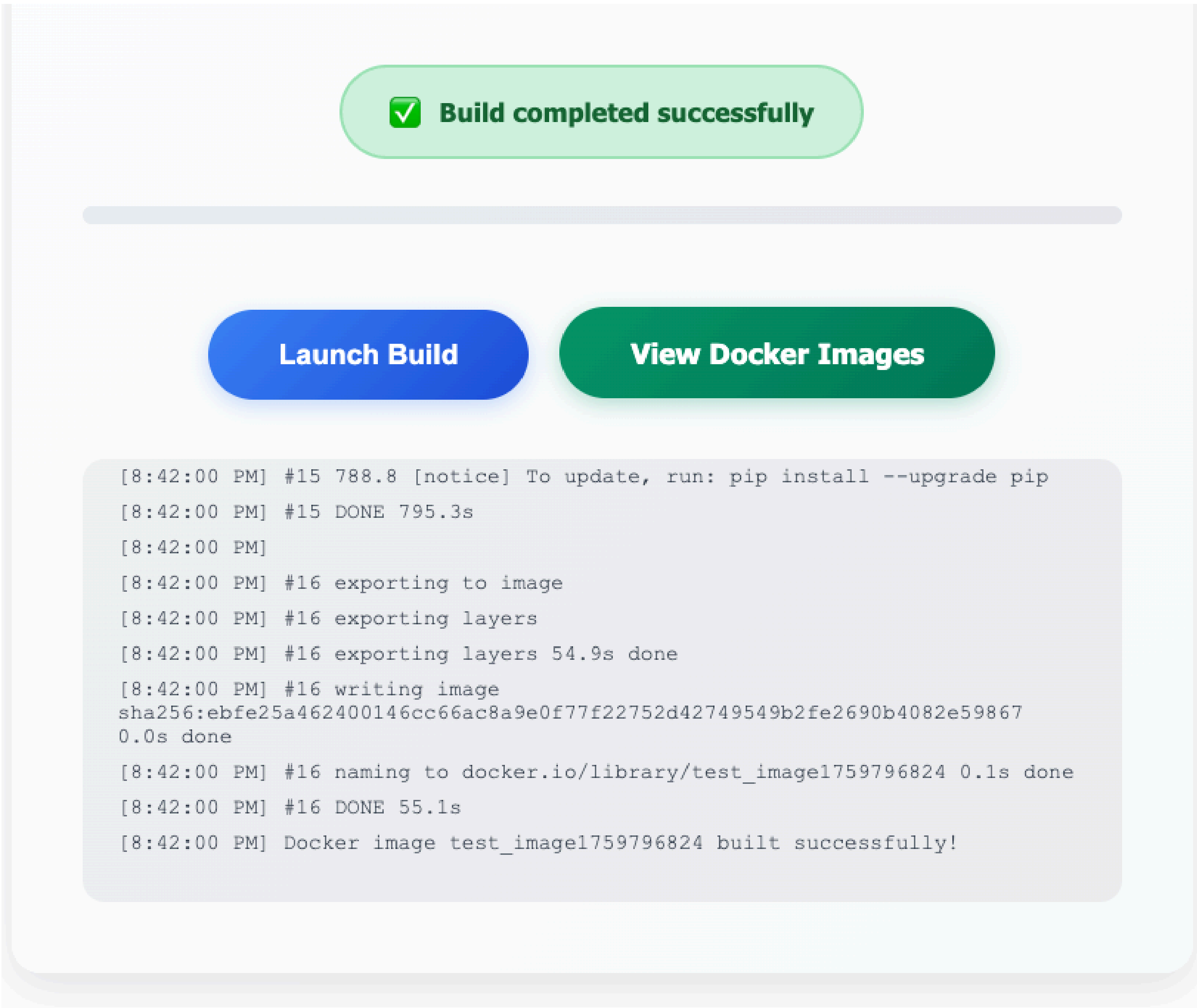
Build in progress...

Building...

(it will take a few mins to start logs…)

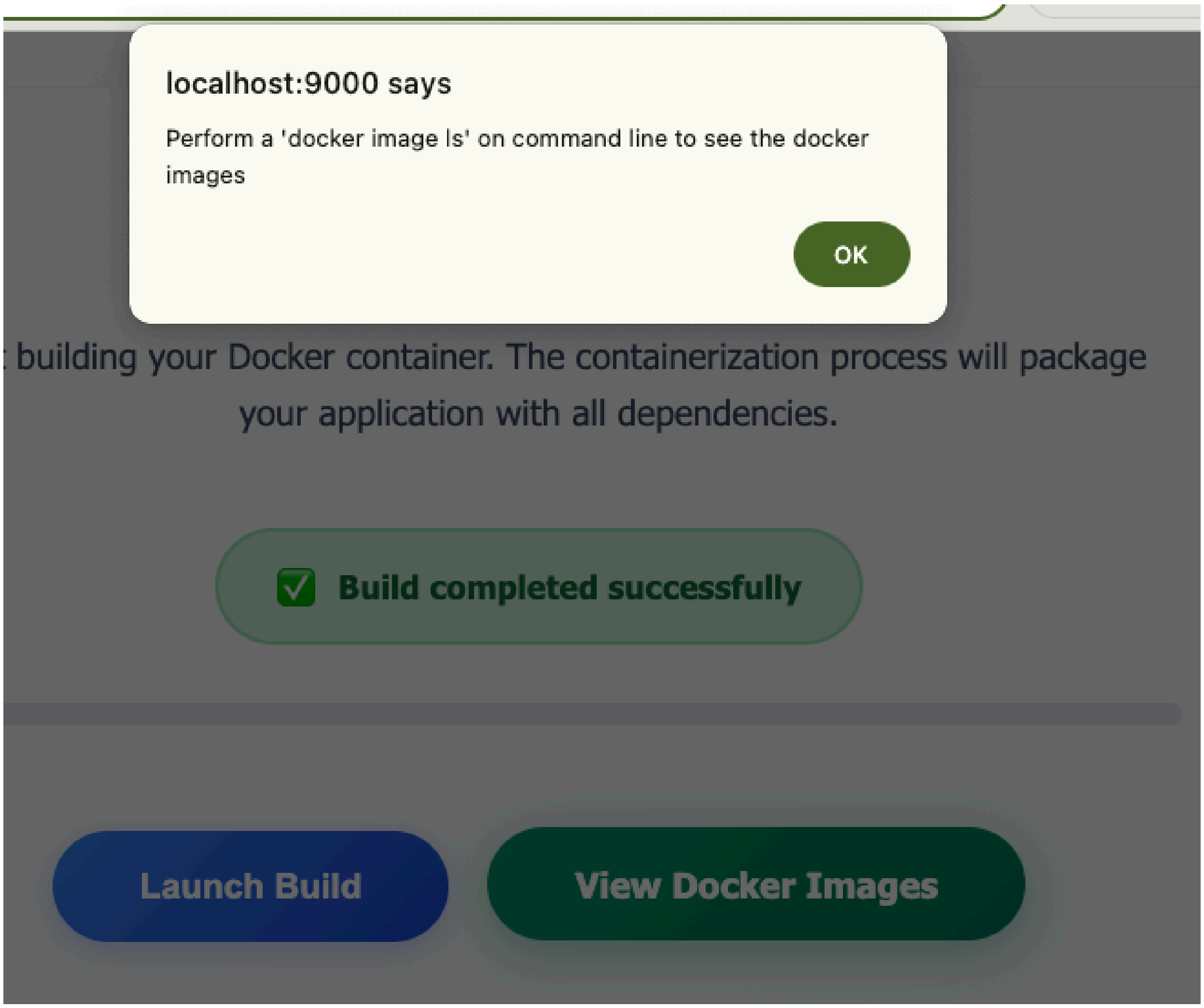


About ~ 10 minutes later



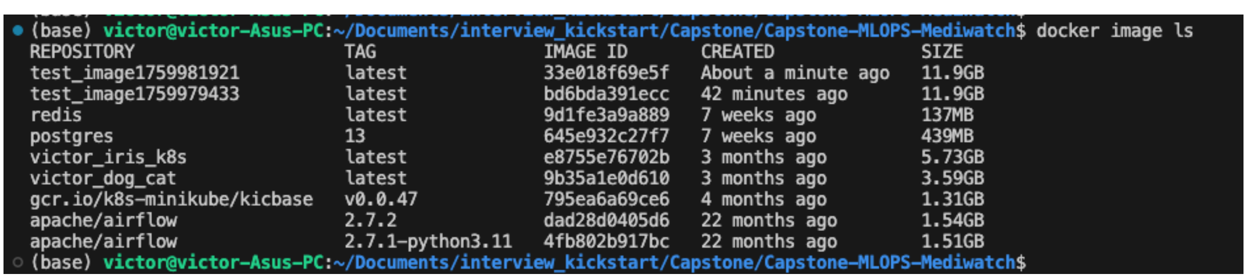
Also …

The ‘View Docker image’ gives this useful message:



And when you run the command:

Docker images after build



```
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker image ls
REPOSITORY                  TAG                IMAGE ID       CREATED            SIZE
test_image1759981921        latest             33e018f69e5f   About a minute ago 11.9GB
test_image1759979433        latest             bd6bda391ecc   42 minutes ago     11.9GB
redis                       latest             9d1fe3a9a889   7 weeks ago        137MB
postgres                    13                 645e932c27f7   7 weeks ago        439MB
victor_iris_k8s             latest             e8755e76702b   3 months ago       5.73GB
victor_dog_cat              latest             9b35a1e0d610   3 months ago       3.59GB
gcr.io/k8s-minikube/kicbase v0.0.47            795ea6a69ce6   4 months ago       1.31GB
apache/airflow              2.7.2              dad28d0405d6   22 months ago      1.54GB
apache/airflow              2.7.1-python3.11   4fb802b917bc   22 months ago      1.51GB
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
```

The new docker image is available at the top of the list

Run Docker image

Lets run the Docker image now that it has been built…

For my image **test_image1759981921**

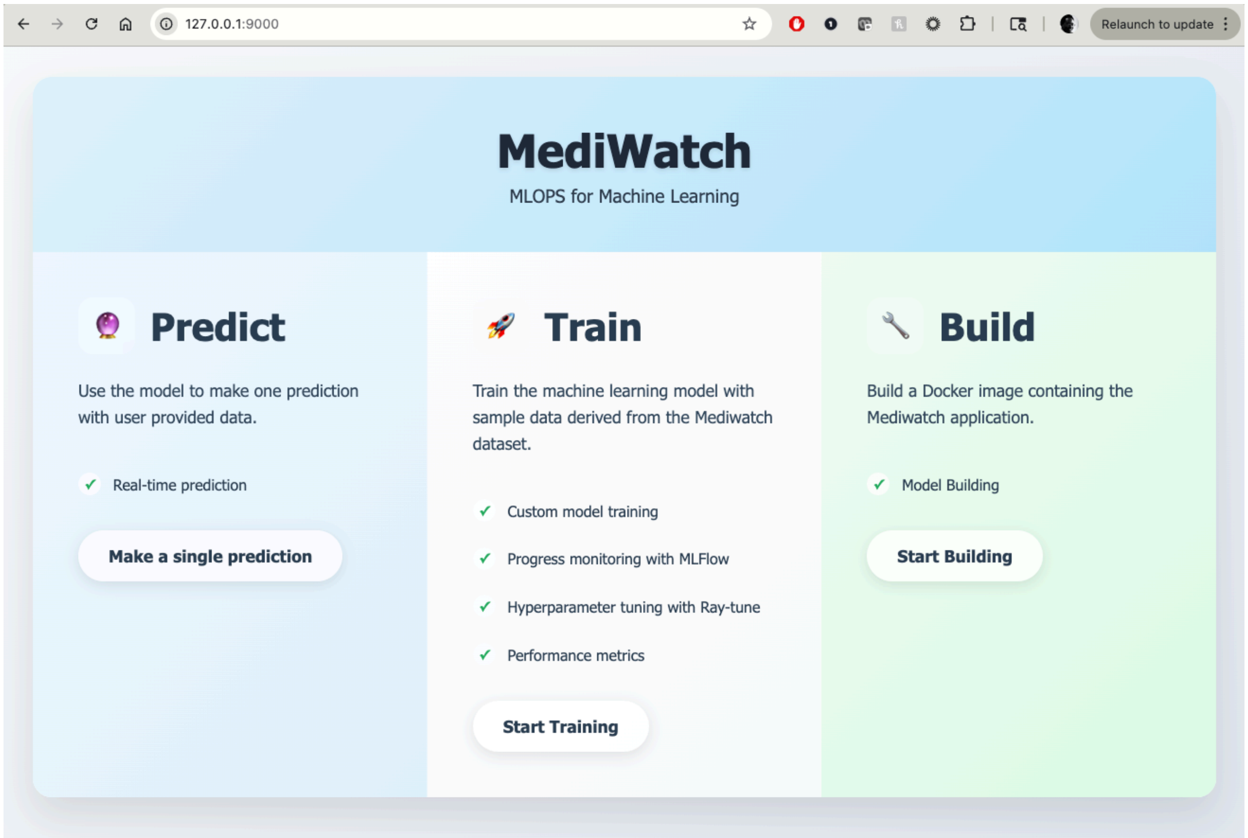
Lets run it …

```
sudo docker run -p 5000:5000 -p 9000:9000 test_image1759981921
```

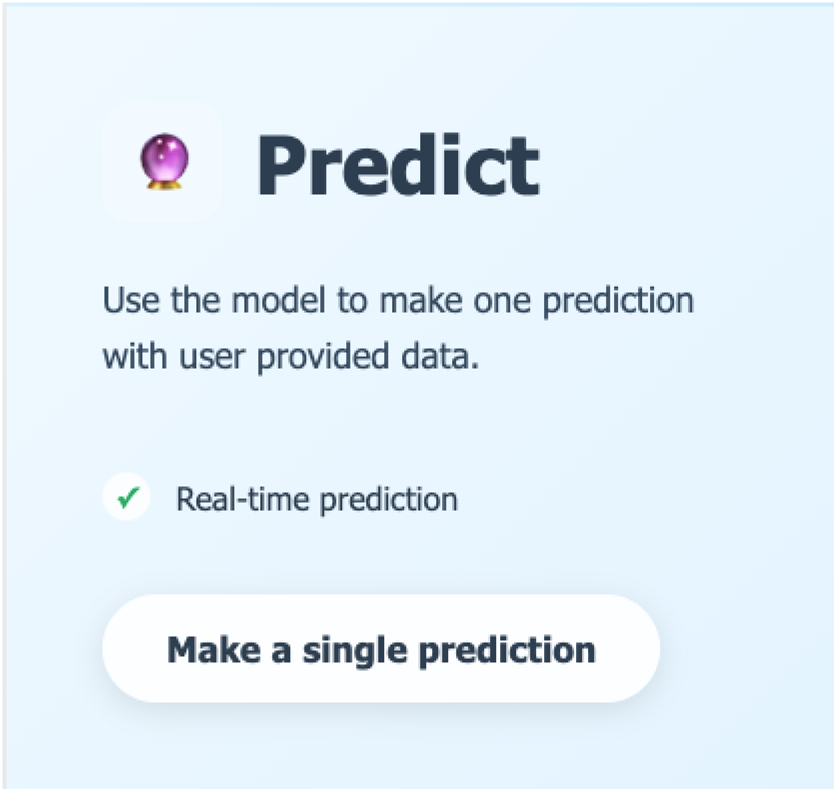
```
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ sudo docker run -p 5000:5000 -p 9000:9000
 test_image1759981921
[sudo] password for victor:
INFO:     Started server process [8]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:9000 (Press CTRL+C to quit)
2025/10/09 04:06:53 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2025/10/09 04:06:53 INFO mlflow.store.db.utils: Updating database tables
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
2025/10/09 04:06:54 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2025/10/09 04:06:54 INFO mlflow.store.db.utils: Updating database tables
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
[2025-10-09 04:06:54 +0000] [62] [INFO] Starting gunicorn 23.0.0
[2025-10-09 04:06:54 +0000] [62] [INFO] Listening at: http://0.0.0.0:5000 (62)
[2025-10-09 04:06:54 +0000] [62] [INFO] Using worker: sync
[2025-10-09 04:06:54 +0000] [63] [INFO] Booting worker with pid: 63
[2025-10-09 04:06:54 +0000] [64] [INFO] Booting worker with pid: 64
[2025-10-09 04:06:54 +0000] [65] [INFO] Booting worker with pid: 65
[2025-10-09 04:06:54 +0000] [66] [INFO] Booting worker with pid: 66
Warning: ray_tune.py not found at /app/client_app/../ray_tune/ray_tune.py
INFO:     172.17.0.1:48776 - "GET / HTTP/1.1" 200 OK
INFO:     172.17.0.1:48780 - "GET /single_prediction HTTP/1.1" 200 OK
/usr/local/lib/python3.12/site-packages/pydantic/_internal/_generate_schema.py:2249: UnsupportedFieldAttributeWarning: The 'alias' attrib
ute with value 'glyburide-metformin' was provided to the `Field()` function, which has no effect in the context it was used. 'alias' is f
ield-specific metadata, and can only be attached to a model field using `Annotated` metadata or by assignment. This may have happened bec
ause an `Annotated` type alias using the `type` statement was used, or if the `Field()` function was attached to a single member of a uni
on type.
  warnings.warn(
/usr/local/lib/python3.12/site-packages/pydantic/_internal/_generate_schema.py:2249: UnsupportedFieldAttributeWarning: The 'alias' attrib
ute with value 'glipizide-metformin' was provided to the `Field()` function, which has no effect in the context it was used. 'alias' is f
```

Predict on Docker image

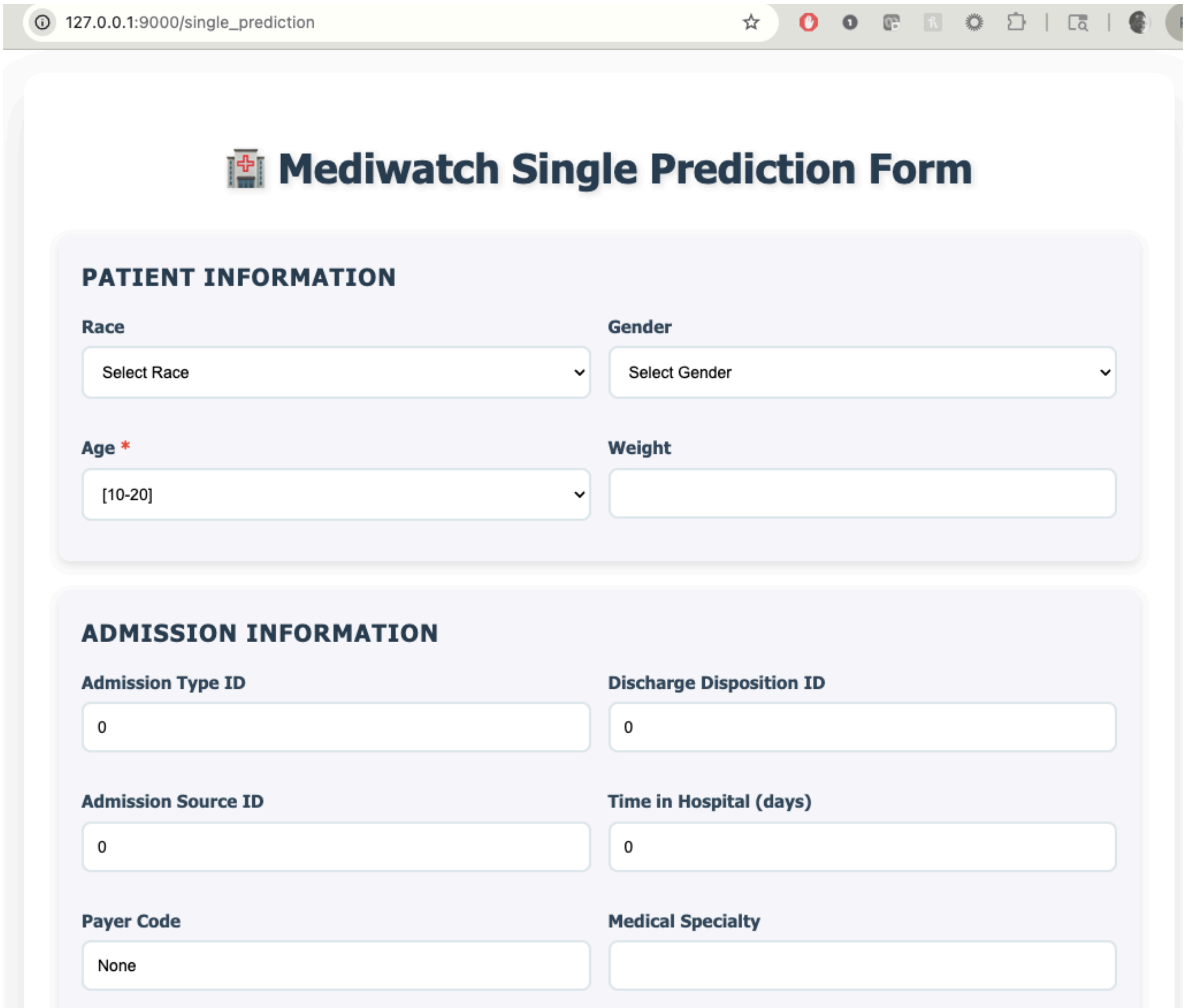
NOTE: Please Don't run Build or Train on the docker image. (I didn't have time to modify the UI to only show the Predict section)


127.0.0.1:9000
Relaunch to update
MediWatch
MLOPS for Machine Learning
Predict
Use the model to make one prediction with user provided data.
Real-time prediction
Make a single prediction
Train
Train the machine learning model with sample data derived from the Mediwatch dataset.
Custom model training
Progress monitoring with MLFlow
Hyperparameter tuning with Ray-tune
Performance metrics
Start Training
Build
Build a Docker image containing the Mediwatch application.
Model Building
Start Building

And then…


Predict
Use the model to make one prediction with user provided data.
Real-time prediction
Make a single prediction

And then…



127.0.0.1:9000/single_prediction

🏥 Mediwatch Single Prediction Form

PATIENT INFORMATION

Race

Select Race

Gender

Select Gender

Age *

[10-20]

Weight

ADMISSION INFORMATION

Admission Type ID

0

Discharge Disposition ID

0

Admission Source ID

0

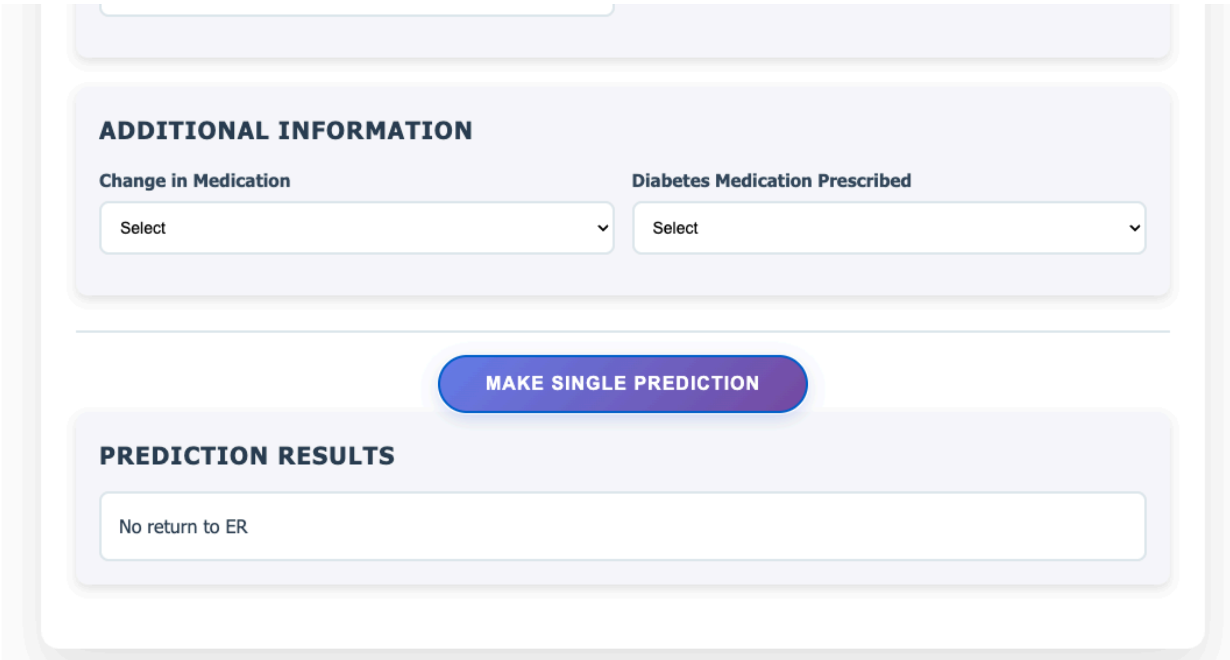
Time in Hospital (days)

0

Payer Code

None

Medical Specialty



Success!

Stop Docker container

Use

```
docker stop <container_id>
```

```
docker rm <container_id>
```

```
no changes added to commit (use "git add" and/or "git commit -a")
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker ps -a
CONTAINER ID   IMAGE                  COMMAND          CREATED         STATUS         PORTS
                NAMES
b1777692ad0d   test_image1759981921   "./startup.sh"   9 minutes ago   Up 9 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp, 0.0.0.0:9
]:9000->9000/tcp   musing_bassi
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker stop b1777692ad0d


b1777692ad0d
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker ps -a
CONTAINER ID   IMAGE                  COMMAND          CREATED          STATUS                      PORTS     NAMES
b1777692ad0d   test_image1759981921   "./startup.sh"   11 minutes ago   Exited (137) 3 seconds ago            musing_bassi
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ sudo docker stop b1777692ad0d
[[sudo] password for victor:
b1777692ad0d
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker ps -a
CONTAINER ID   IMAGE                  COMMAND          CREATED          STATUS                       PORTS     NAMES
b1777692ad0d   test_image1759981921   "./startup.sh"   11 minutes ago   Exited (137) 19 seconds ago            musing_bassi
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker rm b1777692ad0d
b1777692ad0d
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
[(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
```

Kubernetes

Deploy the Docker image on a Kubernetes cluster for effective deployment.

The previous documentation shows building a docker image directly outside minikube

This image needs to be inside minikube in order to work with kubernetes

This can be tagged and copied into minikube as shown below:

And then tag

```
docker tag test_image1760066178:latest test_image1760066178:latest
```

Load this into minikube (takes a LONG while)

```
minikube image load test_image1760066178:latest
```

The problem here is that the copy is inefficient. I literally crashed my Desktop PC 2-3 times when the copy hung my entire OS.

The better way to do this is to build DIRECTLY in minikube. Here is how its done manually …

Start MiniKube

```
minikube status
```

```
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ minikube status
minikube
type: Control Plane
host: Stopped
kubelet: Stopped
apiserver: Stopped
kubeconfig: Stopped
```

Here minikube is OFF, so we need to start it

```
minikube start
```

```
apache/airflow                2.7.1-python3.11   4fb802b917bc   22 months ago   1.51GB
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ minikube start
😄  minikube v1.36.0 on Ubuntu 24.04
🎉  minikube 1.37.0 is available! Download it: https://github.com/kubernetes/minikube/releases/tag/v1.37.0
💡  To disable this notice, run: 'minikube config set WantUpdateNotification false'

✨  Using the docker driver based on existing profile
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.47 ...
🔄  Restarting existing docker container for "minikube" ...
🐳  Preparing Kubernetes v1.33.1 on Docker 28.1.1 ...
🔎  Verifying Kubernetes components...
    ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
    ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡  Some dashboard features require the metrics-server addon. To enable all features please run:

        minikube addons enable metrics-server

🌟  Enabled addons: storage-provisioner, default-storageclass, dashboard
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

And now …

```
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

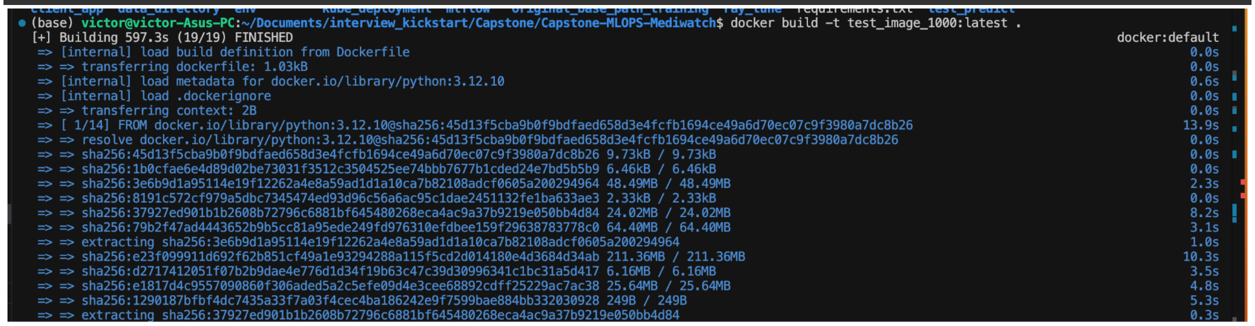
Get into minikube shell and look at the images available

```
apache/airflow                2.7.1-python3.11   4fb802b917bc   22 months ago   1.51GB
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$ eval $(minikube docker-env)
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$ docker images
REPOSITORY                                TAG        IMAGE ID       CREATED         SIZE
registry.k8s.io/kube-scheduler            v1.33.1    398c985c0d95   5 months ago    73.4MB
registry.k8s.io/kube-controller-manager   v1.33.1    ef43894fa110   5 months ago    94.6MB
registry.k8s.io/kube-apiserver            v1.33.1    c6ab243b29f8   5 months ago    102MB
registry.k8s.io/kube-proxy                v1.33.1    b79c189b052c   5 months ago    97.9MB
registry.k8s.io/etcd                      3.5.21-0   499038711c08   6 months ago    153MB
registry.k8s.io/coredns/coredns           v1.12.0    1cf5f116067c   10 months ago   70.1MB
registry.k8s.io/pause                     3.10       873ed7510279   16 months ago   736kB
kubernetesui/dashboard                    <none>     07655ddf2eeb   3 years ago     246MB
kubernetesui/metrics-scraper              <none>     115053965e86   3 years ago     43.8MB
gcr.io/k8s-minikube/storage-provisioner   v5         6e38f40d628d   4 years ago     31.5MB
```

Build in minikube

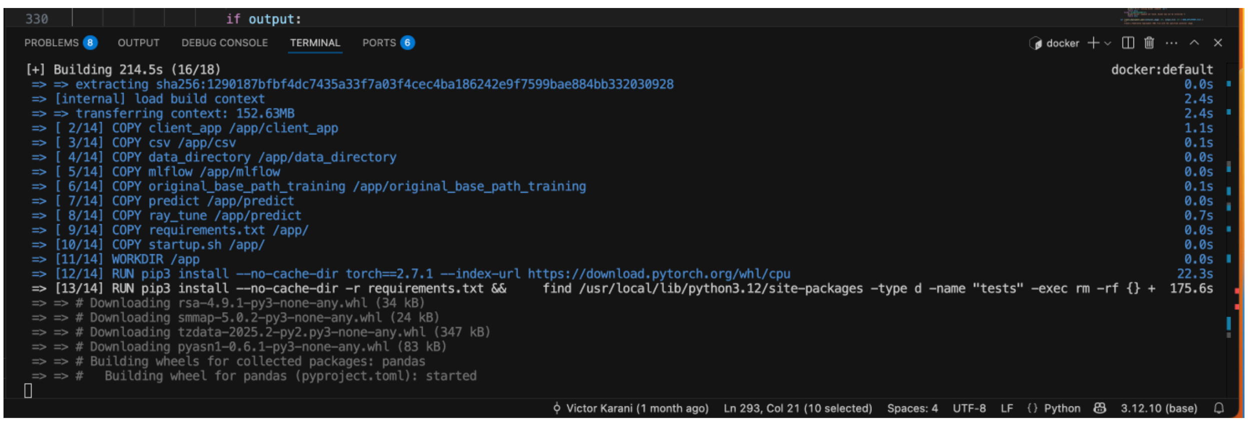
This is using the kube_deployment/Deployment.yaml

```
docker build -t test_image_1000:latest .
```



…

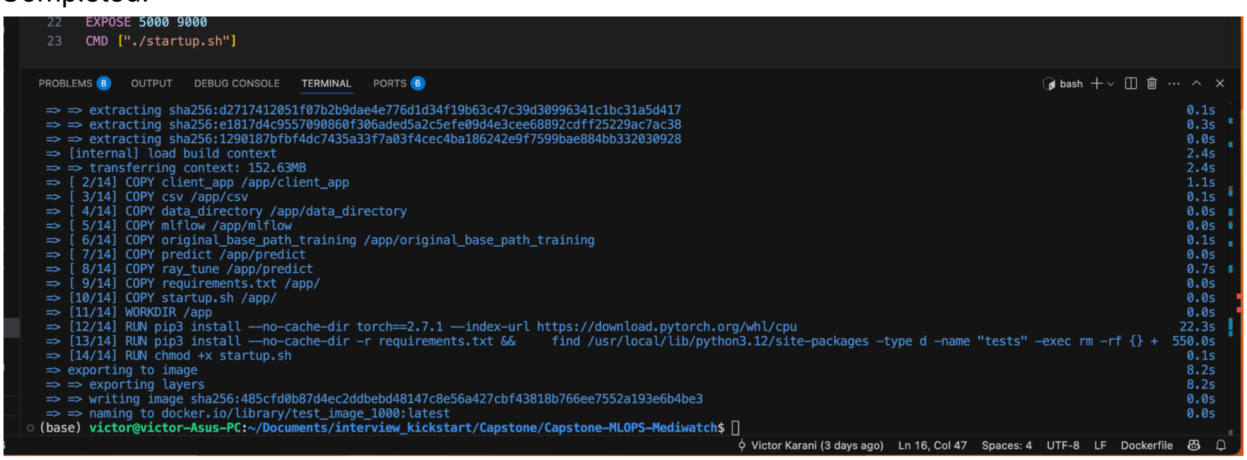
…



…

…

Completed:

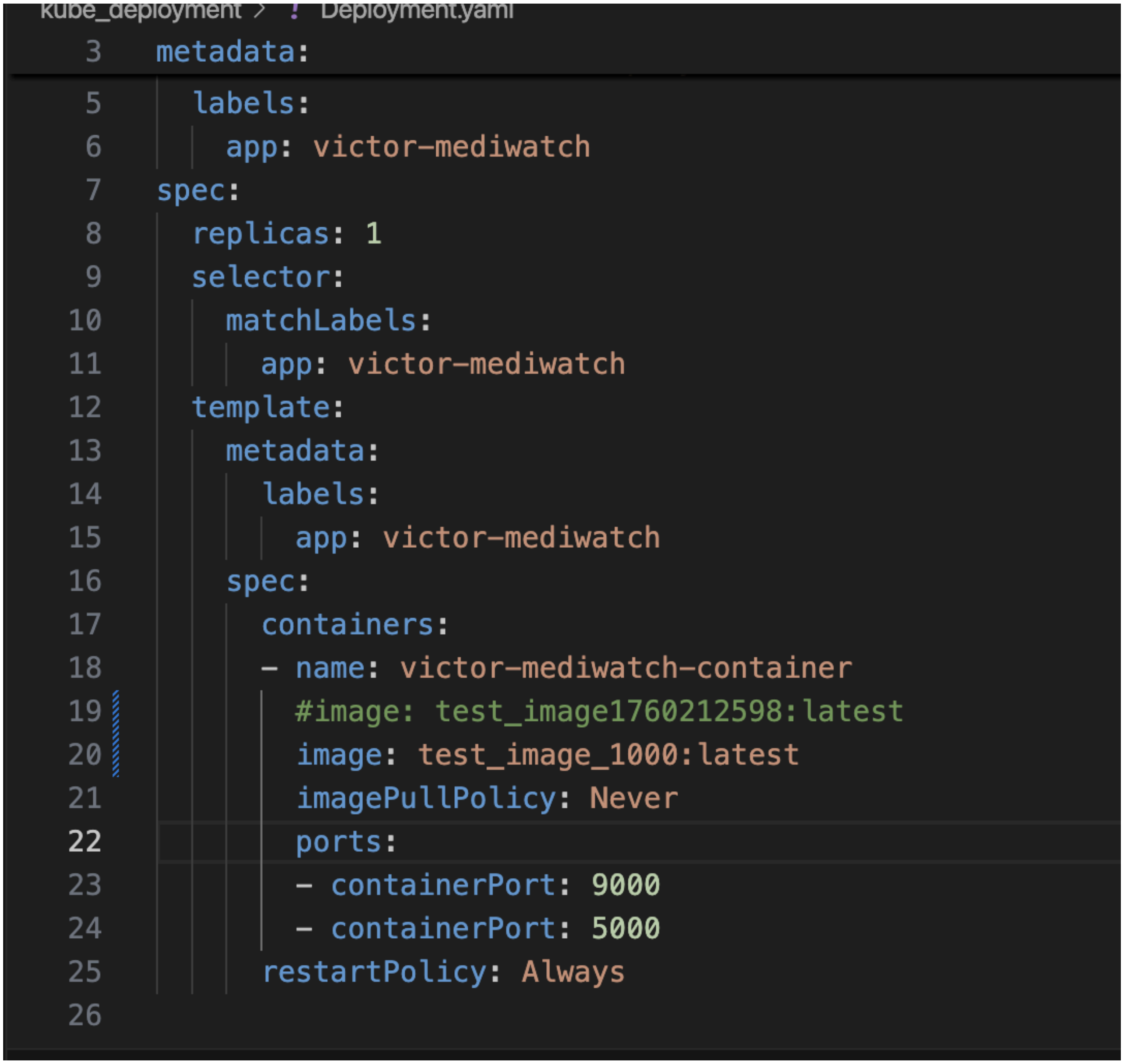


Show built Image in minikube

```
○ (base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
● (base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker images
  REPOSITORY                                TAG        IMAGE ID       CREATED          SIZE
  test_image_1000                           latest     485cfd0b87d4   2 minutes ago    3.77GB
  registry.k8s.io/kube-apiserver            v1.33.1    c6ab243b29f8   5 months ago     102MB
  registry.k8s.io/kube-controller-manager   v1.33.1    ef43894fa110   5 months ago     94.6MB
  registry.k8s.io/kube-scheduler            v1.33.1    398c985c0d95   5 months ago     73.4MB
  registry.k8s.io/kube-proxy                v1.33.1    b79c189b052c   5 months ago     97.9MB
  registry.k8s.io/etcd                      3.5.21-0   499038711c08   6 months ago     153MB
  registry.k8s.io/coredns/coredns           v1.12.0    1cf5f116067c   10 months ago    70.1MB
  registry.k8s.io/pause                     3.10       873ed7510279   16 months ago    736kB
  kubernetesui/dashboard                    <none>     07655ddf2eeb   3 years ago      246MB
  kubernetesui/metrics-scraper              <none>     115053965e86   3 years ago      43.8MB
  gcr.io/k8s-minikube/storage-provisioner   v5         6e38f40d628d   4 years ago      31.5MB
○ (base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
```

Update the Deployment.yaml

With the new image
`test_image_1000:latest`



```yaml
kube_deployment > ! Deployment.yaml
metadata:
  labels:
    app: victor-mediwatch
spec:
  replicas: 1
  selector:
    matchLabels:
      app: victor-mediwatch
  template:
    metadata:
      labels:
        app: victor-mediwatch
    spec:
      containers:
      - name: victor-mediwatch-container
        #image: test_image1760212598:latest
        image: test_image_1000:latest
        imagePullPolicy: Never
        ports:
        - containerPort: 9000
        - containerPort: 5000
      restartPolicy: Always
```

Apply the deployment

```
kubectl apply -f Deployment.yaml
```

```
-rw-rw-r-- 1 victor victor 5235 Oct 11 11:58 test_kubernetes.py
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$ kubectl apply -f Deployment.yaml
deployment.apps/victor-mediwatch-deployment configured
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$ kubectl get pods
NAME                                          READY   STATUS    RESTARTS   AGE
victor-mediwatch-deployment-6985675756-dv28k   1/1     Running   0          6s
```

Apply the Service..

```
victor-mediwatch-deployment-6985675756-dv28k   1/1     Running   0          6s
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$ kubectl  apply -f Service.yaml
service/victor-mediwatch-service unchanged
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$ kubectl get pods
NAME                                           READY   STATUS    RESTARTS   AGE
victor-mediwatch-deployment-6985675756-dv28k   1/1     Running   0          58s
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$
```

Check services and ports…

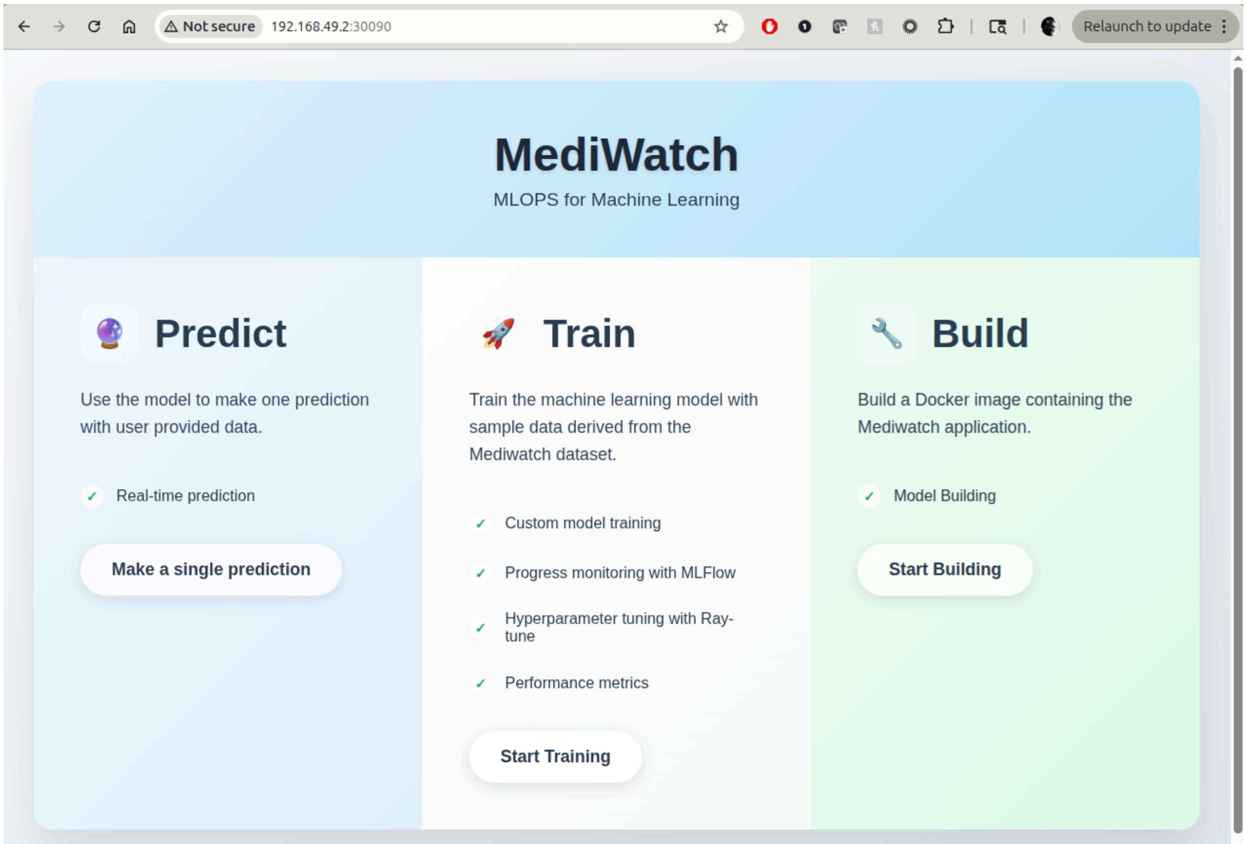
```
kubectl get nodes -o wide
```

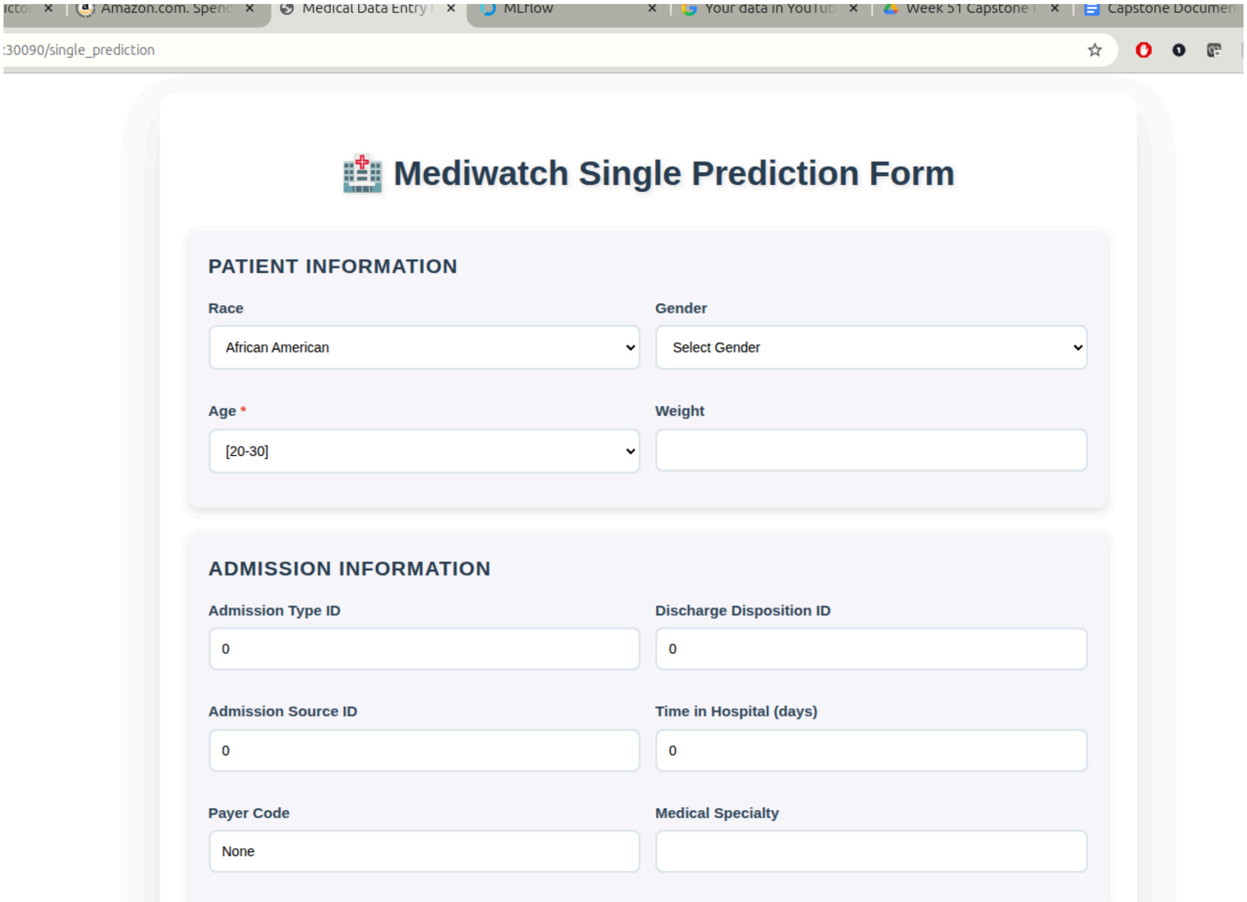
```
victor-mediwatch-deployment-6985675756-dv28k   1/1     Running   0          58s
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$ kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION     CONTAINER-RUNTIME
minikube   Ready    control-plane   3d13h   v1.33.1   192.168.49.2   <none>        Ubuntu 22.04.5 LTS   6.8.0-85-generic   docker://28.1.1
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/kube_deployment$
```

Kubernetes services

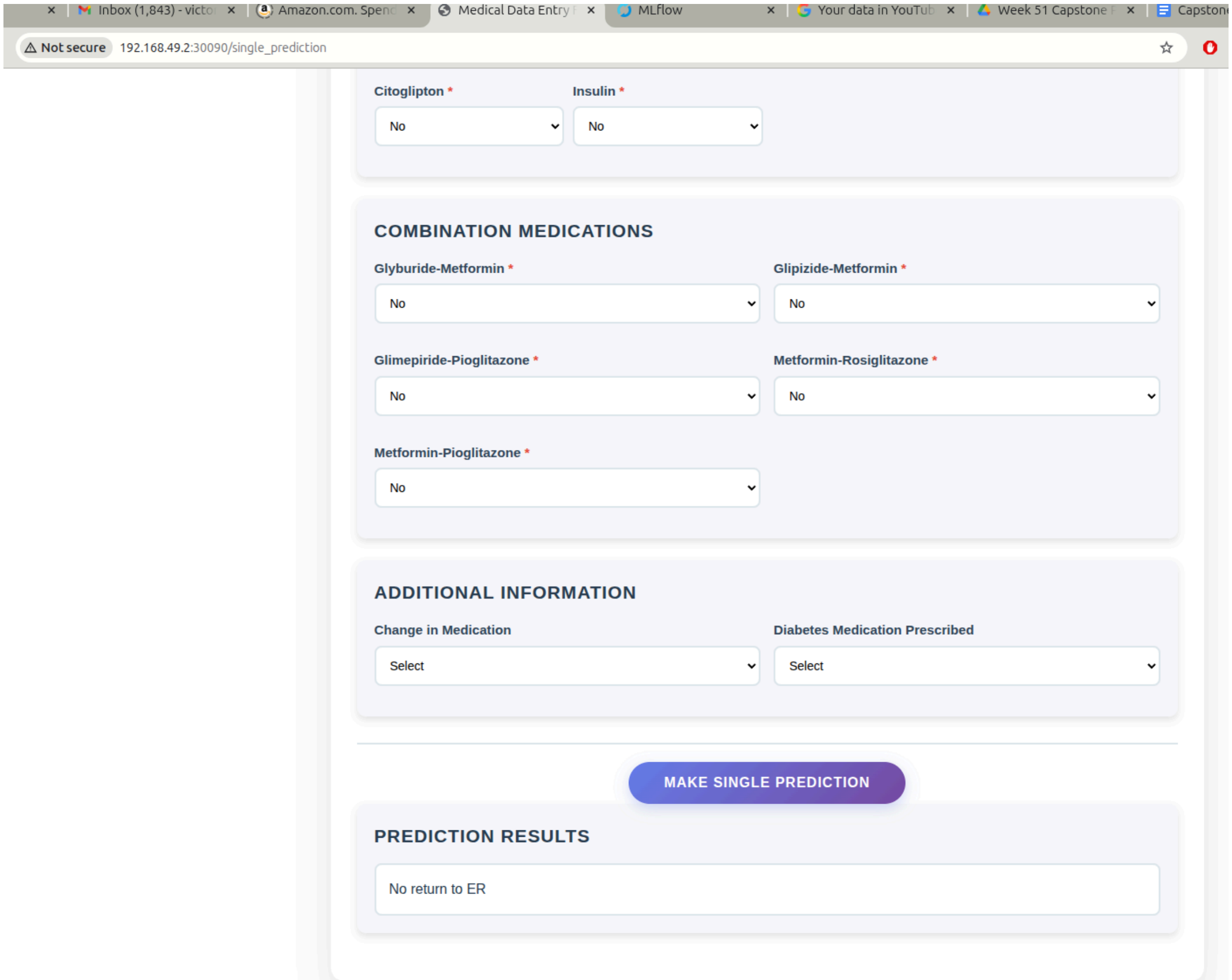
NOTE: Please Don't run Build or Train on the docker image. (I didn't have time to modify the UI to only show the Predict section)

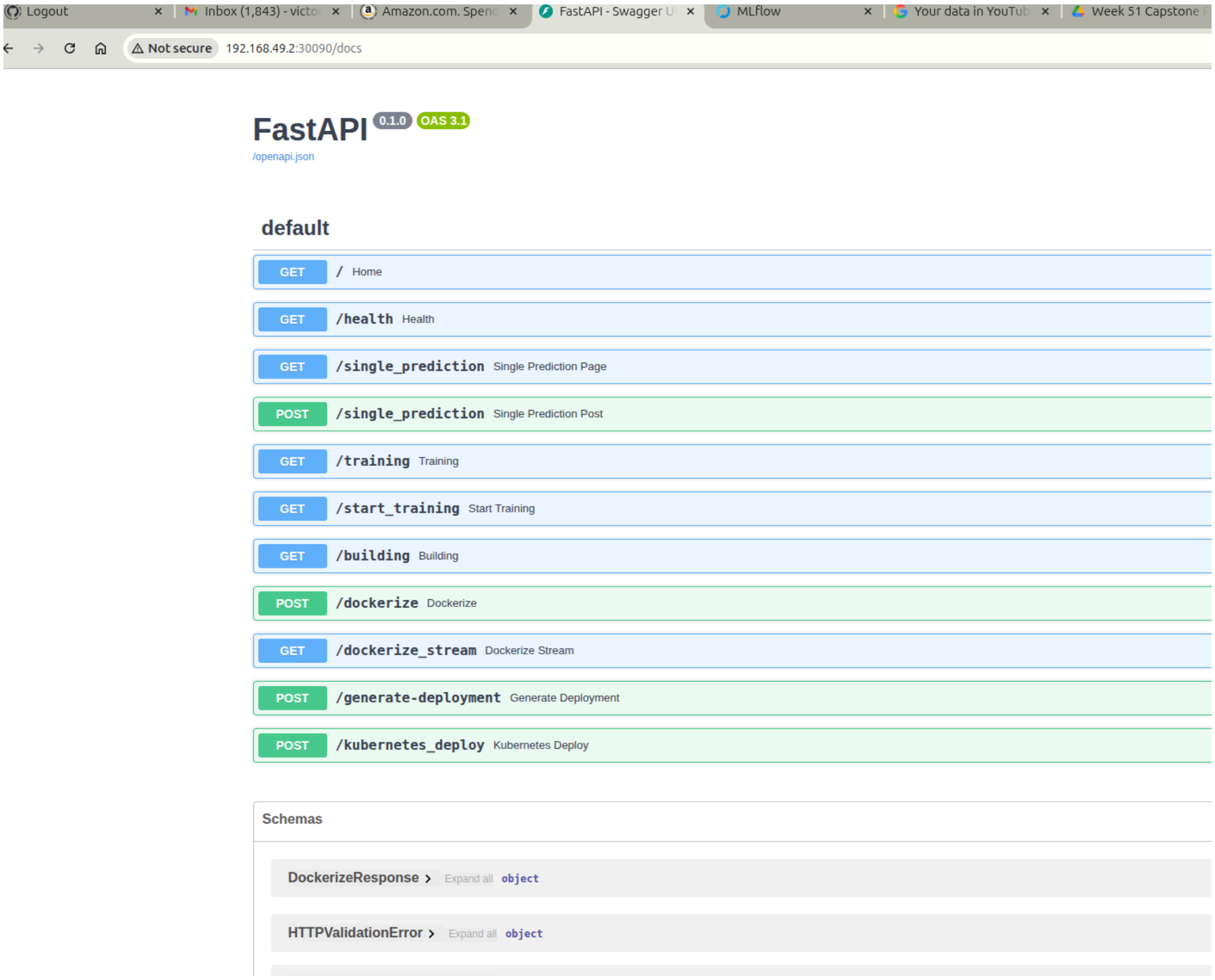
UI in kubernetes at 30090





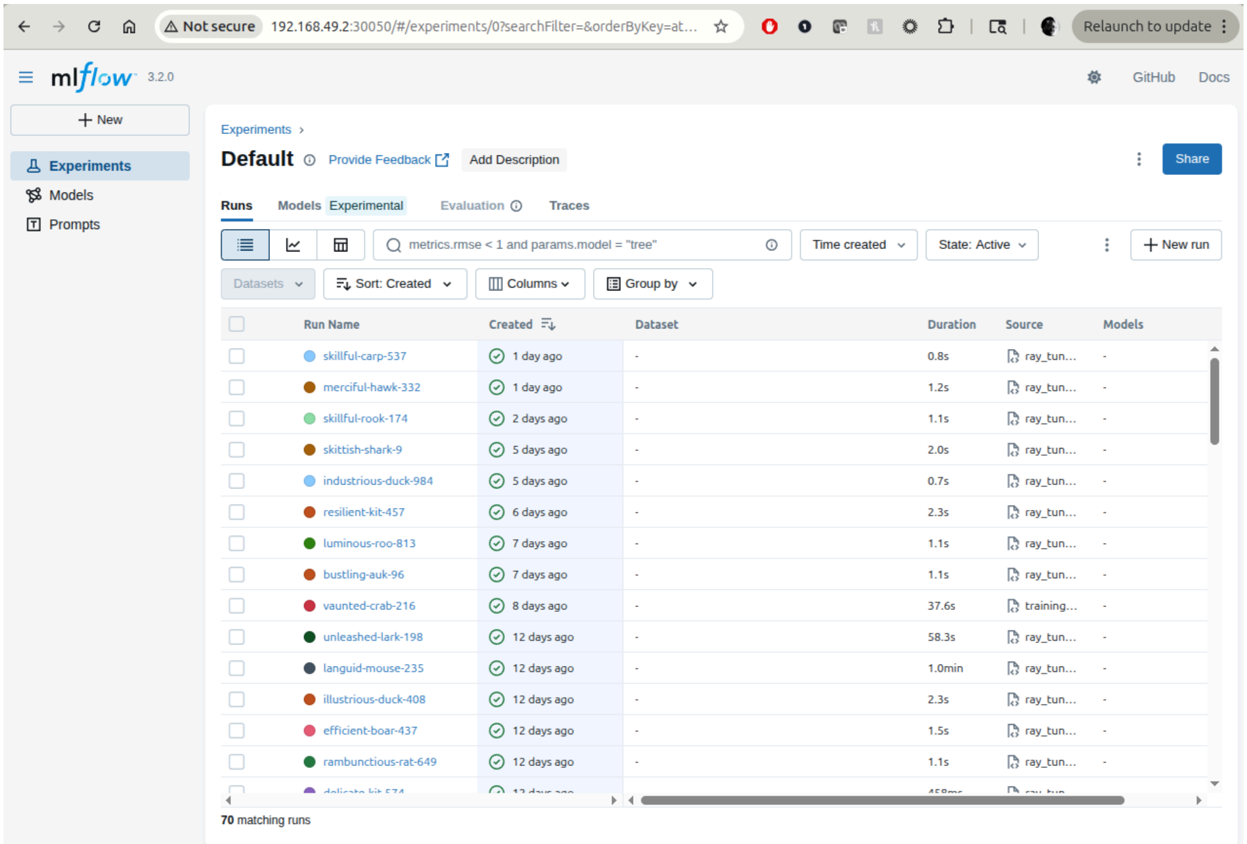
Amazon.com. Spend
Medical Data Entry
MLflow
Your data in YouTub
Week 51 Capstone
Capstone Documen
:30090/single_prediction
Mediwatch Single Prediction Form
PATIENT INFORMATION
Race
African American
Gender
Select Gender
Age *
[20-30]
Weight
ADMISSION INFORMATION
Admission Type ID
0
Discharge Disposition ID
0
Admission Source ID
0
Time in Hospital (days)
0
Payer Code
None
Medical Specialty

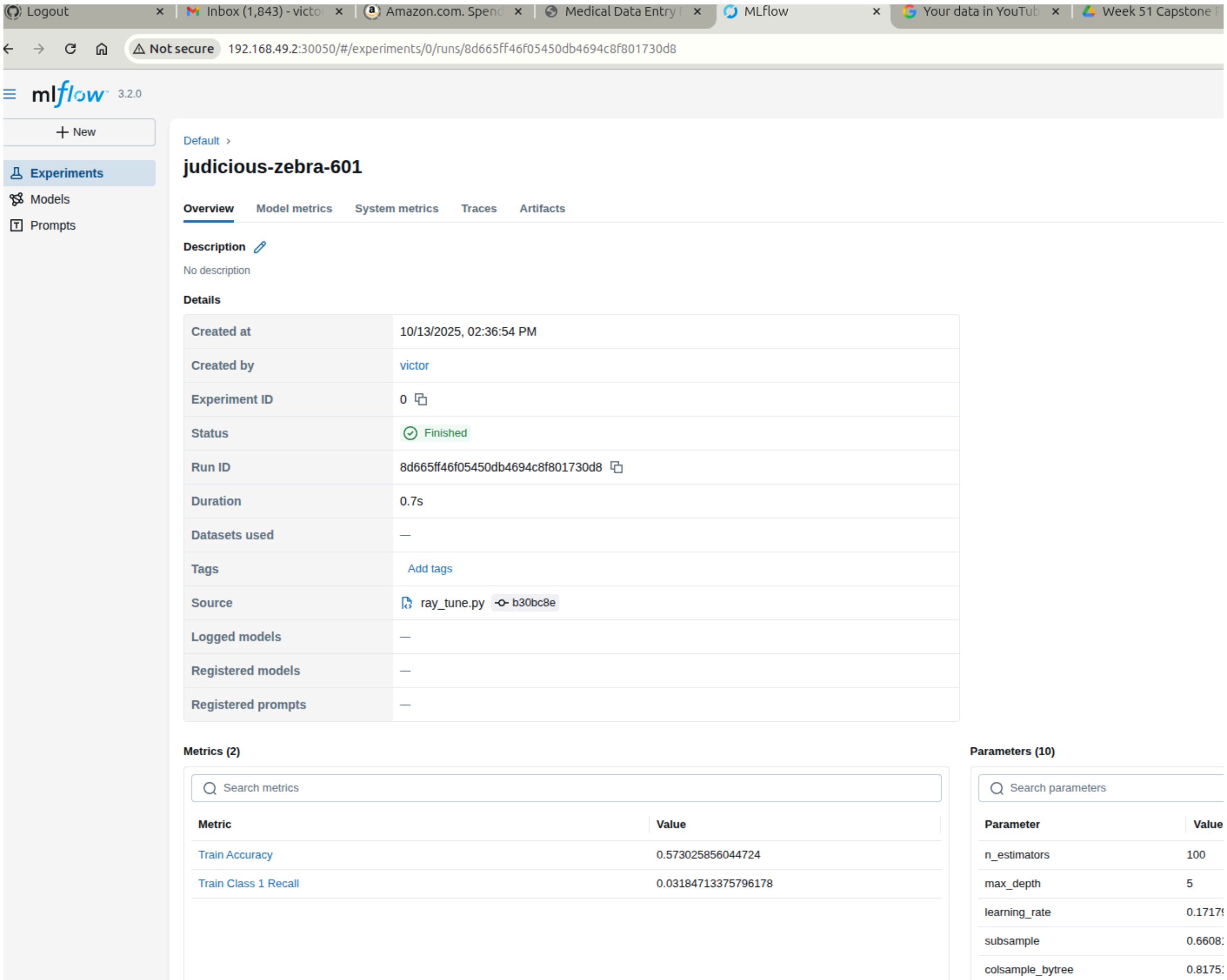

Inbox (1,843) - victo
Amazon.com. Spend
Medical Data Entry
MLflow
Your data in YouTub
Week 51 Capstone
Capstone
Not secure
192.168.49.2:30090/single_prediction
Citoglipton *
No
Insulin *
No
COMBINATION MEDICATIONS
Glyburide-Metformin *
No
Glipizide-Metformin *
No
Glimepiride-Pioglitazone *
No
Metformin-Rosiglitazone *
No
Metformin-Pioglitazone *
No
ADDITIONAL INFORMATION
Change in Medication
Select
Diabetes Medication Prescribed
Select
MAKE SINGLE PREDICTION
PREDICTION RESULTS
No return to ER



Note: training and build UI do not work in kubernetes!!!

Mlflow in kubernetes at 30050

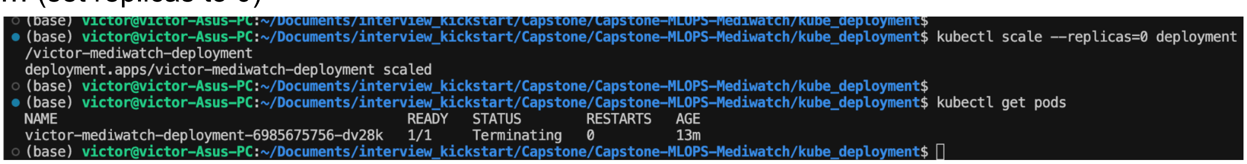




Turn off Kubernetes deployment

And when you are done:

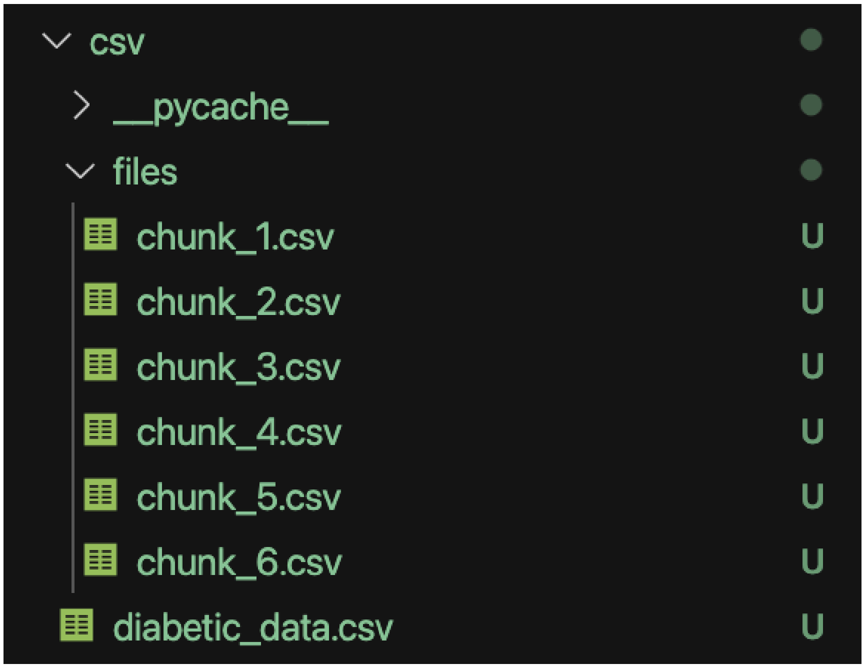
… (set replicas to 0)



Model Monitoring with Evidently

Data Drift Considerations

For this project, we are handed a static data file to use. I saved this locally as ‘diabetic_data.csv’
For a refresher, let us take a look at the csv files directory:



The ‘diabetic_data.csv’ is divided into 6 files: chunk_1.csv … chunk_6.csv. I ONLY train with chunk_1.csv and use ‘chunk_6.csv’ as my new data.

This has 2 implications

1) The model is probably undertrained because we gave it only ⅙ of the potential data it could use
2) Evidently is very likely to find drift when checking model performance between these 2 different sets of data
3) We can then very easily kick off the training_and_docker_build dag to rebuild and update the running instance in kubernetes.

Evidently

As mentioned, I am using evidently to detect drift.
Evidently requires Airflow to be up and running. It also needs the client_app to be running

Config File

Lets take a quick look at the evidently config file located at **evidently/config**:

evidently > config.py > ...

```python
PROJECT_NAME = "Mediwatch Dashboard"

# Evidently Workspace path
WORKSPACE_PATH = "/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/evidently/evidently-data"

# Training data
TRAINING_DATA_FILE = "../csv/files/chunk_1.csv"

# 'New' data
NEW_DATA_FILE= "../csv/files/chunk_5.csv"

# Define threshold for ValueDrift (Jensen-Shannon distance)
# 0.2 is a Common threshold for Jensen-Shannon distance
JS_THRESHOLD=0.1

AIRFLOW_API_URL = 'http://192.168.50.6:8080/api/v1/dags/training_and_docker_build/dagRuns'
AIRFLOW_USERNAME = 'admin'
AIRFLOW_PASSWORD = 'admin123'
```

As mentioned previously, the training file is **chunk_1.csv** and the new data file is **chunk_6.csv**.

Also note the **WORKSPACE_PATH** and **AIRFLOW_API_URL** which need to be set up first.

Evidently Workspace

The evidently workspace is automatically set up in **evidently/evidently-data** . You just need to provide the full path as shown in the config file.

Check for drift

The script **evidently/infer_and_generate_tests_reports.py** will

1) Create and initialize the evidently-data directory if it does not exist and
2) Calculate the drift between the training data and the new data.
3) If it finds drift, it will trigger the **training_and_docker_build** which is described in detail in the Airflow section

This is how to run the script:
First, activate the env

```
conda activate ./env/
```

Then cd to the folder

```
cd evidently/
```

The run the script

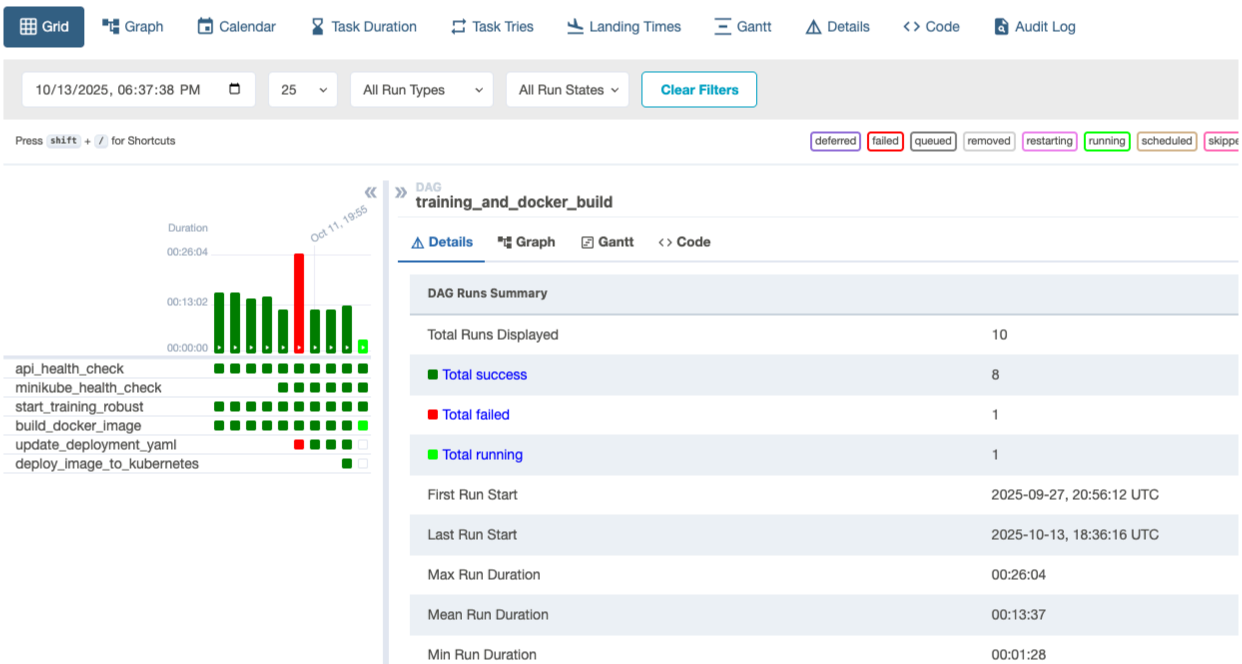
```
python3 infer_and_generate_tests_reports.py
```

Look at the output

```
(/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/env) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/evidently$ python3 infer_and_generate_tests_reports.py
/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/env/lib/python3.10/site-packages/sklearn/preprocessing/_encoders.py:246: UserWarning: Found unknown categories in columns [4, 13, 14, 20, 21] during transform. These unknown categories will be encoded as all zeros
  warnings.warn(
Project: Mediwatch Dashboard already exists. Skip Creation.
Adding Report to project Mediwatch Dashboard

Drift Analysis Results:
Metric ValueDrift(column=readmitted_predictions) has drift and has a value of 0.15238455447137636
WARNING: Data drift detected in the dataset!
DAG has been triggered successfully.
(/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/env) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/evidently$
```

You can see the DAG was triggered

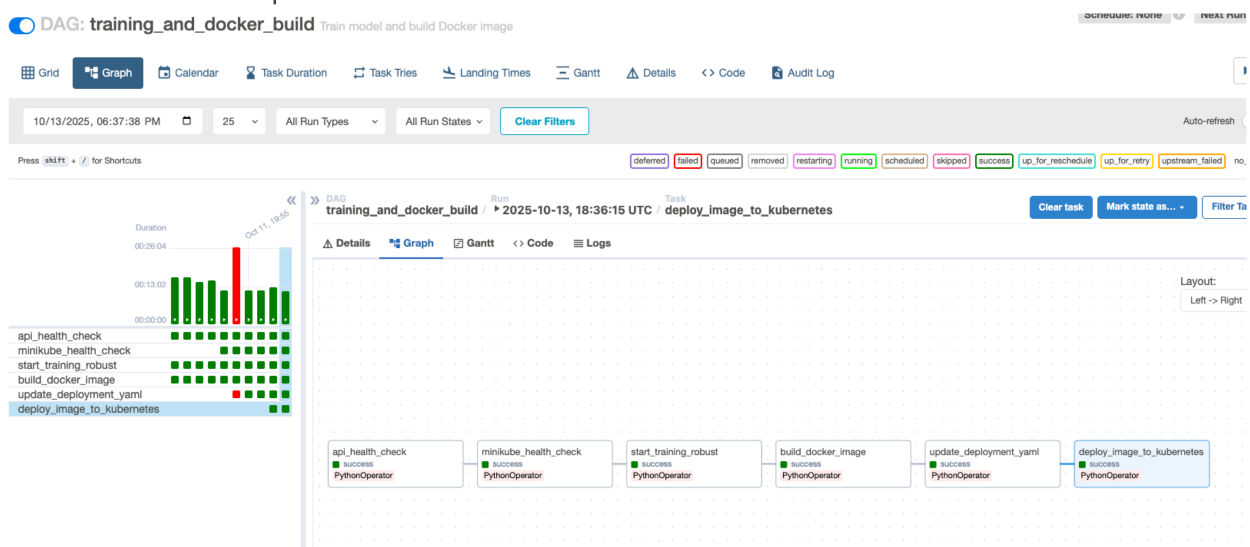
Also, look in airflow, you can see the DAG currently running



For reference, here are my docker images in minikube before the run:

```
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker images
REPOSITORY                                TAG        IMAGE ID       CREATED          SIZE
test_image1760378329                      latest     e0df79810aab   27 minutes ago   3.78GB
test_image1760372126                      latest     a722e9def9b6   2 hours ago      3.77GB
test_image1760369737                      latest     1df71c62037b   3 hours ago      3.77GB
test_image_1000                           latest     485cfd0b87d4   4 hours ago      3.77GB
registry.k8s.io/kube-controller-manager   v1.33.1    ef43894fa110   5 months ago     94.6MB
registry.k8s.io/kube-scheduler            v1.33.1    398c985c0d95   5 months ago     73.4MB
registry.k8s.io/kube-apiserver            v1.33.1    c6ab243b29f8   5 months ago     102MB
registry.k8s.io/kube-proxy                v1.33.1    b79c189b052c   5 months ago     97.9MB
registry.k8s.io/etcd                      3.5.21-0   499038711c08   6 months ago     153MB
registry.k8s.io/coredns/coredns           v1.12.0    1cf5f116067c   10 months ago    70.1MB
registry.k8s.io/pause                     3.10       873ed7510279   16 months ago    736kB
kubernetesui/dashboard                    <none>     07655ddf2eeb   3 years ago      246MB
kubernetesui/metrics-scraper              <none>     115053965e86   3 years ago      43.8MB
gcr.io/k8s-minikube/storage-provisioner   v5         6e38f40d628d   4 years ago      31.5MB
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
```

And after the run completes:



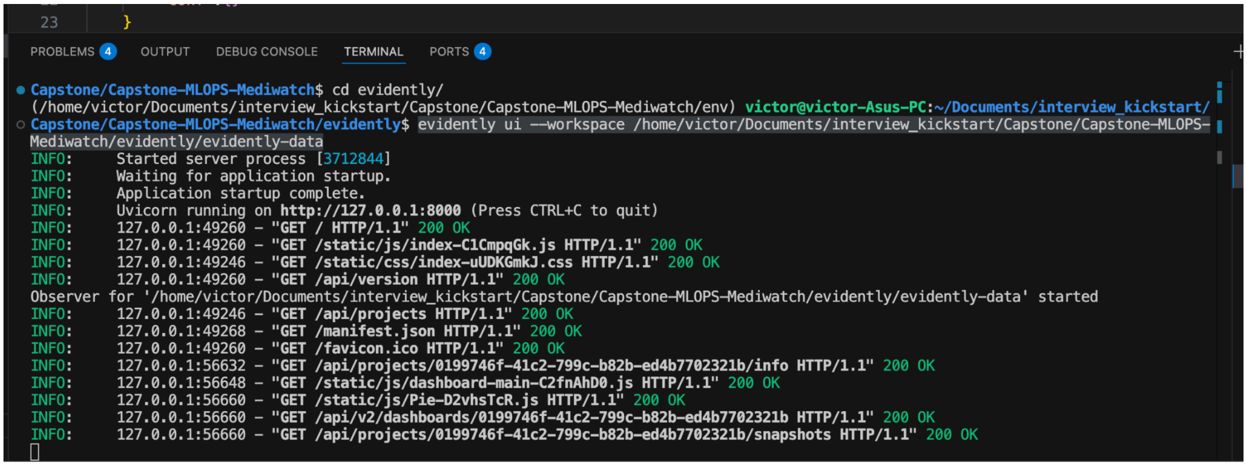
with the new docker image in minikube:

```
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ docker images
REPOSITORY                                TAG        IMAGE ID       CREATED          SIZE
test_image1760380618                      latest     abf284b72488   6 minutes ago    3.77GB
test_image1760378329                      latest     e0df79810aab   44 minutes ago   3.78GB
test_image1760372126                      latest     a722e9def9b6   2 hours ago      3.77GB
test_image1760369737                      latest     1df71c62037b   3 hours ago      3.77GB
test_image_1000                           latest     485cfd0b87d4   4 hours ago      3.77GB
registry.k8s.io/kube-apiserver            v1.33.1    c6ab243b29f8   5 months ago     102MB
registry.k8s.io/kube-controller-manager   v1.33.1    ef43894fa110   5 months ago     94.6MB
registry.k8s.io/kube-scheduler            v1.33.1    398c985c0d95   5 months ago     73.4MB
registry.k8s.io/kube-proxy                v1.33.1    b79c189b052c   5 months ago     97.9MB
registry.k8s.io/etcd                      3.5.21-0   499038711c08   6 months ago     153MB
registry.k8s.io/coredns/coredns           v1.12.0    1cf5f116067c   10 months ago    70.1MB
registry.k8s.io/pause                     3.10       873ed7510279   16 months ago    736kB
kubernetesui/dashboard                    <none>     07655ddf2eeb   3 years ago      246MB
kubernetesui/metrics-scraper              <none>     115053965e86   3 years ago      43.8MB
gcr.io/k8s-minikube/storage-provisioner   v5         6e38f40d628d   4 years ago      31.5MB
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$
```

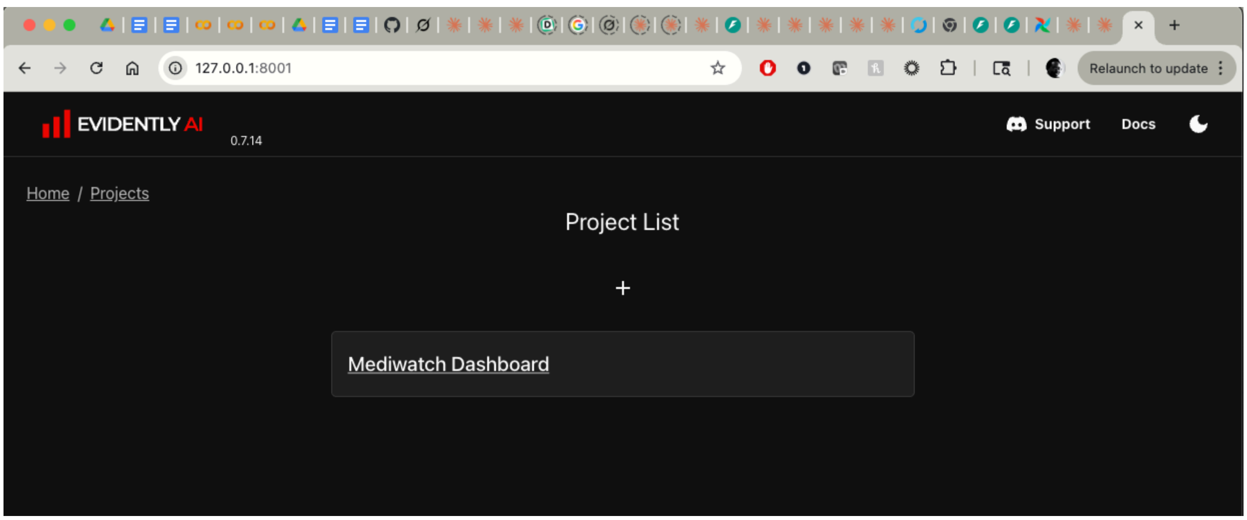
Evidently UI

Now, lets open the evidently UI

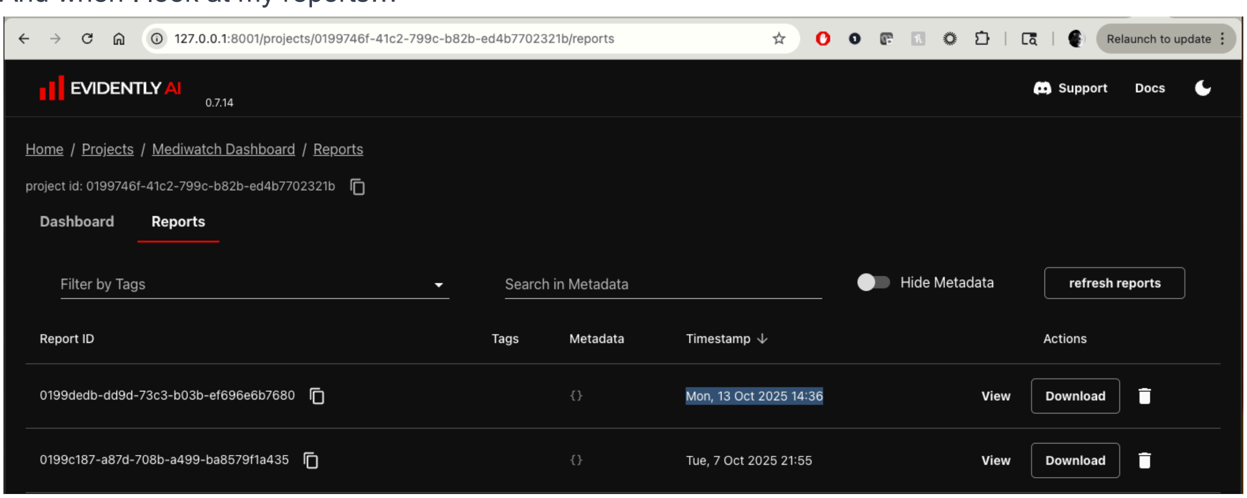
```
evidently ui --workspace
/home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/ev
idently/evidently-data
```



Here is the UI

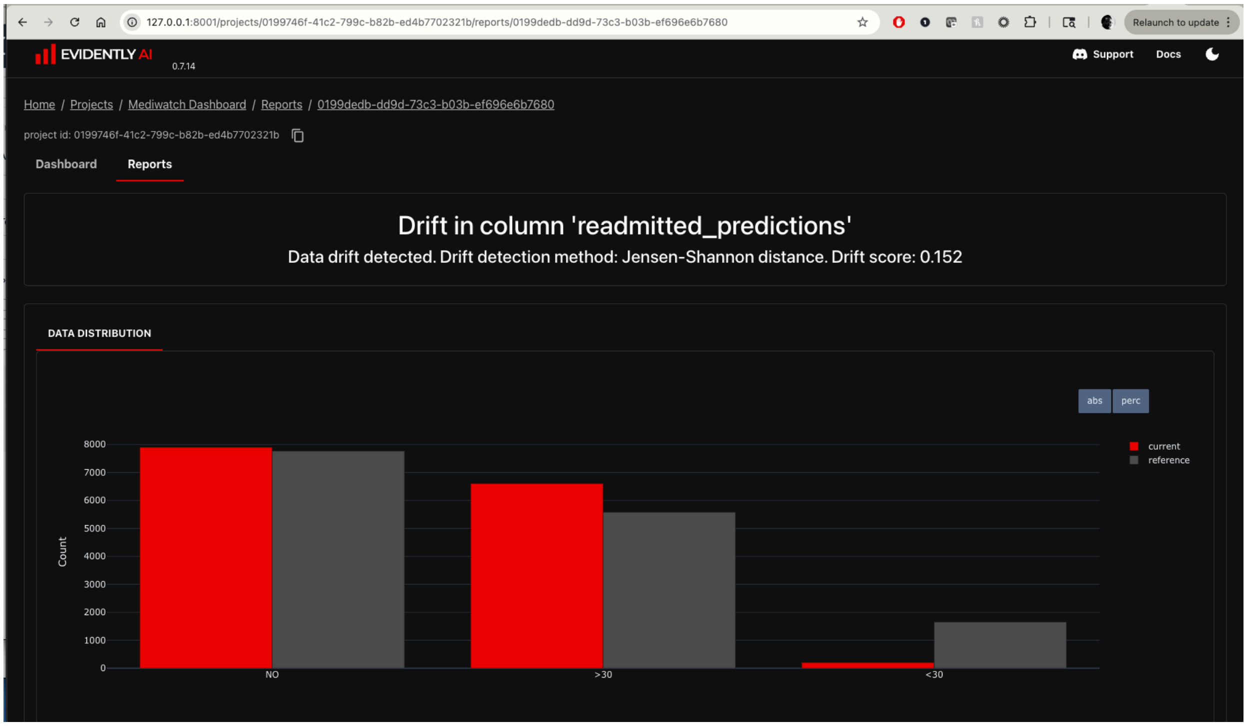


And when I look at my reports…



Let's look at the latest one:

Here is my Data Drift:



Continuous Integration, Deployment and Orchestration

Airflow

I chose to use airflow for my continuous integration and deployment pipeline. In my case look at **airflow/dags/training_api_dag.py**

Right at the bottom are the dags I am using to create the new docker image and deploy to kubernetes

```python
)

#Hit the build docker image endpoint to build the new docker image
build_docker_image = PythonOperator(
    task_id='build_docker_image',
    python_callable=call_dockerize_api_robust,
    dag=dag,
)

update_deployment_yaml = PythonOperator(
    task_id='update_deployment_yaml',
    python_callable=create_deployment_yaml,
    dag=dag
)

deploy_image_to_kubernetes = PythonOperator(
    task_id='deploy_image_to_kubernetes',
    python_callable=deploy_to_kubernetes,
    dag=dag
)


# Set up task dependencies: api_health_check -> kuberneted_health_check -> training -> docker_build -> update_deployment_yaml
api_health_check >> kubernetes_health_check >> start_training_robust >> build_docker_image >> update_deployment_yaml >> deploy_image_to_kubernete
```

Scheduling

The ‘training_and_docker_build’ dag runs the complete pipeline.

The dag can be run on a whatever schedule you desire by modifying the **schedule_interval** below

```python

# Define the combined DAG for training and Docker build
dag = DAG(
    'training_and_docker_build',
    default_args=default_args,
    description='Train model and build Docker image',
    schedule_interval='@daily',
    #schedule_interval=None,
    catchup=False,
    tags=['api', 'training', 'ml', 'docker', 'build'],
)

```

Start Airflow

Use docker compose up -d

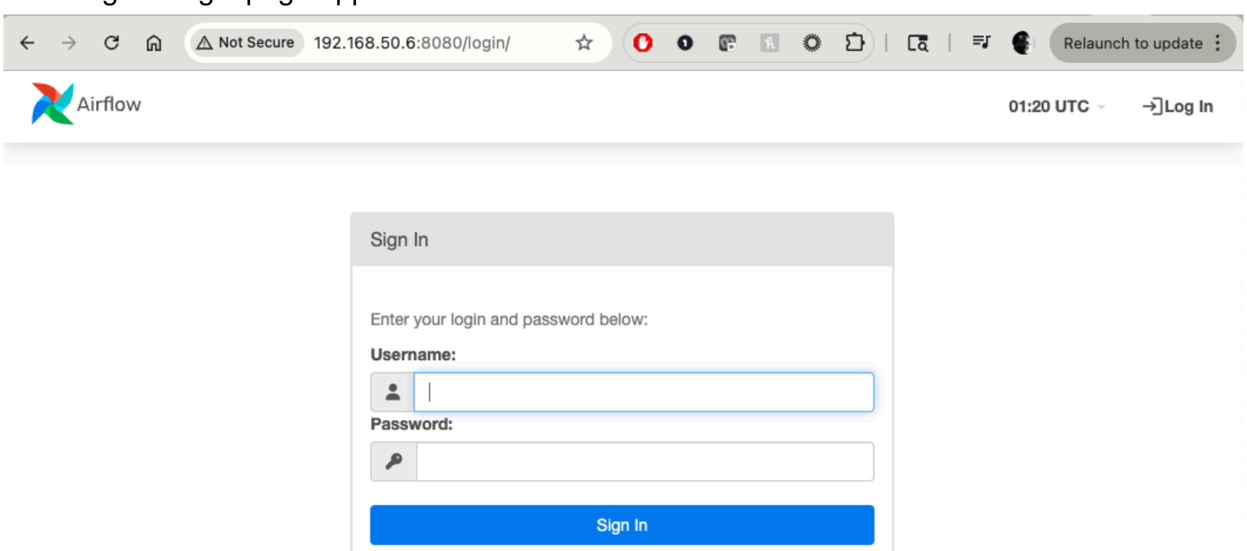
```
conda activate base
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ conda activate base
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ cd airflow/
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/airflow$ docker compose up -d
WARN[0000] /home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/airflow/docker-compose.yaml: the attribute `version` is o
bsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 6/8
 ✔ Network airflow_default               Created                                                          0.0s
 ⠸ Container airflow-redis-1             Waiting                                                          5.5s
 ⠸ Container airflow-postgres-1          Waiting                                                          5.5s
 ✔ Container airflow-init-1              Created                                                          0.1s
 ✔ Container airflow-airflow-scheduler-1 Created                                                          0.0s
 ✔ Container airflow-airflow-webserver-1 Created                                                          0.0s
 ✔ Container airflow-airflow-triggerer-1 Created                                                          0.0s
 ✔ Container airflow-airflow-worker-1    Created                                                          0.0s
```

…

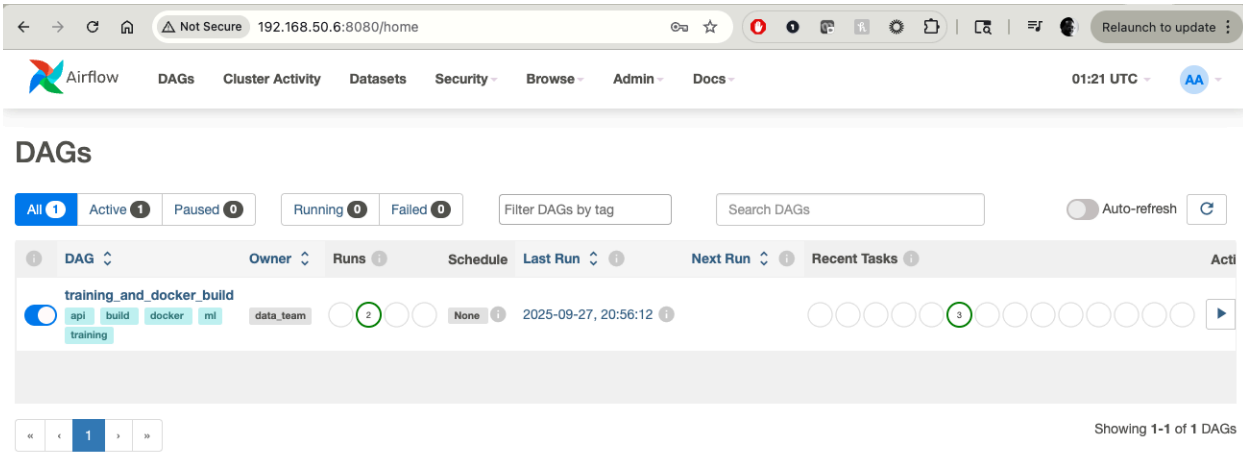
…

```
conda activate base
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ conda activate base
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch$ cd airflow/
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/airflow$ docker compose up -d
WARN[0000] /home/victor/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/airflow/docker-compose.yaml: the attribute `version` is o
bsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 8/8
 ✔ Network airflow_default               Created                                                          0.0s
 ✔ Container airflow-redis-1             Healthy                                                          6.8s
 ✔ Container airflow-postgres-1          Healthy                                                          6.8s
 ✔ Container airflow-airflow-init-1      Exited                                                          18.8s
 ✔ Container airflow-airflow-scheduler-1 Started                                                         19.0s
 ✔ Container airflow-airflow-webserver-1 Started                                                         18.9s
 ✔ Container airflow-airflow-triggerer-1 Started                                                         19.0s
 ✔ Container airflow-airflow-worker-1    Started                                                         18.9s
(base) victor@victor-Asus-PC:~/Documents/interview_kickstart/Capstone/Capstone-MLOPS-Mediwatch/airflow$
```

Note that the ‘exited’ init process above is ok. It does its initial work then goes away. Here is my login page. Note that on my setup, it usually takes ~ 10 s from docker compose up completion to having the login page appear

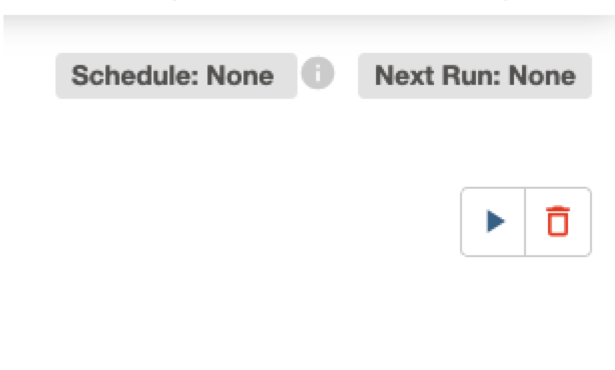


Login as **admin/admin123**



Run the training_and_docker build dag

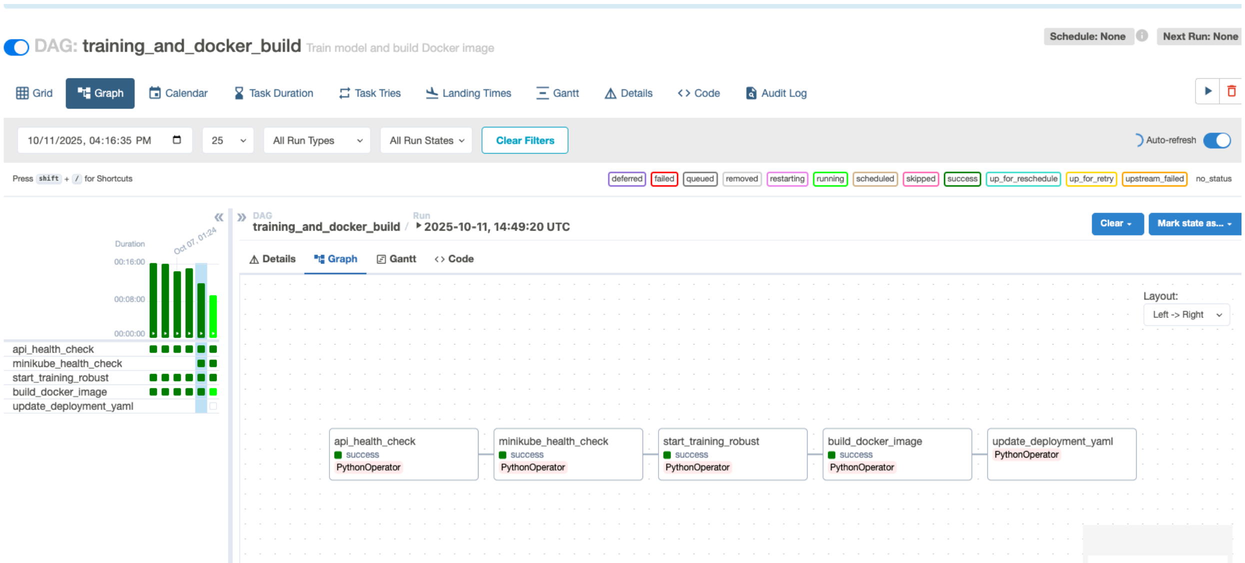
Click on the '**training_and_docker_build**' dag, then click on the run button in the top right



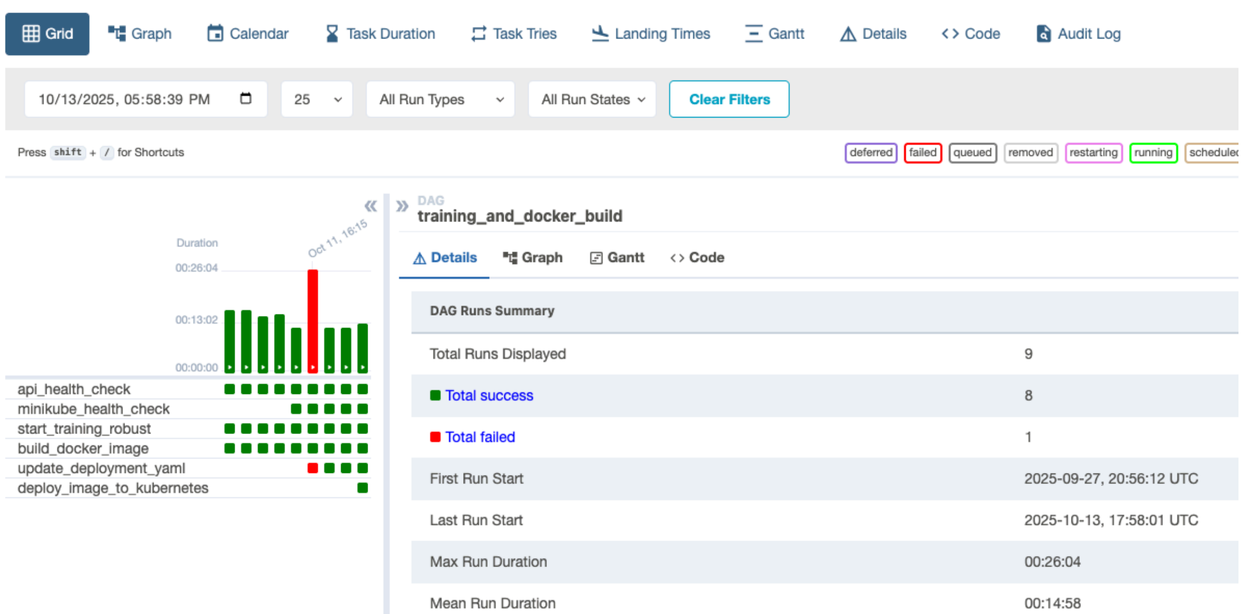
corner

As the dag runs, active items move from bright green (active) to dark green (completed)

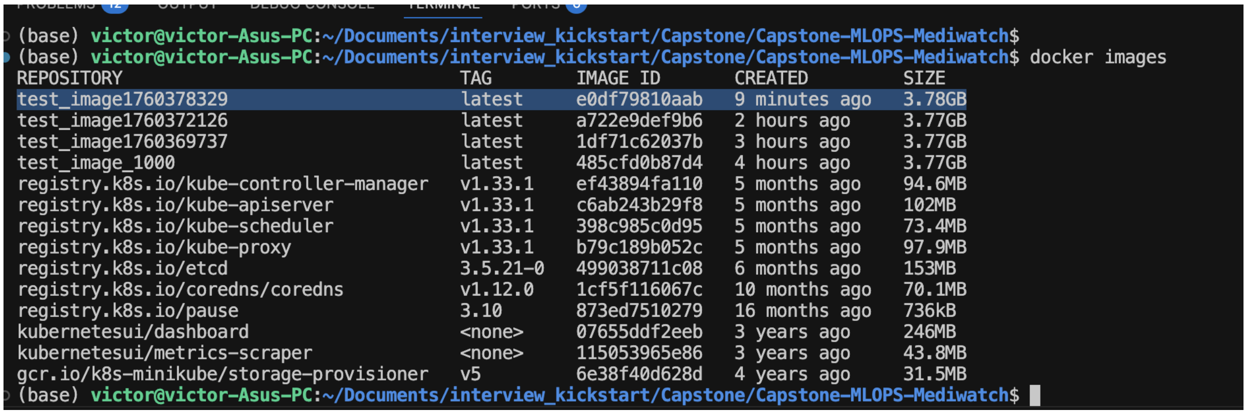
And now build docker image is active



And now it is complete:



And here is the resulting docker image in minikube:



Stop Airflow

Note: you can turn off airflow with docker compose down if you are done with it.